Think & Make 2026-27 — Django Backend Documentation

> **Scope.** This document explains the Django backend in `backend/` of this repository. It was written by reading the source code, settings, migrations and tests (all 54 tests pass as of the time of writing). It covers **only what actually exists**. Where code *refers to* something that does not exist, or where a recommendation is made, it is labelled explicitly:
>
> - **Current implementation** — what the code does today.
> - **Recommended improvement** — a suggestion. It does **not** exist in the code.
> - **Observation** — something noticed while reading the code that a developer should know about (a gap, inconsistency, or likely bug).
>
> Reading level: you know Python reasonably well and are still learning professional Django/DRF architecture.

Table of Contents

1. What is this project?
2. Architecture
3. Project Directory Structure
4. Django settings.py — explained
5. Environment variables
6. Django apps (one section per app)
7. Models and database design
8. Django ORM concepts used here
9. Serializers
10. Views and ViewSets
11. DRF request lifecycle
12. Authentication
13. Permissions and roles
14. SchoolScopedViewSetMixin in depth
15. CORS
16. Throttling
17. Celery
18. Celery Beat schedules
19. Redis
20. File storage
21. URLs and routing
22. Pagination, filtering, searching, ordering
23. Error handling
24. Database migrations
25. Signals and middleware
26. Services, selectors and business logic
27. Repositories
28. Complete feature flows
29. API reference
30. Function Reference
31. Class Reference
32. Relevant concepts to learn
33. How to rebuild this project
34. Development commands
35. Testing
36. Security Considerations
37. Performance considerations
38. Deployment architecture
39. Glossary
40. Recommended Learning Order
41. Backend in One Picture
42. If I had to explain this project in an interview
43. Questions I should be able to answer

1. What is this project?

Think & Make is a school innovation program run by Inqui-Lab (referred to in code as "IIF") with funding partners such as "Datla". Before this backend existed, the program ran on **Google Forms + Google Sheets + Google Drive**: every field visit, kit handover, student-leader assessment and headcount was a Google Form response in a sheet, logins were a shared "Users" sheet with plaintext PINs, and progress tabs were "auto-filled" by hand.

This backend replaces that with:

- A **relational database** (SQLite in local dev, PostgreSQL on Supabase in staging/production).
- A **REST API** built with **Django 5.2 + Django REST Framework (DRF) 3.18**, served under `/api/v1/`, consumed by **two clients** from the same repo: `web/` (Next.js) and `mobile/` (React Native / Expo).
- **JWT authentication** with three roles: `admin`, `iif_staff`, `school`.
- **Server-side data scoping** so a school account can only ever read/write its own school's data.
- **Celery + Redis** for background work: an AI job (Google Gemini) that generates bilingual (English + Telugu) feedback questions for student teams ("InquiBuddy"), and a nightly job that recomputes a school-progress dashboard.
- **File storage** on local disk (dev) or **Supabase Storage** via the S3-compatible API (staging/prod).
- **PDF generation** with WeasyPrint for InquiBuddy feedback reports.
- A **Django admin** themed with `django-unfold`.
- **Management commands** that import the legacy spreadsheet and the legacy user list.

The program's "forms" and how they map to the code

The web/mobile clients present a sequence of forms. The backend names them like this:

Form in the UI	Legacy Google Sheet tab	Backend model(s)	Endpoint(s)
Form 1 — School Enrollment	`School_Enrollment`	`School`, `SchoolGradeEnrollment`	`POST /api/v1/schools/{id}/submit-form1/`
Form 2 — Schools Contact Info	`Schools_Contact_Info`	`School.iif_poc`, `SchoolTeacher`, `SessionSchedule`	`POST /api/v1/schools/{id}/submit-form2/`
Form 3 — Students Count	`Students_Count_Info`	`StudentHeadcount`	`/api/v1/student-headcounts/`
Form 4 — SL (Student Leader) Selection	`SL_Selection_Assessment`	`SLSelection`	`POST /api/v1/sl-selections/bulk-submit/`
Kit handover	`Kits_Info`	`KitDelivery`	`/api/v1/kit-deliveries/`
Session observations	`L1/L2/L3 Session Tracker`, `Session_Progress`	`SessionObservation`	`/api/v1/session-observations/`, `/session-tracker/`, `/session-progress/`
InquiBuddy (AI feedback)	—	`InquibuddySubmission`, `SubmissionFile`, `AIFeedbackQuestion`, `FeedbackReport`	`/api/v1/inquibuddy-submissions/...`
Progress dashboard	—	`SchoolProgress` (derived cache)	`/api/v1/reports/summary/`, `/api/v1/schools/{id}/progress/`

("Form 3" and "Form 4" are the names used in `apps/reports/services.py` and `SchoolProgress.form3_pct/form4_pct`.)

Why is it structured this way?

- **One Django app per "entity family"** (schools, kits, headcounts, ...). Each legacy sheet tab became roughly one app. This keeps each app small and makes it obvious where a feature lives.
- **Shared security logic lives in one place** (`apps/core/permissions.py`, `apps/core/mixins.py`) so every "school form" endpoint enforces the same rule: *a school sees only its own rows*.
- **Derived data is computed, not stored by hand.** The old sheet had manually maintained pivot tabs. Here, trackers are plain queries (`observations/selectors.py`) and the dashboard is a re-computable cache (`reports/services.py`).
- **Slow work runs in the background.** The Gemini call takes 1–3 minutes, so the API returns immediately and a Celery worker does the work.
- **Integrations are isolated in `lib/`.** Gemini and PDF code sit outside the Django apps so they can be swapped or mocked easily.

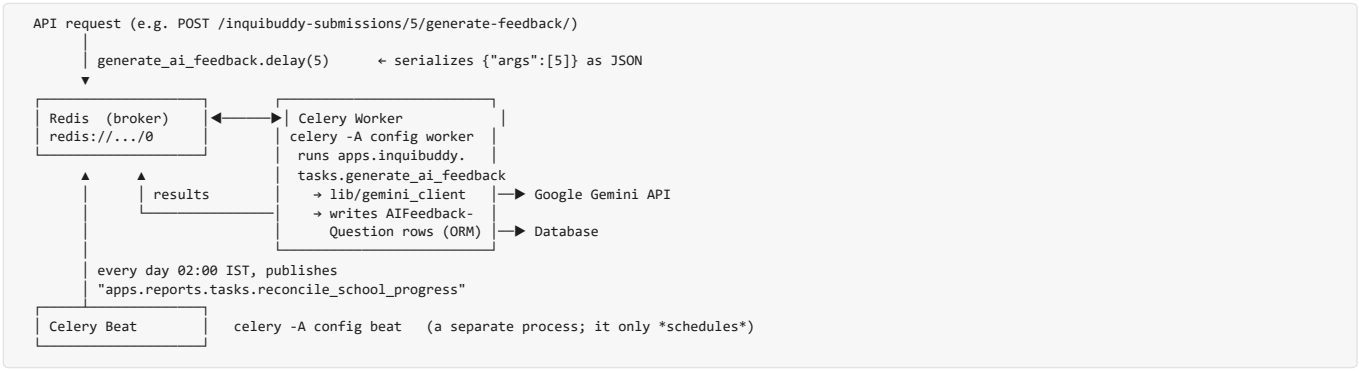
2. Architecture

2.1 The request path (synchronous)



There is **no repository layer** in this project (see §27). Business logic lives in views for most apps, and in `services.py` / `selectors.py` for `reports`, `inquibuddy` and `observations`.

2.2 The background path (asynchronous)



2.3 Where each piece fits

Component	Where configured	Role in this project
Middleware	`MIDDLEWARE` in `config/settings.py`	Django built-ins + `corsheaders`. No custom middleware exists.
CORS	`CORS_ALLOWED_ORIGINS`, `CORS_ALLOW_CREDENTIALS`	Allows the Next.js site (default `http://localhost:3000`) to call the API from a browser. The mobile app is a native HTTP client and is not subject to CORS.
Django Admin	`config/urls.py` (`/admin/`), each app's `admin.py`, `UNFOLD` setting	Back-office CRUD for every model, themed with `django-unfold`. Uses session auth, not JWT.
Redis	`CELERY_BROKER_URL`, `CELERY_RESULT_BACKEND`	Message broker + result store for Celery. Not used as a Django cache.
Celery Worker	`config/celery.py`, `apps/*/tasks.py`	Runs `generate_ai_feedback` and `reconcile_school_progress`.
Celery Beat	`CELERY_BEAT_SCHEDULE`	Triggers `reconcile_school_progress` nightly at 02:00 Asia/Kolkata.
Supabase Storage	`USE_S3_STORAGE` + `SUPABASE_STORAGE_*`	When enabled, `django-storages` (`S3Boto3Storage`) stores every `FileField` in a Supabase bucket via its S3-compatible API and hands out 10-minute signed URLs.
PostgreSQL	`DATABASE_URL` (parsed by `dj-database-url`)	Production DB (Supabase Postgres). Falls back to SQLite `backend/db.sqlite3`.
Gemini	`GEMINI_API_KEY`, `lib/gemini_client.py`	Generates 5 bilingual reflection questions from an idea photo (+ optional audio).
WeasyPrint	`lib/pdf_builder.py`, templates in `apps/inquibuddy/templates/`	HTML → PDF for InquiBuddy feedback reports (needs Pango/GTK native libraries).

3. Project Directory Structure

3.1 Repository root (for context)

```
t and m app and website trail/
├── backend/        ← Django API (this document)
├── web/            ← Next.js website (API client)
├── mobile/         ← React Native / Expo app (API client)
├── docs/           ← Source material: legacy xlsx exports, user-flow manuals (PDF), architecture .docx
└── README.md
```

3.2 `backend/` (actual tree; `.venv/`, `__pycache__/`, `.pytest_cache/` omitted)

```
backend/
├── manage.py                    Django CLI entry point
├── requirements.txt             Pinned Python dependencies
├── pytest.ini                   pytest-django configuration
├── conftest.py                  Shared pytest fixtures (schools, users, API clients)
├── README.md                    Setup + caveats (WeasyPrint on Windows, Celery)
├── .env.example                 Template for environment variables (committed)
├── .env                         Local secrets (git-ignored, NOT committed)
├── .gitignore
├── db.sqlite3                   Local dev database (git-ignored)
├── media/                       Local uploads when USE_S3_STORAGE=False (git-ignored)
│
├── config/                      Django *project* package
│   ├── __init__.py              Imports the Celery app so it loads with Django
│   ├── settings.py              All configuration
│   ├── urls.py                  Root URLconf: /admin/ and /api/v1/
│   ├── celery.py                Celery application object
│   ├── wsgi.py                  WSGI entry point (sync servers: gunicorn, runserver)
│   └── asgi.py                  ASGI entry point (async servers; unused features-wise)
│
├── lib/                         Non-Django integrations
│   ├── gemini_client.py         Google Gemini call for InquiBuddy
│   └── pdf_builder.py           WeasyPrint HTML→PDF rendering
│
└── apps/                        All Django *apps* (Python package)
    ├── core/                    Shared base model, RBAC, school-scoping mixin, audit log, admin theme, import commands
    │   ├── models.py            TimeStampedModel (abstract), AuditLog
    │   ├── permissions.py       Role constants, permission classes, scope_queryset_to_role()
    │   ├── mixins.py            SchoolScopedViewSetMixin
    │   ├── audit.py             record_audit()
    │   ├── admin.py             AuditLog admin (read-only)
    │   ├── static/admin/        theme.css, force-light.js (unfold admin customisation)
    │   ├── management/commands/  import_legacy_sheets.py, import_master_users.py
    │   ├── migrations/0001_initial.py
    │   ├── views.py, tests.py   Empty Django-generated stubs
    │   └── apps.py
    ├── accounts/                Custom User model, login/refresh/me, admin-only user management
    ├── programs/                Partner, ProgramInstance
    ├── schools/                 School + Form 1 / Form 2 submission, grade enrollments, teachers, schedules
    ├── kits/                    KitDelivery
    ├── sl_selection/            SLSelection + bulk submit (Form 4)
    ├── headcounts/              StudentHeadcount (Form 3) + instant mismatch warning
    ├── teams/                   Cluster → Team → Student
    ├── observations/            SessionObservation + tracker/progress selectors
    ├── inquibuddy/              AI feedback flow: submissions, uploads, Celery task, PDFs, signal
    │   ├── services.py, tasks.py, signals.py
    │   └── templates/inquibuddy/ feedback_report.html, feedback_report_all_teams.html,
    │                            _report_page.html, _report_style.html
    ├── files/                   Generic File model + upload/retrieve endpoint
    └── reports/                 SchoolProgress cache, reconciliation service, nightly task, summary API
        └── management/commands/reconcile_school_progress.py
```

A typical app folder contains the standard Django files:

File	Django-specific?	Purpose
`apps.py`	Yes	`AppConfig` — the app's registration with Django (name `apps.<app>`, `BigAutoField` ids).
`models.py`	Yes	Database tables as Python classes.
`migrations/`	Yes	Versioned schema changes generated from `models.py`.
`admin.py`	Yes	Registers models with the Django admin (all use `unfold.admin.ModelAdmin`).
`serializers.py`	DRF	Converts models ⇄ JSON and validates input.
`views.py`	DRF	ViewSets / APIViews: the HTTP endpoints.
`urls.py`	Yes	Routes for this app (mostly a DRF `DefaultRouter`).
`tests.py`	Yes (pytest-django here)	Tests. Several apps still have only the empty generated stub.
`services.py` / `selectors.py`	Project convention	Business logic (write side) / read-only queries. Only in some apps.
`tasks.py`	Celery convention	Background tasks (auto-discovered by Celery).
`signals.py`	Project convention	Signal receivers; imported in `AppConfig.ready()`.

3.3 Folder-by-folder explanation

`config/` — the Django project

- **Why it exists:** `django-admin startproject config .` creates a package holding settings and the root URLconf. Naming it `config` (instead of the project name) is a common convention that makes it obvious it holds configuration.
- **Responsibility:** global settings, root routing, WSGI/ASGI entry points, and the Celery app.
- **Interacts with:** everything. `manage.py`, `wsgi.py`, `asgi.py` and `celery.py` all set `DJANGO_SETTINGS_MODULE=config.settings`. `urls.py` includes each app's `urls.py`.
- **Django-specific:** yes (except `celery.py`).

`apps/` — the project's own Django apps

- **Why it exists:** grouping local apps under one package keeps them separate from `config/` and `lib/`. Because of this, each app's `AppConfig.name` is `"apps.<name>"` and `INSTALLED_APPS` lists `"apps.schools"`, etc.
- **Responsibility:** all domain logic.
- **Project-specific.** (`apps/__init__.py` is empty; it just makes `apps` a package.)

`apps/core/` — shared foundations

- **Why:** code that every other app needs but that belongs to no single entity: the abstract `TimeStampedModel`, role constants, permission classes, school-scoping logic, the audit log and the data-import commands.
- **Interacts with:** imported by nearly every app. It must not import from other apps at module level (it doesn't, except the management commands which import models from many apps — fine, since commands load after all apps are ready).

`apps/<entity>/` — feature apps

Each feature app owns its models and endpoints. Dependencies flow "downwards":

```
programs  ←  schools  ←  kits, sl_selection, headcounts, observations, teams, reports
                         teams ← inquibuddy
              files  ← used by schools, kits, headcounts, inquibuddy (FK "files.File")
    accounts ↔ schools    (User.school FK → schools.School; migration 0002 handles the cycle)
    reports  ↔ schools    (schools/views.py imports reports; reports/models.py imports schools)
    headcounts → reports.services (instant mismatch warning)
```

`lib/` — external integrations

- **Why:** Gemini and WeasyPrint aren't Django concerns. Keeping them in plain Python modules means views/tasks call a simple function (`generate_feedback_questions(...)`, `render_team_feedback_pdf(...)`) and tests can mock that one function.
- **Project-specific**, not a Django app (not in `INSTALLED_APPS`, no models).

`apps/inquibuddy/templates/inquibuddy/` — PDF templates

- Django templates (found through `APP_DIRS: True`) rendered to HTML and converted to PDF. `_report_page.html` is one team's page; `feedback_report.html` wraps one page; `feedback_report_all_teams.html` loops over many.

`apps/core/static/admin/` — admin theme overrides

- `theme.css` (loaded through `UNFOLD["STYLES"]`) and `force-light.js` (through `UNFOLD["SCRIPTS"]`). The JS removes a stale `adminTheme` key from `localStorage` so unfold's Alpine.js "persist" plugin doesn't crash (explained in the file's comment).

`management/commands/` folders

- Django discovers any module in `<app>/management/commands/` as a `python manage.py <name>` command. This project has three: `import_legacy_sheets`, `import_master_users` (in `core`), and `reconcile_school_progress` (in `reports`).

`media/`

- Created at runtime when `USE_S3_STORAGE=False` (`MEDIA_ROOT = BASE_DIR / "media"`). Git-ignored.

---

4. Django settings.py — explained

File: `backend/config/settings.py`. Configuration values are read with **python-decouple** (`from decouple import Csv, config`) — see §5.

4.1 Core settings

Setting	Value in this project	What it means / why	If removed	Dev vs pr
`BASE_DIR`	`Path(__file__).resolve().parent.parent` → `backend/`	Anchor for building file paths (SQLite path, `MEDIA_ROOT`).	Paths that use it break.	Same.
`SECRET_KEY`	`config("DJANGO_SECRET_KEY", default="dev-only-insecure-secret-key")`	Signs sessions, CSRF tokens, password-reset tokens and — because SimpleJWT defaults `SIGNING_KEY` to `SECRET_KEY` — **every JWT**.	Django refuses to start.	**Must** be secret in default va could for
`DEBUG`	`config("DJANGO_DEBUG", default=True, cast=bool)`	Detailed error pages, serves media (see `urls.py`), relaxed host checks.	Defaults to `False` in Django.	**Default** — produ `DJANGO_`
`ALLOWED_HOSTS`	`Csv()` of `DJANGO_ALLOWED_HOSTS`, default `localhost,127.0.0.1`	Host-header allow-list; prevents host-header attacks.	With `DEBUG=False`, every request 400s.	Set to the domain ir
`INSTALLED_APPS`	see 4.2	Apps whose models, admin, templates, static, commands Django loads.	Missing apps' models/migrations disappear.	Same.
`MIDDLEWARE`	see 4.3	Request/response hooks.	Features (CORS, sessions, CSRF) stop.	Same.
`ROOT_URLCONF`	`"config.urls"`	Module holding `urlpatterns`.	Django can't route.	Same.
`TEMPLATES`	DjangoTemplates, `APP_DIRS: True`, default context processors	Needed by the admin and by the InquiBuddy PDF templates (`apps/inquibuddy/templates`).	Admin & PDFs break.	Same.
`WSGI_APPLICATION`	`"config.wsgi.application"`	What `runserver`/gunicorn load.	`runserver` fails.	Same.
ASGI	`config/asgi.py` exists, no `ASGI_APPLICATION` setting	Only needed for ASGI servers (uvicorn/daphne). No async views exist.	Nothing.	—
`DATABASES`	`dj_database_url.config(default=f"sqlite:///{BASE_DIR/'db.sqlite3'}", conn_max_age=600)`	Reads `DATABASE_URL` env var (e.g. `postgres://user:pw@host:5432/db`) and turns it into Django's dict. `conn_max_age=600` keeps DB connections open up to 10 minutes (persistent connections).	No DB.	SQLite in Postgres `DATABAS` `psycopg` installed
`AUTH_USER_MODEL`	`"accounts.User"`	Tells Django to use the custom user model. Must be set **before the first migration**.	Django would use `auth.User`; existing FKs break.	Same.
`AUTH_PASSWORD_VALIDATORS`	Django's 4 default validators	Used by `createsuperuser`, admin password forms, and anything calling `validate_password()`. **Not** called by the API's `UserCreateSerializer` (see §36).	No password-strength checks in admin.	Same.
`LANGUAGE_CODE`	`"en-us"`	Default language for Django messages.	Defaults to `en-us`.	—
`TIME_ZONE`	`"Asia/Kolkata"`	Local time zone for display and for Celery (`CELERY_TIMEZONE = TIME_ZONE`).	Defaults to `America/Chicago`.	—
`USE_I18N`	`True`	Translation machinery on.	—	—
`USE_TZ`	`True`	Datetimes are stored in UTC and are timezone-aware (`timezone.now()` is used in `schools/views.py`).	Naive datetimes.	—
`STATIC_URL`	`"static/"`	URL prefix for static files (admin CSS/JS).	Admin assets 404.	**Observat** `STATIC_` defined, `collect` cannot ru productic deployme
`DEFAULT_AUTO_FIELD`	`"django.db.models.BigAutoField"`	Primary keys are 64-bit integers.	Falls back to `AutoField` + warnings.	—

4.2 `INSTALLED_APPS`

```
"unfold"                      ← must come BEFORE django.contrib.admin so its templates override the admin's
django.contrib.admin / auth / contenttypes / sessions / messages / staticfiles
"rest_framework"              ← DRF
"rest_framework_simplejwt"    ← JWT auth (views/serializers)
"corsheaders"                 ← CORS headers
"django_filters"              ← ?field=value filtering
"apps.core", "apps.accounts", "apps.programs", "apps.schools", "apps.kits",
"apps.sl_selection", "apps.headcounts", "apps.teams", "apps.observations",
"apps.inquibuddy", "apps.files", "apps.reports"
```

Observation: `rest_framework_simplejwt.token_blacklist` is **not** installed even though `SIMPLE_JWT["BLACKLIST_AFTER_ROTATION"] = True`. SimpleJWT only blacklists tokens when that app is installed, so in practice old refresh tokens remain valid until they expire (see §12).

4.3 `MIDDLEWARE` (order matters)

#	Middleware	What it does here
1	`SecurityMiddleware`	Security headers/redirects controlled by `SECURE_*` settings (none are set, so defaults apply).
2	`corsheaders.middleware.CorsMiddleware`	Adds `Access-Control-Allow-*` headers and answers CORS preflight `OPTIONS` requests. Must be placed before `CommonMiddleware` (it is).
3	`SessionMiddleware`	Cookie sessions — used by the Django admin only.
4	`CommonMiddleware`	URL normalisation (`APPEND_SLASH`: a request to `/api/v1/schools` is redirected to `/api/v1/schools/` for GET).
5	`CsrfViewMiddleware`	CSRF protection for cookie-authenticated forms (admin). DRF views are CSRF-exempt at the Django level; DRF enforces CSRF only for `SessionAuthentication`, which this API doesn't use.
6	`AuthenticationMiddleware`	Sets `request.user` from the session (admin). For API requests DRF replaces this with the JWT user.
7	`MessageMiddleware`	Flash messages in the admin.
8	`XFrameOptionsMiddleware`	`X-Frame-Options: DENY` (clickjacking protection).

There is **no custom middleware** in the project.

4.4 `REST_FRAMEWORK`

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": ("rest_framework_simplejwt.authentication.JWTAuthentication",),
    "DEFAULT_PERMISSION_CLASSES": ("rest_framework.permissions.IsAuthenticated",),
    "DEFAULT_FILTER_BACKENDS": ("django_filters.rest_framework.DjangoFilterBackend",),
    "DEFAULT_THROTTLE_CLASSES": ("rest_framework.throttling.ScopedRateThrottle",),
    "DEFAULT_THROTTLE_RATES": {"login": "10/min"},
    "DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 50,
}
```

Key	Meaning
`DEFAULT_AUTHENTICATION_CLASSES`	Every DRF view reads `Authorization: Bearer <access token>`. No session or basic auth for the API.
`DEFAULT_PERMISSION_CLASSES`	Secure-by-default: any view that doesn't set `permission_classes` requires login. (`LoginView` / `TokenRefreshView` from SimpleJWT override this with empty `permission_classes`, so they are open.)
`DEFAULT_FILTER_BACKENDS`	Enables `filterset_fields` on every view.
`DEFAULT_THROTTLE_CLASSES` / `RATES`	Only views that define `throttle_scope` are throttled; only `LoginView` does (`"login"` → 10 requests/minute).
`DEFAULT_PAGINATION_CLASS` / `PAGE_SIZE`	Every list endpoint of a generic view/viewset returns `{count, next, previous, results}` with 50 items per page. Plain `APIView`s are not paginated.
(not set) `EXCEPTION_HANDLER`	DRF's default handler is used (see §23).

4.5 `SIMPLE_JWT`

Key	Value	Meaning
`ACCESS_TOKEN_LIFETIME`	15 minutes	Short-lived token sent with each request.
`REFRESH_TOKEN_LIFETIME`	7 days	Used only at `/auth/refresh/` to get new access tokens.
`ROTATE_REFRESH_TOKENS`	`True`	`/auth/refresh/` returns a **new** refresh token as well.
`BLACKLIST_AFTER_ROTATION`	`True`	Intended to invalidate the old refresh token — **ineffective here** because the `token_blacklist` app isn't installed.
(defaults)	`ALGORITHM="HS256"`, `SIGNING_KEY=SECRET_KEY`, `AUTH_HEADER_TYPES=("Bearer",)`, `USER_ID_CLAIM="user_id"`	Library defaults.

4.6 CORS

```
CORS_ALLOWED_ORIGINS = config("CORS_ALLOWED_ORIGINS", default="http://localhost:3000", cast=Csv())
CORS_ALLOW_CREDENTIALS = True
```

Explained in §15. (The current uncommitted working-tree diff to `settings.py` only adds two blank lines around this block.)

4.7 Celery

```
CELERY_BROKER_URL      = config("CELERY_BROKER_URL",      default="redis://localhost:6379/0")
CELERY_RESULT_BACKEND  = config("CELERY_RESULT_BACKEND",  default="redis://localhost:6379/0")
CELERY_ACCEPT_CONTENT  = ["json"]
CELERY_TASK_SERIALIZER = "json"
CELERY_RESULT_SERIALIZER = "json"
CELERY_TIMEZONE        = TIME_ZONE      # "Asia/Kolkata"
CELERY_BEAT_SCHEDULE   = {"reconcile-school-progress-nightly": {
    "task": "apps.reports.tasks.reconcile_school_progress",
    "schedule": crontab(hour=2, minute=0)}}
```

Explained in §17–§19.

4.8 Storage and media

```
USE_S3_STORAGE = config("USE_S3_STORAGE", default=False, cast=bool)
if USE_S3_STORAGE:
    STORAGES = {"default": {"BACKEND": "storages.backends.s3boto3.S3Boto3Storage"},
                "staticfiles": {"BACKEND": "django.contrib.staticfiles.storage.StaticFilesStorage"}}
    AWS_ACCESS_KEY_ID = config("SUPABASE_STORAGE_ACCESS_KEY_ID")          # required (no default)
    AWS_SECRET_ACCESS_KEY = config("SUPABASE_STORAGE_SECRET_ACCESS_KEY")  # required (no default)
    AWS_STORAGE_BUCKET_NAME = config("SUPABASE_STORAGE_BUCKET", default="think-and-make")
    AWS_S3_ENDPOINT_URL = config("SUPABASE_STORAGE_ENDPOINT_URL", default="")
    AWS_S3_REGION_NAME = config("SUPABASE_STORAGE_REGION", default="ap-south-1")
    AWS_DEFAULT_ACL = None
    AWS_QUERYSTRING_AUTH = True
    AWS_QUERYSTRING_EXPIRE = 600
    AWS_S3_ADDRESSING_STYLE = "path"
else:
    MEDIA_URL = "/media/"
    MEDIA_ROOT = BASE_DIR / "media"
```

Explained in §20.

4.9 Other settings

Setting	Meaning
`GEMINI_API_KEY`	Read by `lib/gemini_client._client()`. Empty default → Gemini calls fail (the Celery task retries, then errors).
`UNFOLD`	Admin theme: titles, forced light theme, brand colour palette (`primary` 50–950 as RGB triples), extra CSS/JS, sidebar search. Uses `django.templatetags.static.static` in lambdas so the URL is resolved per request.

4.10 Settings that are **not** present

These are commonly expected; they are **not configured** in this project (Django defaults apply):

- **Email** (`EMAIL_BACKEND`, etc.) — no email is sent anywhere.
- **Logging** (`LOGGING`) — modules use `logging.getLogger(__name__)` / Celery's task logger, so output goes to Django's default console handlers.
- **Caching** (`CACHES`) — defaults to per-process `LocMemCache`. This matters for throttling (§16).
- **Security hardening** — `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `SECURE_HSTS_SECONDS`, `CSRF_TRUSTED_ORIGINS`, `SECURE_PROXY_SSL_HEADER`.
- **`STATIC_ROOT`** — needed for `collectstatic` in production.

Recommended improvement: add a production settings section (or env-driven toggles) for the above, and run `python manage.py check --deploy`.

---

5. Environment variables

5.1 How configuration is loaded

The project uses **python-decouple**:

```
from decouple import Csv, config
DEBUG = config("DJANGO_DEBUG", default=True, cast=bool)
```

`config(name, default=..., cast=...)` looks for `name` in this order: 1. The real OS environment (`os.environ`). 2. A `.env` file (decouple searches from the calling module's directory upward; here `backend/.env`). 3. `default` — if there's no default and the variable is missing, it raises `UndefinedValueError` at startup.

`cast=bool` converts strings like `"True"`, `"false"`, `"1"`, `"0"`; `cast=Csv()` splits `"a,b,c"` into `["a", "b", "c"]`.

`DATABASE_URL` is the exception: it is read by **dj-database-url** directly from `os.environ` (`dj_database_url.config()`), **not** by decouple. Because of that, a `DATABASE_URL` line written only in `.env` is **not** picked up unless something exports it into the real environment. (The `.env.example` shows it commented out.) **Recommended improvement:** `dj_database_url.parse(config("DATABASE_URL", default=...))` so it behaves like the other variables.

5.2 Variables

Variable	Purpose	Required?	Default	Used by
`DJANGO_SECRET_KEY`	Django + JWT signing key	Required in prod	`dev-only-insecure-secret-key`	`SECRET_KEY`
`DJANGO_DEBUG`	Debug mode	Optional	`True`	`DEBUG`, `config/urls.py` (media serving)
`DJANGO_ALLOWED_HOSTS`	Allowed Host headers (CSV)	Required in prod	`localhost,127.0.0.1`	`ALLOWED_HOSTS`
`DATABASE_URL`	DB connection string (read from OS env)	Required in prod	SQLite `backend/db.sqlite3`	`DATABASES`
`CORS_ALLOWED_ORIGINS`	Browser origins allowed to call the API (CSV)	Required in prod	`http://localhost:3000`	corsheaders
`CELERY_BROKER_URL`	Redis broker URL	Optional	`redis://localhost:6379/0`	Celery
`CELERY_RESULT_BACKEND`	Redis result backend URL	Optional	`redis://localhost:6379/0`	Celery
`USE_S3_STORAGE`	Switch local disk ↔ Supabase Storage	Optional	`False`	storage block
`SUPABASE_STORAGE_ACCESS_KEY_ID`	S3 access key	**Required if** `USE_S3_STORAGE=True`	none	`AWS_ACCESS_KEY_ID`
`SUPABASE_STORAGE_SECRET_ACCESS_KEY`	S3 secret key	**Required if** `USE_S3_STORAGE=True`	none	`AWS_SECRET_ACCESS_KEY`
`SUPABASE_STORAGE_BUCKET`	Bucket name	Optional	`think-and-make`	`AWS_STORAGE_BUCKET_NAME`
`SUPABASE_STORAGE_ENDPOINT_URL`	S3 endpoint, e.g. `https://<project>.supabase.co/storage/v1/s3`	Effectively required for Supabase	`""` (would mean AWS)	`AWS_S3_ENDPOINT_URL`
`SUPABASE_STORAGE_REGION`	Region	Optional	`ap-south-1`	`AWS_S3_REGION_NAME`
`GEMINI_API_KEY`	Google Gemini key	Required for InquiBuddy AI	`""`	`lib/gemini_client.py`

The mobile/web apps have their own configuration and are out of scope here.

5.3 Why secrets must not be committed

- Git history is permanent: a secret committed once stays retrievable even after deletion.
- Anyone with `SECRET_KEY` can forge JWTs for any user (including admin).
- Storage keys give direct read/write access to every uploaded file, bypassing the API's permission checks.
- The project does this correctly for `.env` (git-ignored; only `.env.example` is committed). **Observation:** the legacy user import command contains default login passwords in source (see §36).

6. Django apps

Each section below follows the same outline: purpose, dependencies, models, serializers, views, URLs, permissions, services/tasks, tests, then the app's complete flow. File-level details (every class and function) are in §30–§31; model fields are in §7.

6.1 Core app (`apps/core`)

Purpose. Shared foundations used by every other app.

Piece	File	What it provides
`TimeStampedModel`	`models.py`	Abstract base with `created_at` (`auto_now_add`) and `updated_at` (`auto_now`). Almost every model inherits it.
`AuditLog`	`models.py`	"Who changed what" table, written explicitly via `record_audit()`.
Role constants	`permissions.py`	`ADMIN="admin"`, `IIF_STAFF="iif_staff"`, `SCHOOL="school"` (duplicated as `User.ADMIN` etc.).
Permission classes	`permissions.py`	`IsAdmin`, `IsAdminOrIIFStaff`, `IsSchoolUser`, `IsAdminOrIIFStaffOrReadOnly`.
`scope_queryset_to_role()`	`permissions.py`	The one function that implements "school sees only its own rows".
`SchoolScopedViewSetMixin`	`mixins.py`	Read-scoping + forced school on write, for school-form viewsets.
`record_audit()`	`audit.py`	Helper to write an `AuditLog` row.
`AuditLogAdmin`	`admin.py`	Read-only admin (all fields in `readonly_fields`).
Import commands	`management/commands/`	`import_legacy_sheets`, `import_master_users`.
Admin theme assets	`static/admin/`	`theme.css`, `force-light.js`.

Dependencies: DRF (`BasePermission`, `SAFE_METHODS`); the import commands depend on almost every app's models plus `openpyxl`. **Views/URLs:** none (`views.py` is an empty stub). **Tests:** `tests.py` is an empty stub; the scoping logic is tested indirectly through other apps' tests.

Flow: core has no endpoints. Other apps call it: a viewset's `get_queryset()` → `scope_queryset_to_role()`; a viewset's `perform_update()` → `record_audit()`.

Observation: `audit.py`'s docstring says to see "`accounts.services` and `inquibuddy.services`". `apps/accounts/services.py` **does not exist**; audit calls for users are made directly in `accounts/views.py`. The `SchoolScopedViewSetMixin` docstring lists "teams, observations, inquibuddy" among its users; in reality only `kits`, `sl_selection`, `headcounts` and `observations` (the `SessionObservationViewSet`) use the mixin — `teams` and `inquibuddy` call `scope_queryset_to_role()` directly.

6.2 Accounts app (`apps/accounts`)

Purpose. Custom user model with roles, JWT login, "who am I", and admin-only user management. Replaces the old plaintext-PIN "Users" sheet.

- **Models:** `User(AbstractUser)` with `role`, `school` (FK), `display_name`.
- **Serializers:** `ThinkAndMakeTokenObtainPairSerializer` (adds `role` and `school_id` claims to the JWT), `UserSerializer` (read/update), `UserCreateSerializer` (write-only `password`, hashes it).
- **Views:** `LoginView(TokenObtainPairView)` with `throttle_scope="login"`; `MeView(APIView)`; `UserViewSet(ModelViewSet)` with `IsAdmin` and audit logging on create/update.
- **URLs:** `auth/login/`, `auth/refresh/` (SimpleJWT's `TokenRefreshView` used directly), `auth/me/`, router `users/`.
- **Permissions:** login/refresh are public (SimpleJWT's token views set empty `permission_classes`, i.e. open); `me` requires login; `users` requires `role == "admin"`.
- **Admin:** `UserAdmin` combines Django's `UserAdmin` with unfold's `ModelAdmin` and adds a "Think & Make" fieldset (`role`, `school`, `display_name`).
- **Tests:** `tests.py` is an empty stub (no direct tests for login/refresh/me/users).

Flow (login): `POST /api/v1/auth/login/ {username,password}` → `ScopedRateThrottle` (10/min) → `ThinkAndMakeTokenObtainPairSerializer.validate()` → Django's `authenticate()` checks the hashed password → `get_token(user)` adds `role`, `school_id` → response `{refresh, access}`.

6.3 Programs app (`apps/programs`)

Purpose. The two top-level grouping entities: `Partner` (e.g. "Datla") and `ProgramInstance` (one program year/city run, e.g. `DATLA-HYD-2026`).

- **Serializers:** `PartnerSerializer` (`id`, `name`), `ProgramInstanceSerializer` (`id`, `code`, `label`, `partner`, `year`, `is_active`).
- **Views:** `PartnerViewSet`, `ProgramInstanceViewSet` (`select_related("partner")`) — both `ModelViewSet` + `IsAdminOrIIFStaffOrReadOnly`.
- **URLs:** `partners/`, `instances/`.
- **Permissions:** everyone logged in may **read** (the forms' Partner→School picker needs the list, including school accounts); only admin/IIF staff may write. Not school-scoped: a school user sees all partners and instances.
- **Tests:** stub only.

6.4 Schools app (`apps/schools`)

Purpose. The `School` record and its Form 1 (enrollment) and Form 2 (contact info) submissions, plus child tables parsed from legacy free-text cells.

- **Models:** `School`, `SchoolGradeEnrollment`, `SchoolTeacher`, `SessionSchedule`.
- **Serializers:** `SchoolSerializer` (full read, nested children read-only, `form*_submitted*` read-only); `SchoolForm1SubmitSerializer` (all Form 1 fields required; `grades` multiple-choice 6–9); `SchoolContactSubmitSerializer` with nested input serializers `ContactTeacherInputSerializer`, `ContactSessionScheduleInputSerializer`; simple ModelSerializers for the three child models.
- **Views:** `SchoolViewSet` (+ custom actions `submit-form1`, `submit-form2`, `progress`), `SchoolGradeEnrollmentViewSet`, `SchoolTeacherViewSet`, `SessionScheduleViewSet`.
- **URLs:** `schools/`, `school-grade-enrollments/`, `school-teachers/`, `session-schedules/`.
- **Permissions:** all four viewsets use `IsAdminOrIIFStaffOrReadOnly` and `scope_queryset_to_role()`. For `School`, the scoping field is `"id"` (a school user sees only the school whose `id == user.school_id`).

- **Important consequence:** a **school-role user cannot POST** `submit-form1` / `submit-form2` or PATCH a school — those are non-safe methods and the permission class only allows them for admin/IIF staff. The tests confirm this is exercised by IIF staff (`iif_staff_client`).
- **Business rules (in the view):**
- Once `form1_submitted` is true, only an admin may PATCH/PUT the school (`perform_update` → 403) or re-submit Form 1 (`submit_form1` → 400).
- Form 2 requires Form 1 first; once submitted, only an admin can re-submit.
- `submit_form1` stores the chosen grades as `grades_offered="6,7"`, sets `total_sections=len(grades)` and `get_or_create`s one `SchoolGradeEnrollment` per grade (never overwriting existing counts).
- `submit_form2` **replaces** all `SchoolTeacher` and `SessionSchedule` rows for the school (delete + `bulk_create`), and requires a schedule for every grade in `grades_offered`.
- **Tests:** `tests.py` — 19 tests covering scoping, 401 for anonymous, partner filter, Form 1/Form 2 validation and locking.

6.5 Kits app (`apps/kits`)

Purpose. Records kit handover (`KitDelivery`) per school: date, who delivered/received, which grades' kits, proof photo and acknowledgement letter (FKs to `files.File`).

- **View:** `KitDeliveryViewSet(SchoolScopedViewSetMixin, ModelViewSet)`, `IsAuthenticated`, `filterset_fields=["school","date_of_delivery"]`.
- **Rule:** only **one** delivery per school for non-admins — a second create returns **403** `"Kit handover has already been recorded..."`. Admins may add more.
- **Tests:** 5 tests (auto-assign school, anti-spoofing, one-per-school rule, admin override).

6.6 SL Selection app (`apps/sl_selection`)

Purpose. Student Leader (SL) assessments — Form 4. One row per assessed student.

- **Serializers:** `SLSelectionSerializer` (per-row CRUD), `SLEntryInputSerializer` (one SL inside a bulk request), `SLSelectionBulkSubmitSerializer` (one grade/section + `entries[]`).
- **View:** `SLSelectionViewSet(SchoolScopedViewSetMixin, ModelViewSet)` + custom action `bulk-submit`.
- **Rule:** for non-admins, a grade/section can be bulk-submitted only once (400 `"SL Selection data has already been submitted..."`).
- **Tests:** 6 tests.

6.7 Headcounts app (`apps/headcounts`)

Purpose. Form 3 — per grade/section counts of students, SLs, clusters, teams, with files for the teams-info photo and the student database, and enum statuses `extraction_status` / `count_validation`.

- **View:** `StudentHeadcountViewSet(SchoolScopedViewSetMixin, ModelViewSet)`. Overrides `create()` and `update()` to add a `mismatch_warning` key to the response, computed synchronously by `apps.reports.services.reconcile_section()`.
- **Constraint:** unique `(school, grade, section)` → duplicate create returns 400 (DRF's auto-generated `UniqueTogetherValidator`).
- **Tests:** 3 tests here + 2 in `reports/tests.py` for `mismatch_warning`.

6.8 Teams app (`apps/teams`)

Purpose. Hierarchy `Cluster (school, grade, section, number)` → `Team (team_code, sl_name)` → `Student (name)`.

- **Views:** `ClusterViewSet` (prefetches `teams__students`), `TeamViewSet`, `StudentViewSet`; all `IsAdminOrIIFStaffOrReadOnly` with `scope_queryset_to_role()` using `school_id`, `cluster__school_id` and `team__cluster__school_id` respectively.
- **Nested serializers:** `ClusterSerializer.teams` → `TeamSerializer.students` → `StudentSerializer` (all read-only nesting).
- **Note:** the README says these rows are meant to come from an OCR'd student database, but **no import for Teams/Clusters/Students exists yet** (explicitly stated in `backend/README.md`). They can be created via the API (admin/IIF) or the admin.
- **Tests:** stub only (used as fixtures in InquiBuddy tests).

6.9 Observations app (`apps/observations`)

Purpose. One `SessionObservation` per observed session (school, grade, section, unit 1–3, session number). The legacy "Session Tracker" and "Session Progress" pivot tabs are replaced by queries in `selectors.py`.

- **Views:** `SessionObservationViewSet(SchoolScopedViewSetMixin, ModelViewSet)`; `SessionTrackerView(APIView)` (`?unit=&grade=`); `SessionProgressView(APIView)`.
- **URLs:** `session-observations/`, `session-tracker/`, `session-progress/`.
- **Tests:** stub only.

6.10 InquiBuddy app (`apps/inquibuddy`)

Purpose. The AI evaluation flow for a team's idea: upload idea photo (+ optional audio) → generate 5 bilingual questions with Gemini (background job) → download a PDF report (per team, or all teams in a section).

- **Models:** `InquibuddySubmission` (1:1 with `Team`; status `pending → processing → evaluated`), `SubmissionFile`, `AIFeedbackQuestion`, `FeedbackReport`.
- **Signal:** `post_save` on `Team` auto-creates a `pending` submission (`signals.py`, wired in `InquibuddyConfig.ready()`).
- **Services:** `attach_submission_file`, `get_or_create_individual_report`, `create_all_teams_report` (+ private helpers).
- **Task:** `generate_ai_feedback` (Celery, `max_retries=2`, 30 s delay).
- **View:** `InquibuddySubmissionViewSet(ReadOnlyModelViewSet)` + actions `upload-photo`, `upload-audio`, `generate-feedback`, `generate-feedback-bulk`, `report`, `report-all`.
- **Permissions:** `IsAuthenticated` + `scope_queryset_to_role(..., school_field="team__cluster__school_id")`. School users **can** use every action (they're all scoped by `get_object()` / `get_queryset()`).
- **Tests:** 3 tests (signal, upload → enqueue with mocked `.delay`, photo required).

6.11 Files app (`apps/files`)

Purpose. A single `File` table for every upload (school photo, kit proof, acknowledgement letter, teams-info photo, student database, InquiBuddy photo/audio, generated PDFs). Storage backend is local disk or Supabase S3.

- **View:** `FileViewSet = CreateModelMixin + RetrieveModelMixin + GenericViewSet` → only `POST /files/` and `GET /files/{id}/`. No list, update or delete.
- **Serializers:** `FileUploadSerializer` (input: `file`, `entity_type`, `entity_id`), `FileSerializer` (output with computed absolute `url`).
- **Tests:** stub only.

6.12 Reports app (`apps/reports`)

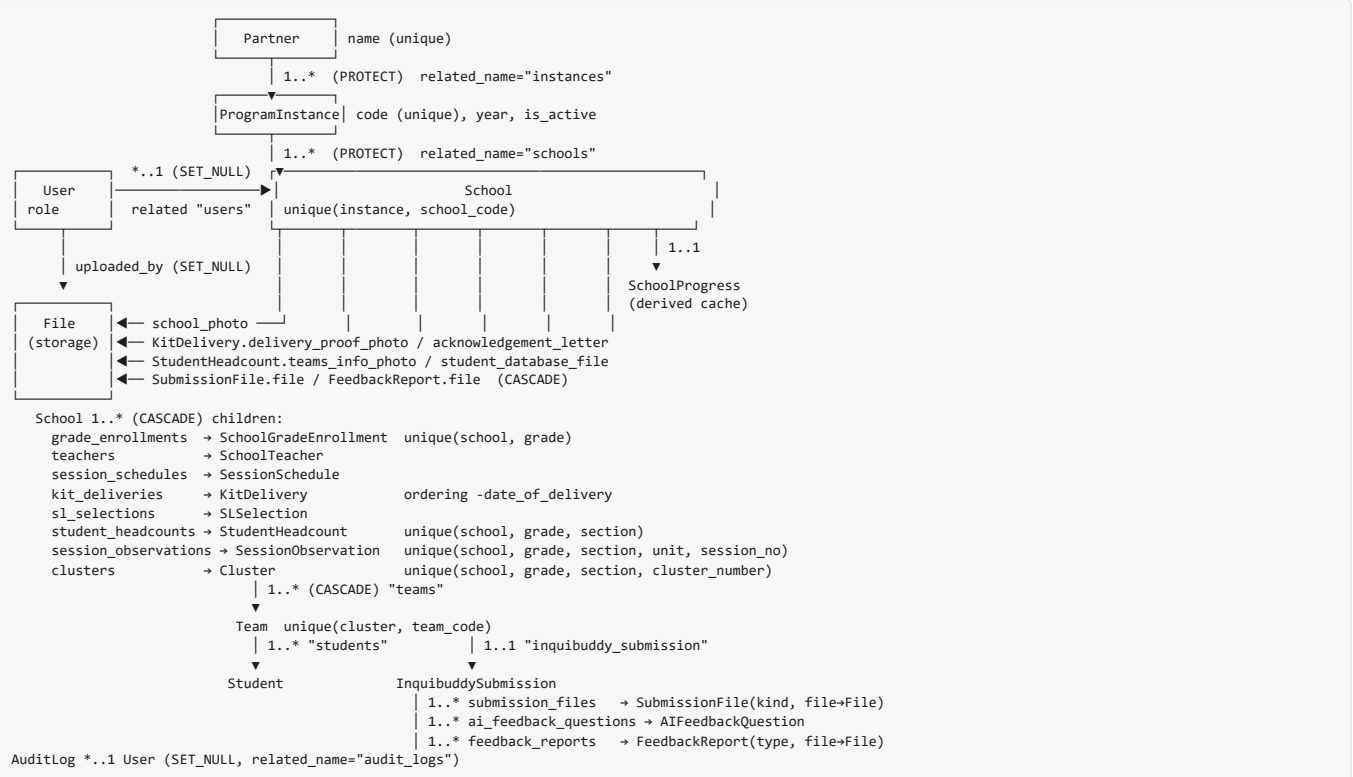
Purpose. The School Progress Dashboard. `SchoolProgress` is a **derived cache** (one row per school) recomputed from School / StudentHeadcount / SLSelection / KitDelivery.

- **Services:** `reconcile_school`, `reconcile_section`, `reconcile_all_schools`, `check_headcount_mismatch` (+ helpers).
- **Selectors:** `compute_kpis`.
- **Task:** `reconcile_school_progress` (nightly via Celery Beat).
- **Command:** `python manage.py reconcile_school_progress`.
- **View:** `ReportsSummaryView` → `GET reports/summary/` returns `{kpis, schools}`; school users get only their row and `kpis: null`. Single-school detail is `GET schools/{id}/progress/` (in the schools app).
- **Tests:** 20 tests — the best-tested module.

---

7. Models and database design

7.1 Entity-relationship overview



In plain language: a *Partner* funds one or more *program instances*; each instance enrolls *schools*. Almost everything else hangs off a school. Students are organised as school → cluster (per grade/section) → team → student, and each team has exactly one InquiBuddy submission. Uploaded files live in one shared `File` table that other rows point to.

Design choices worth noticing: - **Grade and section are repeated columns**, not foreign keys to a "Section" table. `StudentHeadcount`, `SLSelection`, `SessionObservation` and `Cluster` all store `grade` + `section` strings. That's why `reports/services.py` normalises sections (`" a " → "A"`) before comparing. - **`grades_offered` is a comma-separated string** on `School` (e.g. `"6,7,8"`), parsed in Python where needed. - **`external_ref`** on School, KitDelivery, SLSelection, StudentHeadcount keeps the Google Form "Submission ID" so the import command can upsert idempotently. - **`related_name="+"`** on the `File` FKs means "no reverse accessor" — you can't do `file.school_set`. - **`on_delete` strategy:** `PROTECT` at the top (you can't delete a partner/instance that still has schools), `CASCADE` for data owned by a school/team, `SET_NULL` for optional links to files and users.

7.2 Common base: `TimeStampedModel` (abstract)

Field	Type	Notes
`created_at`	`DateTimeField(auto_now_add=True)`	Set once on insert.
`updated_at`	`DateTimeField(auto_now=True)`	Set on every `save()` (but **not** on `QuerySet.update()` or `bulk_create`-then-update).

`class Meta: abstract = True` → no table; fields are copied into each child. All models below inherit it **except** `AuditLog` (has its own `created_at`) and `User` (inherits `AbstractUser`).

Unless stated, fields are `null=False`, `blank=False`, no `default`, not unique, no explicit `db_index` (FKs get an index automatically). Every model has an implicit `id` `BigAutoField` primary key.

7.3 Model: `User` (`apps/accounts/models.py`)

Why: a real account per person/school with a hashed password and a role, replacing shared PINs. Extends `AbstractUser`, so it has all standard fields (`username` unique, `password`, `email`, `first_name`, `last_name`, `is_staff`, `is_superuser`, `is_active`, `date_joined`, `last_login`, `groups`, `user_permissions`).

Field	Type	null / blank	default	Relationship	Notes
`role`	`CharField(20, choices)`	no / no	—	—	`admin`, `iif_staff`, `school`
`school`	`ForeignKey("schools.School")`	yes / yes	—	many users → one school, `on_delete=SET_NULL`, `related_name="users"`	Only for `role=school`. Not enforced by a constraint.
`display_name`	`CharField(150)`	no / yes	`""` implicitly	—	Human name.

`__str__` → `"username (role)"`. No custom methods. **Observation:** nothing prevents a `school` user with `school=None`, or an admin with a school; `scope_queryset_to_role` would then filter by `school_id=None` and return nothing.

7.4 Model: `AuditLog` (`apps/core/models.py`)

Field	Type	Details
`user`	FK `AUTH_USER_MODEL`	`null=True, blank=True, SET_NULL`, `related_name="audit_logs"`
`action`	`CharField(100)`	e.g. `create_user`, `update_user`, `generate_ai_feedback`, `download_individual_report`, `download_all_teams_report`
`entity_type`	`CharField(100)`	e.g. `User`, `InquibuddySubmission`, `FeedbackReport`
`entity_id`	`CharField(64)`	`blank=True, default=""` (string so it can hold any key type, e.g. `"all_teams"`)
`before`, `after`	`JSONField`	`null=True, blank=True` — snapshots
`created_at`	`DateTimeField(auto_now_add=True)`	

`Meta`: `ordering=["-created_at"]`; indexes on `(entity_type, entity_id)` and `(user, created_at)` — so "history of entity X" and "what did user Y do" are fast.

7.5 Model: `Partner` / `ProgramInstance` (`apps/programs/models.py`)

`Partner`: `name` `CharField(150, unique=True)`.

`ProgramInstance`:

Field	Type	Details
`code`	`CharField(50, unique=True)`	e.g. `DATLA-HYD-2026`
`label`	`CharField(200, blank=True)`	
`partner`	FK `Partner`	`on_delete=PROTECT`, `related_name="instances"`
`year`	`PositiveIntegerField`	
`is_active`	`BooleanField(default=True)`	Not used in any query currently.

7.6 Model: `School` (`apps/schools/models.py`)

Why: the central entity. Combines data from the legacy `School_Enrollment` and `Schools_Contact_Info` tabs. Many fields are nullable/blank so a pre-seeded roster row (just code + name) is valid; strictness is applied by the Form 1 serializer instead.

Field	Type	null / blank / default	Notes
`instance`	FK `ProgramInstance`	required, `PROTECT`, `related_name="schools"`	
`school_code`	`CharField(20)`	required	Unique **per instance** (constraint `unique_school_code_per_instance`).
`name`	`CharField(200)`	required	
`district`	`CharField(100)`	blank	
`location`	`CharField(200)`	blank	
`distance_to_iif_km`	`DecimalField(6,2)`	null, blank	
`gender_type`	`CharField(10, choices)`	blank	`boys`, `girls`, `co-ed`
`school_type`	`CharField(20, choices)`	blank	`government`, `private`, `aided`
`medium`	`CharField(10, choices)`	blank	`telugu`, `english`, `urdu`
`grades_offered`	`CharField(50)`	blank	CSV, e.g. `"6,7,8"`
`total_sections`	`PositiveIntegerField`	null, blank	Set to `len(grades)` on Form 1.
`principal_name` / `principal_phone`	`CharField(150)` / `CharField(20)`	blank	
`principal_email`	`EmailField`	blank	
`principal_acknowledged`	`BooleanField`	default `False`	
`lab_room`, `internet`, `kit_storage`	`BooleanField`	null, blank (tri-state)	
`smart_board`	`CharField(20, choices)`	blank, default `""`	`yes`, `no`, `yes_not_working` (was a boolean before migration 0002).
`maps_link`	`URLField`	blank	
`school_photo`	FK `files.File`	null, blank, `SET_NULL`, `related_name="+"`	
`iif_poc`	`CharField(150)`	blank	Required by Form 2.
`observations`, `next_steps`	`TextField`	blank	
`visited_by`	`CharField(150)`	blank	
`visit_date`	`DateField`	null, blank	
`form1_submitted` / `form1_submitted_at`	`BooleanField(default=False)` / `DateTimeField(null)`		Lock flag for Form 1.
`form2_submitted` / `form2_submitted_at`	same		Lock flag for Form 2.

Field	Type	null / blank / default	Notes
external_ref	CharField(64)	blank, default ""	Google Form submission id.

Meta: UniqueConstraint(fields=["instance","school_code"], name="unique_school_code_per_instance"), ordering=["school_code"]. Reverse accessors: users, grade_enrollments, teachers, session_schedules, kit_deliveries, sl_selections, student_headcounts, session_observations, clusters, progress.

7.7 School child models

Model	Fields	Constraints / Meta
SchoolGradeEnrollment	school FK (CASCADE, grade_enrollments), grade PositiveSmallInt, total_sections PositiveInt default 0, total_students PositiveInt default 0	UniqueConstraint(school, grade)
SchoolTeacher	school FK (CASCADE, teachers), name Char(150), phone Char(20, blank), grades_taught Char(50, blank, e.g. "6,7")	—
SessionSchedule	school FK (CASCADE, session_schedules), grade PositiveSmallInt, day_of_week Char(3, choices mon … sat), time TimeField	—

Observation: import_legacy_sheets maps sun in its day map, but SessionSchedule.DAY_CHOICES has no Sunday. Model choices aren't enforced by the database, so a Sunday row could be stored by the importer but would fail validation through the API.

7.8 Model: KitDelivery (apps/kits/models.py)

Field	Type	Details
school	FK School	CASCADE, kit_deliveries
delivered_by, received_by	CharField(150, blank)	
date_of_delivery	DateField	required
grade_6_kit … grade_9_kit	BooleanField(default=False)	one flag per grade
delivery_proof_photo, acknowledgement_letter	FK files.File	null, blank, SET_NULL, +
external_ref	CharField(64, blank, default "")	

Meta.ordering = ["-date_of_delivery"] — so school.kit_deliveries.first() is the **latest** delivery (relied on by reports.services._compute_kit_status).

7.9 Model: SLSelection (apps/sl_selection/models.py)

Field	Type	Details
school	FK School	CASCADE, sl_selections
grade	PositiveSmallInt	
section	CharField(5)	
teacher	CharField(150, blank)	
sl_name	CharField(150)	required
interested_in_role, attendance_above_90, speaks_clearly, speaks_loudly, understands_english, teacher_acknowledged	BooleanField(default=False)	assessment checkboxes
sl_status	CharField(20, choices, default "pending")	selected, not_selected, pending
external_ref	CharField(64, blank)	

Meta.ordering = ["school","grade","section"]. No uniqueness constraint (a section has many SLs).

7.10 Model: StudentHeadcount (apps/headcounts/models.py)

Field	Type	Details
school	FK School	CASCADE, student_headcounts
grade, section	PositiveSmallInt, CharField(5)	
total_sl, total_clusters, total_teams, total_students	PositiveIntegerField(default=0)	
teams_info_photo, student_database_file	FK files.File	null, blank, SET_NULL, +
extraction_status	choices ok / extraction_error / needs_validation, default ok	
count_validation	choices ok / mismatch / needs_validation, default ok	**Written by reports.services.check_headcount_mismatch.**
external_ref	CharField(64, blank)	

Meta: UniqueConstraint(["school","grade","section"], name="unique_headcount_per_section").

7.11 Models: `Cluster`, `Team`, `Student` (`apps/teams/models.py`)

Model	Fields	Constraints / ordering
`Cluster`	`school` FK (CASCADE, `clusters`), `grade`, `section` Char(5), `cluster_number` PositiveInt	unique `(school, grade, section, cluster_number)`; ordering by those
`Team`	`cluster` FK (CASCADE, `teams`), `team_code` Char(20) (e.g. `T001`), `sl_name` Char(150)	unique `(cluster, team_code)`; ordering `team_code`
`Student`	`team` FK (CASCADE, `students`), `name` Char(150)	—

Creating a `Team` (via `.save()` / `.create()`) fires the `post_save` signal that creates its `InquibuddySubmission`.

7.12 Model: `SessionObservation` (`apps/observations/models.py`)

Field	Type	Details
`school`	FK School	CASCADE, `session_observations`
`grade`, `section`	PositiveSmallInt, Char(5)	
`unit`	PositiveSmallInt, choices 1/2/3 ("Level 1..3")	
`session_no`	PositiveSmallInt	
`observed_by`	Char(150, blank)	
`date`	DateField	required
`notes`	TextField(blank)	

Constraint `unique_session_observation` on `(school, grade, section, unit, session_no)` — a session can be recorded only once.

7.13 InquiBuddy models (`apps/inquibuddy/models.py`)

Model	Fields	Notes
`InquibuddySubmission`	`team` **OneToOneField** Team (CASCADE, `inquibuddy_submission`), `status` choices `pending` / `processing` / `evaluated` default `pending`, `evaluation_count` PositiveInt default 0	One per team. Class constants `PENDING`, `PROCESSING`, `EVALUATED`.
`SubmissionFile`	`submission` FK (CASCADE, `submission_files`), `file` FK `files.File` (CASCADE, `+`), `kind` `photo` / `audio`	Service keeps at most one of each kind.
`AIFeedbackQuestion`	`submission` FK (CASCADE, `ai_feedback_questions`), `order` PositiveSmallInt, `question_en`, `question_te` TextField	ordering `(submission, order)`
`FeedbackReport`	`submission` FK **nullable** (CASCADE, `feedback_reports`), `type` `individual` / `all_teams`, `file` FK `files.File` (CASCADE)	`submission=None` for all-teams reports.

7.14 Model: `File` (`apps/files/models.py`)

Field	Type	Details
`file`	`FileField(upload_to=upload_path)`	Stored path: `"<entity_type>/<entity_id or 'misc'>/<filename>"`
`mime_type`	Char(100, blank)	From the upload's `content_type`
`size`	PositiveInt default 0	bytes
`uploaded_by`	FK User	`null=True`, SET_NULL, `uploaded_files`
`entity_type`	Char(100)	required — what the file belongs to (client-supplied for `/files/`)
`entity_id`	Char(64, blank)	
`legacy_url`	URLField(blank)	Original Google Drive link (fallback). Not set anywhere in code yet.

`Meta.indexes = [Index(fields=["entity_type","entity_id"])]`.

7.15 Model: `SchoolProgress` (`apps/reports/models.py`)

Field	Type	Details
`school`	**OneToOneField** School	CASCADE, `related_name="progress"`
`form1_status`, `form2_status`, `kit_status`	choices `not_started` / `in_progress` / `complete`, default `not_started`	
`form3_pct`, `form4_pct`	PositiveSmallInt default 0	0–100
`mismatch_flags`	`JSONField(default=list, blank=True)`	list of `{grade, section, total_sl, sl_selection_count}`

Why a cache? Computing percentages across several tables for every school on every dashboard load would be slow and complex. Instead the value is precomputed (nightly, or on first read) and can always be rebuilt from source tables — it's never the source of truth.

8. Django ORM concepts used in this project

The **ORM** (Object-Relational Mapper) turns Python method calls into SQL. A **QuerySet** is lazy: nothing hits the database until you iterate, slice, call `list()`, `.count()`, `.exists()`, `.get()`, etc.

Concept	Where it's used	Equivalent SQL (conceptual)
`.objects.all()`	`User.objects.all().order_by("username")` (UserViewSet)	`SELECT * FROM accounts_user ORDER BY username`
`.filter()`	`queryset.filter(school_id=user.school_id)` in `scope_queryset_to_role`	`... WHERE school_id = %s`
Filter across relations (`__`)	`filter(team__cluster__school_id=...)`	`JOIN teams_team, teams_cluster ... WHERE cluster.school_id = %s`
`.get()`	`InquibuddySubmission.objects...get(id=submission_id)` (task); `school.student_headcounts.get(grade=, section=)`	`SELECT ... WHERE id=%s` — raises `DoesNotExist` / `MultipleObjectsReturned`
`.create()`	`File.objects.create(...)`, `AuditLog.objects.create(...)`	`INSERT ...`
`.get_or_create()`	`SchoolGradeEnrollment.objects.get_or_create(school=, grade=)`; `SchoolProgress.objects.get_or_create(school=)`; signal	`SELECT`; if none, `INSERT`
`.update_or_create()`	`reconcile_school`; import commands	`SELECT`; then `UPDATE` or `INSERT`
`QuerySet.update()`	import command: `School.objects.filter(pk=...).update(**updates)`	`UPDATE ... WHERE id=%s` (bypasses `save()`, `auto_now`, signals)
`.delete()`	`school.teachers.all().delete()`; `SubmissionFile.objects.filter(...).delete()`	`DELETE ... WHERE ...` (+ cascades handled by Django)
`.exists()`	`KitDelivery.objects.filter(school=school).exists()`	`SELECT 1 ... LIMIT 1` — cheaper than `.count()`
`.count()`	`School.objects.count()`, `submissions.count()`	`SELECT COUNT(*)`
`.first()`	`school.kit_deliveries.first()`; `submission_files.filter(kind=PHOTO).first()`	`... ORDER BY <ordering or pk> LIMIT 1`
`.values()` / `.values_list()`	`observations.selectors`; `team.students.values_list("name", flat=True)`	`SELECT only these columns` → dicts / tuples
`.annotate()` + `Count`	`session_progress()` → `.values("school_id","school__school_code","unit").annotate(sessions_completed=Count("id"))`	`SELECT school_id, code, unit, COUNT(id) ... GROUP BY school_id, code, unit`
`.distinct()`	`generate_feedback_bulk` (join to `submission_files` could duplicate rows)	`SELECT DISTINCT`
`.select_related()`	Nearly every viewset (`select_related("school")`, `"team__cluster__school__instance__partner"`)	`JOIN` in the same query — for FK / 1:1 "forward" relations
`.prefetch_related()`	`SchoolViewSet` (`grade_enrollments`, `teachers`, `session_schedules`); `ClusterViewSet` (`teams__students`); InquiBuddy; `reconcile_all_schools`	One extra `SELECT ... WHERE fk IN (...)` per relation, joined in Python — for reverse FK / M2M
`.only()`	migration `0002_form1_fields`	`SELECT id, smart_board`
`bulk_create()`	Form 2 teachers/schedules; SL bulk submit; AI questions	One multi-row `INSERT` — **does not call `save()` or send signals**
`transaction.atomic()`	`import_legacy_sheets`, `import_master_users`	`BEGIN ... COMMIT` / `ROLLBACK` on exception
`refresh_from_db()`	tests	Re-`SELECT` the row
`save(update_fields=[...])`	task, views, services	`UPDATE ... SET only those columns` — avoids overwriting concurrent changes to other columns

Not used: `Q` objects, `F` expressions, `aggregate()`, `bulk_update()`, `select_for_update()`.

- `F` **would matter here:** the task does `submission.evaluation_count += 1; save(...)` — a read-modify-write in Python. `F("evaluation_count") + 1` would do it atomically in SQL (see §37).
- **Transactions matter here:** `submit_form2` deletes teachers/schedules then bulk-creates new ones **without** `transaction.atomic()`. Django runs each statement in autocommit mode (`ATOMIC_REQUESTS` isn't set), so if the second `bulk_create` failed you'd be left with teachers deleted and the form marked submitted. **Recommended improvement:** wrap the multi-write actions (`submit_form1`, `submit_form2`, `bulk_submit`, `attach_submission_file`) in `transaction.atomic()`.

Example: what `select_related` saves

```
# SessionObservationViewSet.queryset
SessionObservation.objects.select_related("school")
```

Without it, serializing 50 observations and calling `str(obs)` (which reads `obs.school.school_code`) would run 1 query for the list + 50 queries for schools — the **N+1 problem**. With it: 1 query with a JOIN.

9. Serializers

9.1 Serializer vs ModelSerializer

- `serializers.Serializer`: you declare every field by hand; you write `create()` / `update()` yourself if you call `save()`. Used here for **input-only** payloads that don't map 1:1 to a model: `SLSelectionBulkSubmitSerializer`, `SLEntryInputSerializer`, `ContactTeacherInputSerializer`, `ContactSessionScheduleInputSerializer`.
- `serializers.ModelSerializer`: generates fields from the model (types, `max_length`, choices, `required` derived from `blank` / `default` / `null`), generates `UniqueTogetherValidator`s from unique constraints, and provides default `create()` / `update()`. Used for everything else.

9.2 The validation pipeline (what happens on `serializer.is_valid()`)

```
serializer = XSerializer(instance?, data=request.data, partial=?)
serializer.is_valid(raise_exception=True)
   │
   ├─ 1. to_internal_value(): for each writable field
   │       field.run_validation(value)   → type conversion, required/allow_null/allow_blank,
   │                                       max_length, choices, field-level validators
   │       validate_<field_name>(value)  → your per-field hook (e.g. validate_teachers)
   ├─ 2. Serializer-level validators (Meta.validators or auto: UniqueTogetherValidator)
   ├─ 3. validate(attrs)                 → your cross-field hook (e.g. Form 2 grade check)
   └─ errors? → raise ValidationError → DRF returns HTTP 400 {field: [messages]}
serializer.save(**extra)   → merges extra kwargs into validated_data
   ├─ instance given → update(instance, validated_data)
   └─ otherwise      → create(validated_data)
serializer.data            → to_representation(instance) → JSON-able dict
```

9.3 Serializer catalogue

Serializer	File	Type	Notable details
`ThinkAndMakeTokenObtainPairSerializer`	accounts	extends SimpleJWT `TokenObtainPairSerializer`	Overrides classmethod `get_token(user)` to add `role`, `school_id` claims.
`UserSerializer`	accounts	ModelSerializer	Fields `id, username, email, display_name, role, school, is_active, date_joined`; `date_joined` read-only. Used for list/retrieve/update.
`UserCreateSerializer`	accounts	ModelSerializer	`password = CharField(write_only=True, min_length=8)`; `create()` pops password, calls `user.set_password()` (hashing), saves.
`PartnerSerializer`, `ProgramInstanceSerializer`	programs	ModelSerializer	Plain. `partner` is a PK.
`SchoolSerializer`	schools	ModelSerializer	Nested read-only `grade_enrollments`, `teachers`, `session_schedules` (`many=True, read_only=True`); `read_only_fields` = the four `form*_submitted*` fields so clients can't flip locks.
`SchoolForm1SubmitSerializer`	schools	ModelSerializer (on `School`)	`grades = MultipleChoiceField(choices=[6,7,8,9])` (not a model field; popped by the view). `extra_kwargs` makes almost every field `required=True` with `allow_blank=False` / `allow_null=False`; `principal_email`, `school_photo`, `observations`, `next_steps` optional.
`ContactTeacherInputSerializer`	schools	Serializer	`name` required; `phone`, `grades_taught` optional.
`ContactSessionScheduleInputSerializer`	schools	Serializer	`grade` int, `day_of_week` choice, `time` TimeField (`"10:30:00"`).
`SchoolContactSubmitSerializer`	schools	ModelSerializer (on `School`)	Fields `iif_poc` (required), `teachers` (nested many), `session_schedules` (nested many). `validate_teachers` → at least one. `validate()` → every grade in `self.instance.grades_offered` must have a schedule.
`SchoolGradeEnrollmentSerializer`, `SchoolTeacherSerializer`, `SessionScheduleSerializer`	schools	ModelSerializer	Plain, include `school`.
`KitDeliverySerializer`	kits	ModelSerializer	`extra_kwargs={"school": {"required": False}}` so school users may omit it.
`SLSelectionSerializer`	sl_selection	ModelSerializer	Same `school` not required.
`SLEntryInputSerializer`	sl_selection	Serializer	One SL: name + booleans + `sl_status` choice (all required).
`SLSelectionBulkSubmitSerializer`	sl_selection	Serializer	`school = PrimaryKeyRelatedField(queryset=School.objects.all(), required=False)`, `grade`, `section`, `teacher`, `teacher_acknowledged`, `entries = SLEntryInputSerializer(many=True)`; `validate_entries` → at least one. No `create()` — the view builds rows itself.
`StudentHeadcountSerializer`	headcounts	ModelSerializer	`school` not required; auto `UniqueTogetherValidator(school, grade, section)`.
`ClusterSerializer` → `TeamSerializer` → `StudentSerializer`	teams	ModelSerializer	Nested read-only (`teams`, `students`).
`SessionObservationSerializer`	observations	ModelSerializer	`school` not required; auto unique validator on 5 fields.
`InquibuddySubmissionSerializer`	inquibuddy	ModelSerializer	Nested read-only `submission_files`, `ai_feedback_questions`, `feedback_reports`; `team_code = CharField(source="team.team_code", read_only=True)`, `sl_name` likewise; `status`, `evaluation_count` read-only.
`SubmissionFileSerializer`, `AIFeedbackQuestionSerializer`, `FeedbackReportSerializer`	inquibuddy	ModelSerializer	`file` is output as the File **id** (a PK), not a URL. Clients fetch the URL via `GET /files/{id}/`.

Serializer	File	Type	Notable details
`FileSerializer`	files	ModelSerializer	`url = SerializerMethodField()` → `get_url()` returns `request.build_absolute_uri(obj.file.url)` (with S3 enabled, `file.url` is already an absolute signed URL and is returned unchanged by `build_absolute_uri`).
`FileUploadSerializer`	files	ModelSerializer	Input `file`, `entity_type`, `entity_id`.
`SchoolProgressSerializer`	reports	ModelSerializer	`school_code`, `school_name` via `source="school.code/name"`.

9.4 `UniqueTogetherValidator` and why it matters

For a model with `UniqueConstraint(fields=["school","grade","section"])`, `ModelSerializer` automatically adds a `UniqueTogetherValidator`. That validator runs **during `is_valid()`**, **before** `perform_create()`, and it needs **all** the constrained fields to be present in the input. It also forces those fields to be required. This is why `SchoolScopedViewSetMixin.get_serializer()` injects `school` into the incoming data **before** the serializer is built — see §14.

9.5 Full validation flow examples

Form 1 (`SchoolForm1SubmitSerializer`) 1. View loads the school via `get_object()` (scoped). 2. `SchoolForm1SubmitSerializer(school, data=request.data, partial=False)`. 3. Each field validated: e.g. missing `principal_name` → `{"principal_name": ["This field is required."]}` → 400. 4. `grades` must be a list of choices from `[6,7,8,9]` → returned as a `set`. 5. View pops `grades`, sorts them, calls `serializer.save(grades_offered="6,7", total_sections=2, form1_submitted=True, form1_submitted_at=now)` → `ModelSerializer.update()` sets attributes and `instance.save()`.

Form 2 (`SchoolContactSubmitSerializer`) 1. Field validation: `iif_poc` non-blank; each teacher/schedule item validated by the nested serializer. 2. `validate_teachers([])` → `"Add at least one teacher."`. 3. `validate(attrs)`: `active_grades = {6,7}` from `self.instance.grades_offered`; `scheduled = {6}` → error `{"session_schedules": "Missing a session schedule for grade(s): 7."}`. 4. View pops `teachers` / `session_schedules` (they aren't model fields — `ModelSerializer.update()` would choke on nested data otherwise), saves `iif_poc` + flags, then replaces child rows.

10. Views and ViewSets

10.1 DRF view types used

Type	Where	What you get
`APIView`	`MeView`, `SessionTrackerView`, `SessionProgressView`, `ReportsSummaryView`	You write `get()` / `post()`; auth/permission/throttle still run. No pagination/filtering unless you add it.
SimpleJWT views (`GenericAPIView` subclasses)	`LoginView(TokenObtainPairView)`, `TokenRefreshView`	POST-only token endpoints with no auth or permission classes (open).
`ModelViewSet`	Users, Partners, Instances, Schools (+ children), Kits, SL, Headcounts, Clusters/Teams/Students, Observations	list, create, retrieve, update, partial_update, destroy.
`ReadOnlyModelViewSet`	`InquibuddySubmissionViewSet`	list + retrieve only, plus custom `@action`s.
`GenericViewSet` + mixins	`FileViewSet` (`CreateModelMixin`, `RetrieveModelMixin`)	create + retrieve only.
`@action`	schools (`submit-form1`, `submit-form2`, `progress`), sl_selection (`bulk-submit`), inquibuddy (6 actions)	Extra routes on a viewset. `detail=True` → `/resource/{pk}/name/`; `detail=False` → `/resource/name/`.

10.2 ViewSet hook methods used (framework "magic")

Hook	Called by	When	Used here for
`get_queryset()`	`list()`, and `get_object()` for retrieve/update/destroy/detail actions	every such request	Role scoping + `select_related` / `prefetch_related`.
`get_object()`	retrieve/update/destroy and your `detail=True` actions	—	`get_queryset()` → `filter_queryset()` → `get(pk=...)` → 404 if not in the scoped set → `check_object_permissions()`.
`get_serializer_class()`	`get_serializer()`	every request that serializes	`UserViewSet` (create vs others), `FileViewSet`.
`get_serializer()`	`create()`, `update()`, `list()`, `retrieve()`	—	`SchoolScopedViewSetMixin` injects `school`.
`perform_create(serializer)`	`CreateModelMixin.create()` after `is_valid()`	POST	Forcing school, the "one kit per school" rule, audit, file metadata.
`perform_update(serializer)`	`UpdateModelMixin.update()` after `is_valid()`	PUT/PATCH	Form 1 lock (schools), audit (users).
`create()` / `update()`	the router's dispatch	POST / PUT / PATCH	Headcounts wraps them to add `mismatch_warning`.

10.3 Endpoint tables

All paths are relative to `/api/v1/`. "Scoped" means `scope_queryset_to_role` limits school users to their own school's rows (others → 404). Pagination (P) = `PageNumberPagination`, 50 per page.

Accounts

Method	Endpoint	Purpose	Auth	Permission	Serializer	Response
POST	`auth/login/`	Get tokens	none	open, throttled 10/min	`ThinkAndMakeTokenObtainPairSerializer`	`{refresh, access}`
POST	`auth/refresh/`	New access (and refresh) token	none	open	SimpleJWT `TokenRefreshSerializer`	`{access, refresh}`
GET	`auth/me/`	Current user	JWT	IsAuthenticated	`UserSerializer`	user object
GET/POST	`users/`	List (P) / create user	JWT	IsAdmin	`UserSerializer` / `UserCreateSerializer`	user(s)
GET/PUT/PATCH/DELETE	`users/{id}/`	Manage user	JWT	IsAdmin	`UserSerializer`	user / 204

Programs

Method	Endpoint	Purpose	Permission	Serializer
GET, POST	`partners/`	List (P) / create	read: any logged-in; write: admin/IIF	`PartnerSerializer`
GET, PUT, PATCH, DELETE	`partners/{id}/`		same	
GET, POST	`instances/`	List (P) / create	same	`ProgramInstanceSerializer`
GET, PUT, PATCH, DELETE	`instances/{id}/`		same	

Schools

Method	Endpoint	Purpose	Permission	Serializer	Filters
GET, POST	`schools/`	List (P, scoped) / create	read any; write admin/IIF	`SchoolSerializer`	`instance`, `instance__partner`, `school_code`, `district`
GET, PUT, PATCH, DELETE	`schools/{id}/`	Detail / draft save	same; PUT/PATCH after Form 1 → admin only (403)	`SchoolSerializer`	
POST	`schools/{id}/submit-form1/`	Submit Form 1	admin/IIF	in: `SchoolForm1SubmitSerializer`, out: `SchoolSerializer`	
POST	`schools/{id}/submit-form2/`	Submit Form 2	admin/IIF	in: `SchoolContactSubmitSerializer`, out: `SchoolSerializer`	

Method	Endpoint	Purpose	Permission	Serializer	Filters
GET	`schools/{id}/progress/`	Progress row (created if missing)	any, scoped	`SchoolProgressSerializer`	
CRUD	`school-grade-enrollments/`		read any (scoped); write admin/IIF	`SchoolGradeEnrollmentSerializer`	`school`, `grade`
CRUD	`school-teachers/`		same	`SchoolTeacherSerializer`	`school`
CRUD	`session-schedules/`		same	`SessionScheduleSerializer`	`school`, `grade`

School-form resources (all `IsAuthenticated` + `SchoolScopedViewSetMixin`)

Method	Endpoint	Purpose	Serializer	Filters	Special
CRUD	`kit-deliveries/`	Kit handover	`KitDeliverySerializer`	`school`, `date_of_delivery`	2nd create per school → 403 (non-admin)
CRUD	`sl-selections/`	SL rows	`SLSelectionSerializer`	`school`, `grade`, `section`, `sl_status`	
POST	`sl-selections/bulk-submit/`	Many SLs for a grade/section	in: `SLSelectionBulkSubmitSerializer`, out: list of `SLSelectionSerializer`		201; duplicate section → 400 (non-admin)
CRUD	`student-headcounts/`	Form 3	`StudentHeadcountSerializer`	`school`, `grade`, `section`, `extraction_status`, `count_validation`	create/update add `mismatch_warning`
CRUD	`session-observations/`	Session observed	`SessionObservationSerializer`	`school`, `grade`, `section`, `unit`	

Teams (`IsAdminOrIIFStaffOrReadOnly`, scoped)

Method	Endpoint	Serializer	Filters
CRUD	`clusters/`	`ClusterSerializer` (nested teams→students)	`school`, `grade`, `section`
CRUD	`teams/`	`TeamSerializer` (nested students)	`cluster`
CRUD	`students/`	`StudentSerializer`	`team`

Observations — read models (`IsAuthenticated`, not paginated)

Method	Endpoint	Query params	Response
GET	`session-tracker/`	`unit` (default 1), `grade` (default 6)	list of `{school_id, school__school_code, section, session_no, date, observed_by}`
GET	`session-progress/`	—	list of `{school_id, school__school_code, unit, sessions_completed}`

InquiBuddy (`IsAuthenticated`, scoped by `team__cluster__school_id`)

Method	Endpoint	Purpose	Response
GET	`inquibuddy-submissions/`	List (P); filters `team__cluster`, `status`, `team__cluster__school`, `team__cluster__grade`, `team__cluster__section`	submissions
GET	`inquibuddy-submissions/{id}/`	Detail	submission
POST	`inquibuddy-submissions/{id}/upload-photo/`	multipart `file`	submission
POST	`inquibuddy-submissions/{id}/upload-audio/`	multipart `file`	submission
POST	`inquibuddy-submissions/{id}/generate-feedback/`	Enqueue AI job	submission (`status: "processing"`)
POST	`inquibuddy-submissions/generate-feedback-bulk/`	body `school`, `grade`, `section`	`{"queued": n}`
GET	`inquibuddy-submissions/{id}/report/`	Download team PDF	PDF attachment
GET	`inquibuddy-submissions/report-all/?school=&grade=&section=`	Download combined PDF	PDF attachment

Files (`IsAuthenticated`)

Method	Endpoint	Purpose	Serializer
POST	`files/`	multipart upload (`file`, `entity_type`, `entity_id`)	in `FileUploadSerializer` → out same (id, file, entity_type, entity_id)
GET	`files/{id}/`	Metadata + URL	`FileSerializer`

Reports

Method	Endpoint	Permission	Response
GET	`reports/summary/`	IsAuthenticated (scoped)	`{"kpis": {...} or null, "schools": [SchoolProgress...]}` (not paginated)

11. DRF request lifecycle

Example: a school user calls `POST /api/v1/kit-deliveries/` with `{"date_of_delivery":"2026-06-01","grade_6_kit":true}`.

```
HTTP request
   ▼
Django (WSGIHandler)  – builds HttpRequest
   ▼
Middleware (request phase) – Security → Cors → Session → Common → Csrf → Auth → Messages → XFrame
   ▼
URL resolver  – config/urls.py: "api/v1/" → include(api_v1) → apps.kits.urls → router
                → KitDeliveryViewSet.as_view({"get":"list","post":"create"})
   ▼
APIView.dispatch()
   ├─ initialize_request()  → wraps HttpRequest in DRF Request (parsers: JSON, form, multipart)
   ├─ initial()
   │    ├─ perform_authentication() → JWTAuthentication: reads "Authorization: Bearer ...",
   │    │                             verifies signature+expiry, loads User by "user_id" claim
   │    ├─ check_permissions()      → IsAuthenticated.has_permission()   (fail → 401/403)
   │    └─ check_throttles()        → ScopedRateThrottle: view has no throttle_scope → skip
   ├─ handler = self.create
   │    ├─ get_serializer(data=request.data)
   │    │     └─ SchoolScopedViewSetMixin: copies data, sets data["school"] = user.school_id
   │    ├─ serializer.is_valid(raise_exception=True)   → field validation (400 on error)
   │    ├─ perform_create(serializer)
   │    │     └─ KitDeliveryViewSet: "already has a delivery?" → 403
   │    │     └─ mixin: serializer.save(school=user.school)
   │    │           └─ ModelSerializer.create() → KitDelivery.objects.create(...)  → ORM → INSERT
   │    └─ Response(serializer.data, status=201)
   └─ finalize_response() → content negotiation (JSONRenderer / BrowsableAPIRenderer)
   ▼
Middleware (response phase, reverse order) – adds CORS headers, X-Frame-Options, etc.
   ▼
HTTP 201 {"id": 12, "school": 3, "date_of_delivery": "2026-06-01", ...}
```

What happens at each stage: 1. **Middleware** — generic cross-cutting behaviour. CORS headers are added here, before DRF ever sees the request. 2. **URL resolver** — matches the path; the router maps HTTP method → viewset action. 3. **Authentication** — decides *who* you are (`request.user`). An invalid/expired token raises `AuthenticationFailed` → 401. No header → `AnonymousUser`. 4. **Permissions** — decides *whether* you may call this endpoint. `has_permission()` runs for every request; `has_object_permission()` runs inside `get_object()` (not used by custom classes here). 5. **Throttling** — rate limiting (only the login view). 6. **Queryset** — `get_queryset()` narrows data to what the user may see; `filter_queryset()` applies `?field=value`. 7. **Serializer / validation** — input → Python → validated_data. 8. **Business logic** — `perform_create`, custom actions, services. 9. **ORM → database**. 10. **Serializer → Response** — Python → JSON; exceptions anywhere above are converted to error responses by DRF's exception handler.

12. Authentication

12.1 What exists

Feature	Exists?	Where
Login	Yes	`POST /api/v1/auth/login/` → `LoginView` (SimpleJWT `TokenObtainPairView`)
Registration (self sign-up)	**No**	Accounts are created by an admin (`POST /users/`), the Django admin, or `import_master_users`.
Password hashing	Yes	Django's `set_password()` → default `PBKDF2PasswordHasher` (no `PASSWORD_HASHERS` setting, so argon2 is **not** active despite the model docstring mentioning it).
JWT access tokens	Yes	15 min lifetime, HS256, signed with `SECRET_KEY`.
Refresh tokens	Yes	7 days, rotated on each refresh (`/auth/refresh/`).
Token blacklisting	**Configured but ineffective**	`token_blacklist` app not installed.
Logout	**No**	No endpoint; clients just discard tokens.
Current user	Yes	`GET /api/v1/auth/me/`
Password change/reset via API	**No**	Admin can set passwords in Django admin.

12.2 How login works step by step

1. Client sends `POST /api/v1/auth/login/` with `{"username": "...", "password": "..."}`.
2. `ScopedRateThrottle` checks `throttle_scope="login"`: more than 10 requests/min from the same IP → 429.
3. `TokenObtainPairSerializer.validate()` calls `django.contrib.auth.authenticate(username=..., password=...)`. Django loads the user, re-hashes the password with the stored salt/iterations and compares. It also rejects `is_active=False` users.
4. On failure → 401 `{"detail": "No active account found with the given credentials"}`.
5. On success → `get_token(user)` (overridden in `ThinkAndMakeTokenObtainPairSerializer`) creates a `RefreshToken` and adds custom claims `role` and `school_id`. The access token is derived from it and inherits those claims.
6. Response `{"refresh": "<jwt>", "access": "<jwt>"}`. `update_last_login` is only called if SimpleJWT's `UPDATE_LAST_LOGIN` is enabled (default `False`), so `last_login` isn't updated by API logins.

A decoded access token payload looks like:

```
{
  "token_type": "access",
  "exp": 1790000000,
  "iat": 1789999100,
  "jti": "5f1c...",
  "user_id": "17",
  "role": "school",
  "school_id": 3
}
```

(`user_id` type depends on SimpleJWT version; the claims `role` / `school_id` are the project's.)

Why add claims? So the client knows the user's role/school without an extra `/auth/me/` round-trip. **The server never trusts these claims for authorization** — `JWTAuthentication` loads the real `User` row, and permission checks read `request.user.role` from the database.

12.3 How each request is authenticated

`JWTAuthentication.authenticate(request)`: 1. Reads `Authorization` header; expects `Bearer <token>`. 2. Validates signature, `exp`, and `token_type == "access"`. 3. Looks up `User` by the `user_id` claim; rejects inactive users. 4. Returns `(user, token)` → DRF sets `request.user` and `request.auth`.

If no header is present, DRF sets `request.user = AnonymousUser`; then `IsAuthenticated` fails and — because `JWTAuthentication` defines `authenticate_header()` — DRF returns **401** (with `WWW-Authenticate: Bearer realm="api"`) instead of 403. The test `test_unauthenticated_request_rejected` asserts this.

12.4 Refresh

`POST /api/v1/auth/refresh/ {"refresh": "<token>"}` → validates the refresh token → returns a new access token and (because of rotation) a new refresh token. Clients should store the new refresh token.

12.5 Authentication vs Authorization

- **Authentication** = *who are you?* (JWT → `request.user`).
- **Authorization** = *what may you do?* (permission classes + queryset scoping + business rules like "Form 1 already submitted").

12.6 Realistic request

```
GET /api/v1/schools/ HTTP/1.1
Host: localhost:8000
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

For a school user linked to school id 3 the response contains exactly one result — school 3 — even though the database has 50 schools.

12.7 Django admin authentication

`/admin/` uses Django's **session** login (username/password form, CSRF-protected). Only users with `is_staff=True` can enter; `import_master_users` sets `is_staff` and `is_superuser` for `admin`-role accounts only.

13. Permissions and roles

13.1 Roles that exist

Defined in `User.ROLE_CHOICES` and mirrored as constants in `apps/core/permissions.py`:

Role	Value	Who	Typical login
Admin	`admin`	IIF administrator	email
IIF Staff	`iif_staff`	Field staff who visit schools	email
School	`school`	One account per school	school username (e.g. `TMDA...`)

No other roles exist. Django's `is_staff` / `is_superuser` flags only govern the Django admin, not the API.

13.2 Permission classes (`apps/core/permissions.py`)

Class	`has_permission` returns True when	Used by
`IsAdmin`	authenticated **and** `role == "admin"`	`UserViewSet`
`IsAdminOrIIFStaff`	authenticated and role in (admin, iif_staff)	**not used anywhere**
`IsSchoolUser`	authenticated and role == school	**not used anywhere**
`IsAdminOrIIFStaffOrReadOnly`	authenticated **and** (method in `SAFE_METHODS` = GET/HEAD/OPTIONS **or** role in (admin, iif_staff))	programs, schools (+ children), teams
DRF `IsAuthenticated`	logged in	school-form viewsets, InquiBuddy, files, observations APIViews, reports, `MeView`

13.3 Effective permission matrix (based on the code)

Resource	Admin	IIF Staff	School
Users (`users/`)	full CRUD	✗ 403	✗ 403
Partners / Instances	full	full	read all
Schools	full; may edit/resubmit after lock	full; can't PATCH/PUT after Form 1 submitted (403) or resubmit Form 1/2 (400)	read **own** school only; no writes
Grade enrollments / teachers / schedules	full	full	read own
Kit deliveries	full; many per school	CRUD; only one create per school (403)	CRUD **own**; one create (403); school forced
SL selections	full; bulk-submit repeatable	CRUD; bulk-submit once per section (400)	CRUD own; school forced
Student headcounts	full	full	CRUD own; school forced
Session observations	full	full	CRUD own; school forced
Clusters / Teams / Students	full	full	read own
InquiBuddy submissions & actions	all	all	own teams only (all actions)
Files	upload; retrieve **any** id	same	same (not scoped!)
Session tracker / progress	all rows	all rows	own rows
Reports summary	all + KPIs	all + KPIs	own row, `kpis: null`

13.4 How the backend prevents unauthorized access (3 layers)

1. **Permission class** — rejects the whole request early (401/403).
2. **Queryset scoping** — `scope_queryset_to_role()` in `get_queryset()`. A school user asking for another school's row gets **404** (the row is "invisible"), which also avoids leaking that the row exists.
3. **Write enforcement** — `SchoolScopedViewSetMixin` overwrites `school` in the input and on save; `bulk_submit` ignores the body's `school` for school users.

```
def scope_queryset_to_role(queryset, request, school_field="school"):
    user = request.user
    if user.role == SCHOOL:
        return queryset.filter(**{school_field: user.school_id})
    return queryset
```

Note that the function is **allow-by-default for non-school roles**: admin and iif_staff see everything (the docstring says per-instance assignment for IIF staff "can be layered on later" — not implemented).

Observations (authorization gaps visible in code): - `GET /files/{id}/` **is not scoped.** Any authenticated user (including a school account) can fetch metadata and a (signed) URL for **any** file id — e.g. another school's photos or generated PDFs — by iterating ids. This is an IDOR risk. See §36. - **File FKs aren't ownership-checked.** A school user can set `school_photo` / `delivery_proof_photo` /etc. to any existing `File` id. - **"Contact admin to make changes" locks only guard create.** For kits, a school or IIF user can still `PATCH` / `DELETE` the existing delivery. For SL selections, a non-admin can `DELETE` rows (or `PATCH` them) and then bulk-submit again. - `scope_queryset_to_role` **assumes an authenticated user.** It's only reached after `IsAuthenticated`-style permissions, so that's fine today, but `AnonymousUser` has no `role` attribute.

14. SchoolScopedViewSetMixin in depth

File: `apps/core/mixins.py`. Used by `KitDeliveryViewSet`, `SLSelectionViewSet`, `StudentHeadcountViewSet`, `SessionObservationViewSet`.

```
class SchoolScopedViewSetMixin:
    school_field = "school"

    def get_queryset(self):
        return scope_queryset_to_role(super().get_queryset(), self.request, school_field=self.school_field)

    def get_serializer(self, *args, **kwargs):
        if self.request.user.is_authenticated and self.request.user.role == SCHOOL and "data" in kwargs:
            data = kwargs["data"]
            data = data.copy() if hasattr(data, "copy") else dict(data)
            data[self.school_field] = self.request.user.school_id
            kwargs["data"] = data
        return super().get_serializer(*args, **kwargs)

    def perform_create(self, serializer):
        if self.request.user.role == SCHOOL:
            serializer.save(**{self.school_field: self.request.user.school})
        else:
            serializer.save()
```

14.1 Why the mixin exists

Four viewsets hold "school form" data where the **school role can write**. Every one needs identical rules: read only your rows, write only to your school. A mixin placed **before** `ModelViewSet` in the class bases (`class KitDeliveryViewSet(SchoolScopedViewSetMixin, viewsets.ModelViewSet)`) overrides the generic methods and calls `super()` to reach DRF's implementation (Python's MRO: mixin → ModelViewSet → ... → GenericAPIView).

14.2 `school_field`

Name of the FK to `School` on the model. Default `"school"`. A subclass could set `"team__cluster__school"` etc., but note `get_serializer` and `perform_create` use it as an **input field name** too, so it only works cleanly when it's a direct field.

14.3 `get_queryset()`

Called by `list()` and by `get_object()`. Takes the class's `queryset` (e.g. `KitDelivery.objects.select_related("school")`), then filters by `school_id` for school users. Everything downstream (retrieve, update, delete, filters, pagination) works on the scoped set, so **a school user can never reach another school's row** — they get 404.

14.4 `get_serializer()` — injecting school *before* validation

Called by DRF's `create()` / `update()` with `data=request.data`. For school users it copies the data (a `QueryDict` from multipart/form requests is immutable; `.copy()` gives a mutable copy; a plain dict from JSON also has `.copy()`) and **overwrites** `school` with the user's own school id.

Why here instead of only in `perform_create()`? Because of `UniqueTogetherValidator`. `StudentHeadcount` has a unique constraint on `(school, grade, section)` and `SessionObservation` on `(school, grade, section, unit, session_no)`. DRF's auto-generated validator: - makes `school` effectively **required** in the input, and - runs during `is_valid()` — before `perform_create()` is ever called.

If a school client omitted `school`, validation would fail with "This field is required" even though the server intends to fill it in. And if the server only set it later in `perform_create`, the uniqueness check would have run against the wrong (client-supplied or missing) school. Injecting first means the uniqueness check runs against the **correct** school.

This also applies to **updates** (PUT/PATCH): a school user can't move an existing row to another school because `school` is overwritten in the incoming data.

14.5 `perform_create()` — enforcing ownership on save

Even though `school` was injected, `perform_create` passes `school=request.user.school` again to `serializer.save()`. Extra kwargs to `save()` override `validated_data`, so this is a second, independent guarantee (defence in depth). For admin/IIF staff the client must send `school` explicitly.

14.6 Why school filtering is security-sensitive

The system holds data for many schools (student names, principal phone numbers, photos). School accounts belong to people outside the organisation. If one school could read or write another's data it would be both a privacy breach and a data-integrity problem (fake submissions for another school, locking its forms).

14.7 Why request body / query params must not be trusted

Everything in the request is attacker-controlled: body, query string, headers other than a verified token. A malicious or buggy client can send any `school` id. The only trustworthy identity is `request.user`, which comes from a cryptographically verified token and a database lookup.

14.8 Attack example

A School A user (school id 1) sends:

```
POST /api/v1/kit-deliveries/
Authorization: Bearer <School A token>
Content-Type: application/json

{"school": 2, "date_of_delivery": "2026-06-01", "grade_6_kit": true}
```

What happens: 1. `get_serializer()` replaces `"school": 2` with `"school": 1`. 2. Validation runs against school 1. 3. `perform_create()` saves with `school=<School 1>`. 4. Result: the delivery is recorded for School A; School B is untouched. (`test_school_user_cannot_spoof_another_schools_delivery` proves this.)

Reading: `GET /api/v1/kit-deliveries/?school=2` → `get_queryset()` already filters to school 1, then the `?school=2` filter is applied on top → empty list. `GET /api/v1/kit-deliveries/<id of B's row>/` → 404.

14.9 Edge cases

- **IIF staff/admin omitting `school`:** `school` is `required=False` in these serializers, so for models without a unique constraint (kits, SL) validation passes and `serializer.save()` inserts `school_id=NULL` → database `IntegrityError` (NOT NULL) → HTTP 500. **Recommended improvement:** require `school` for non-school roles in `validate()`.
- **School user whose `school` is `None`:** injection sets `school=None`; reads return nothing.

---

15. CORS

15.1 What CORS is

Cross-Origin Resource Sharing is a browser rule. A page loaded from origin `http://localhost:3000` (scheme + host + port) may not read responses from `http://localhost:8000` unless the server explicitly allows it with `Access-Control-Allow-Origin`. For "non-simple" requests (e.g. JSON body, `Authorization` header) the browser first sends a **preflight** `OPTIONS` request asking for permission.

15.2 Why browsers enforce it

To stop a malicious website from using a visitor's browser (and its cookies) to read data from another site. It is **not** a server-side security boundary — curl, Postman and the **mobile app** ignore CORS completely. That's why the settings comment says "the mobile app is a native HTTP client".

15.3 This project's configuration

Setting	Value	Effect
`CORS_ALLOWED_ORIGINS`	env CSV, default `["http://localhost:3000"]`	Only these origins get `Access-Control-Allow-Origin` echoed back.
`CORS_ALLOW_CREDENTIALS`	`True`	Adds `Access-Control-Allow-Credentials: true`, letting the browser send/receive cookies on cross-origin requests.
middleware	`CorsMiddleware` near the top	Handles preflights and adds headers.

Default headers allowed by django-cors-headers include `authorization` and `content-type`, so the web client's `Authorization: Bearer` header works.

15.4 Cookies and credentials

The API authenticates with a **header**, not a cookie, so `CORS_ALLOW_CREDENTIALS=True` isn't needed by the JWT flow. It would matter only if the frontend sent cookies (e.g. `fetch(..., {credentials: "include"})`). With credentials allowed you must **never** use a wildcard origin (the library forbids `*` with credentials anyway).

15.5 Development vs production

- Dev: `http://localhost:3000` (Next.js dev server).
- Prod: set `CORS_ALLOWED_ORIGINS=https://your-web-domain` exactly (scheme + host, no trailing slash, no path).
- If the Django admin is served on a different domain than the API behind HTTPS, `CSRF_TRUSTED_ORIGINS` would also be needed (not set).

16. Throttling

16.1 Configuration

```
"DEFAULT_THROTTLE_CLASSES": ("rest_framework.throttling.ScopedRateThrottle",),
"DEFAULT_THROTTLE_RATES": {"login": "10/min"},
```

and in `LoginView`: `throttle_scope = "login"`.

16.2 How it works

- `ScopedRateThrottle` is applied to every view, but **only throttles views that define `throttle_scope`** (`get_cache_key` returns `None` otherwise). Only `LoginView` does.
- Cache key = scope + identity. Identity = `user.pk` if authenticated, else client IP (`REMOTE_ADDR`, or `X-Forwarded-For` depending on `NUM_PROXIES`). Login requests are anonymous, so it's **per IP**.
- The throttle stores a list of request timestamps in Django's **cache**. When there are already 10 timestamps in the last 60 seconds it raises `Throttled` → **HTTP 429** `{"detail": "Request was throttled. Expected available in N seconds."}` with a `Retry-After` header.

16.3 Why login endpoints are throttled

To slow down **brute-force** and **credential-stuffing** attacks. The legacy system used short, guessable PIN-style passwords, so this matters more than usual.

16.4 Caveats in this project

- **No `CACHES` setting** → default `LocMemCache`, which is **per process**. With N gunicorn workers an attacker effectively gets 10×N attempts/min, and counters reset on restart. **Recommended improvement:** point `CACHES` at Redis (already in the stack).
- Behind a reverse proxy all clients may appear to share one IP unless `NUM_PROXIES` is configured — the whole site could be throttled together.
- `/auth/refresh/` is not throttled (low risk; it requires a valid refresh token).

17. Celery

17.1 Concepts

Term	Meaning
Celery	A Python library for running functions ("tasks") asynchronously in separate worker processes.
Task	A function decorated with `@shared_task` (or `@app.task`). Calling `task.delay(args)` doesn't run it — it sends a message.
Broker	The message queue that carries task messages from Django to workers. Here: **Redis**.
Worker	A long-running process (`celery -A config worker`) that pulls messages from the broker and executes tasks.
Result backend	Where a task's return value/state is stored. Here: Redis. The project never reads results (`AsyncResult` isn't used); clients poll the **database** (`InquibuddySubmission.status`) instead.
Celery Beat	A scheduler process that sends task messages on a timetable. It doesn't execute tasks itself.

17.2 The Celery app (`config/celery.py` + `config/__init__.py`)

```
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")
app = Celery("think_and_make")
app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()
```

- `config_from_object(..., namespace="CELERY")` reads every Django setting starting with `CELERY_` (`CELERY_BROKER_URL` → `broker_url`, etc.).
- `autodiscover_tasks()` imports `tasks.py` from every installed app → finds `apps.inquibuddy.tasks` and `apps.reports.tasks`.
- `config/__init__.py` does `from config.celery import app as celery_app` so the app is created whenever Django starts. That's what makes `@shared_task` functions bind to this app, and it's why the CLI is `celery -A config ...` (Celery finds `config.celery_app`).

17.3 Settings

Setting	Value	Meaning
`CELERY_BROKER_URL`	`redis://localhost:6379/0`	Redis host/port/database 0 for messages.
`CELERY_RESULT_BACKEND`	`redis://localhost:6379/0`	Same Redis DB for results.
`CELERY_ACCEPT_CONTENT`	`["json"]`	Worker refuses non-JSON messages (e.g. pickle) — safer, since pickle can execute code on deserialisation.
`CELERY_TASK_SERIALIZER`	`"json"`	Task arguments are JSON-encoded. That's why tasks receive `submission_id: int`, not a model instance.
`CELERY_RESULT_SERIALIZER`	`"json"`	Results JSON-encoded.
`CELERY_TIMEZONE`	`"Asia/Kolkata"`	Beat interprets `crontab()` in IST.

17.4 Tasks in the project

`apps.inquibuddy.tasks.generate_ai_feedback(self, submission_id)`

Decorator: `@shared_task(bind=True, max_retries=2, default_retry_delay=30)`. - `bind=True` → first arg `self` is the task instance, giving access to `self.retry()`. - `max_retries=2` → at most 3 attempts total. - `default_retry_delay=30` → wait 30 s between attempts.

Steps: 1. Load submission (`select_related("team")`). If it was deleted → `DoesNotExist` raised (task fails). 2. Find the photo `SubmissionFile`. If none → log a warning and return (**status stays `processing`**). 3. Find optional audio. 4. Read bytes from storage (`photo.file.file.read()`), call `lib.gemini_client.generate_feedback_questions(...)`. 5. On any exception → `raise self.retry(exc=exc)` (re-queued in 30 s). After the final retry the original exception propagates and the task fails — **status remains `processing`** (see Observation). 6. Delete old questions; `bulk_create` new `AIFeedbackQuestion` rows with `order` 1..n. 7. Set `status="evaluated"`, `evaluation_count += 1`, `save(update_fields=[...])`. 8. `record_audit(user=None, action="generate_ai_feedback", ..., after={"question_count": n})`.

Called via `.delay(submission.id)` from `generate_feedback` and `generate_feedback_bulk` actions. `.delay(*args)` is shorthand for `.apply_async(args=args)`; `apply_async` would allow options such as `countdown`, `eta`, `queue` (not used here).

Observation: the code comment says retries are there so status isn't "left stuck on processing", but when retries are exhausted (or no photo exists) nothing resets status. The client would poll forever. **Recommended improvement:** on final failure set status back to `pending` (or add a `failed` status) — e.g. check `self.request.retries >= self.max_retries` before re-raising.

`apps.reports.tasks.reconcile_school_progress()`

Plain `@shared_task`, no retries. Imports and calls `reconcile_all_schools()` (import inside the function avoids importing app models at module-load time). Triggered by Beat (below) — can also be run manually via the management command.

17.5 Worker vs Beat

	Celery Worker	Celery Beat
Does	Executes tasks	Sends task messages on a schedule
How many	Can scale to many processes/machines	Must run **exactly one** instance (otherwise duplicate runs)
Needed for	InquiBuddy AI + nightly job execution	Nightly job trigger only
Command	`celery -A config worker -l info`	`celery -A config beat -l info`

Without a worker, `generate-feedback` still returns `processing` immediately but nothing happens (stated in `backend/README.md`). Without Beat, the nightly reconciliation never runs (but `GET schools/{id}/progress/` still creates a row on first read, and `reconcile_school_progress` can be run by hand).

18. Celery Beat schedules

```
CELERY_BEAT_SCHEDULE = {
    "reconcile-school-progress-nightly": {
        "task": "apps.reports.tasks.reconcile_school_progress",
        "schedule": crontab(hour=2, minute=0),
    },
}
```

Item	Value
Schedule name	`reconcile-school-progress-nightly`
Task	`apps.reports.tasks.reconcile_school_progress` (name = module path + function name, which is `shared_task`'s default)
Schedule	`crontab(hour=2, minute=0)` — unspecified fields default to `*`: **every day at 02:00**, every month, every weekday, in `CELERY_TIMEZONE` = **Asia/Kolkata**.
What it does	For every school, recompute its `SchoolProgress` row and mismatch flags.
Data it changes	Upserts `reports_schoolprogress` (one row per school); may update `headcounts_studentheadcount.count_validation` (`ok` ⇄ `mismatch`).
Dependencies	Redis broker, a running worker, a running beat process, DB access.
Failure scenarios	Worker/beat/Redis down → the run is simply missed (no catch-up); a single failing school is logged via `logger.exception` and skipped (tested); an unexpected error outside the loop fails the task (no retries configured). Because reconciliation is idempotent, the next run fixes everything.

The settings comment notes the schedule is static in code (no `django-celery-beat` database scheduler), so changing the time requires a deploy.

19. Redis

- **What:** an in-memory key-value data store, very fast, supports lists/sets/pub-sub. Commonly used as a cache, queue or message broker.
- **Why here:** solely as Celery's **broker** and **result backend**. It is **not** configured as Django's cache (no `CACHES`) and not used for sessions.
- **Broker vs result backend:** the broker holds *pending messages* (tasks waiting to run; Celery uses Redis lists). The result backend stores *outcomes* (keys like `celery-task-meta-<task_id>` with state and return value, expiring after a day by default). Both point to the same DB here, which is fine at this scale.
- **Redis URL anatomy:** `redis://[:password@]host:port/db` → `redis://localhost:6379/0` = local server, default port, database **0**. Redis has 16 numbered logical databases (0–15) by default; separating e.g. broker `/0`, results `/1`, cache `/2` avoids key collisions and allows flushing one without the others.
- **Development:** run Redis locally (Docker `docker run -p 6379:6379 redis:7`, WSL, or a Windows-compatible Redis server). The `redis` Python client (`redis==8.1.0`) and `kombu` are in `requirements.txt`.
- **Production considerations:** use a managed Redis with a password and TLS (`rediss://`), set `maxmemory-policy noeviction` for a broker (evicting queued messages loses tasks), monitor queue length, and never expose Redis publicly.

20. File storage

20.1 Static files vs media files

- **Static files:** shipped with the code — admin CSS/JS, `apps/core/static/admin/theme.css`. Served under `STATIC_URL="static/"`. In production they're collected with `collectstatic` into `STATIC_ROOT` (not configured here).
- **Media files:** uploaded by users or generated at runtime — school photos, kit proofs, InquiBuddy photos/audio, generated PDFs. Stored through Django's **default storage**.

20.2 Local media storage (default, `USE_S3_STORAGE=False`)

- `MEDIA_ROOT = backend/media`, `MEDIA_URL = "/media/"`.
- `config/urls.py` serves them via `django.conf.urls.static.static()` **only if `DEBUG` and not S3**. Django's dev static view is not for production.
- `File.file.url` → `/media/<entity_type>/<entity_id>/<name>`; `FileSerializer.get_url` turns it into `http://host/media/...`.
- Tests override `MEDIA_ROOT` to a temp dir (`conftest._isolated_media_root`).

20.3 S3-compatible storage / Supabase Storage (`USE_S3_STORAGE=True`)

Supabase Storage exposes an **S3-compatible API**, so the standard AWS tooling works: `django-storages`' `S3Boto3Storage` backend uses `boto3` to talk to it.

Setting	Meaning
`STORAGES["default"] = S3Boto3Storage`	All `FileField`s save to the bucket. (Django ≥4.2 `STORAGES` dict.)
`STORAGES["staticfiles"] = StaticFilesStorage`	Static files stay on local disk.
Bucket — `AWS_STORAGE_BUCKET_NAME`	Container for objects, default `think-and-make`. Should be **private**.
Access key / Secret key — `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`	Credentials generated in Supabase (Storage → S3 access keys). Full access to the bucket — server-side only.
Endpoint — `AWS_S3_ENDPOINT_URL`	Supabase's S3 endpoint (`https://<project-ref>.supabase.co/storage/v1/s3`). Empty would mean real AWS S3.
Region — `AWS_S3_REGION_NAME`	Region used for request signing, default `ap-south-1`. Must match the Supabase project's region.
`AWS_DEFAULT_ACL = None`	Don't send ACL headers; objects inherit the bucket policy (private).
`AWS_QUERYSTRING_AUTH = True`	`file.url` returns a **signed URL** (query-string authentication) instead of a plain public URL.
`AWS_QUERYSTRING_EXPIRE = 600`	Signed URL valid for **10 minutes**.
`AWS_S3_ADDRESSING_STYLE = "path"`	URLs like `https://endpoint/bucket/key` rather than virtual-host style `https://bucket.endpoint/key`. Supabase requires path style.

20.4 Signed URLs explained

A signed URL contains the object key plus query parameters (`X-Amz-Algorithm`, `X-Amz-Credential`, `X-Amz-Date`, `X-Amz-Expires=600`, `X-Amz-Signature`). The signature is an HMAC computed with the secret key; the storage server recomputes it and checks expiry. Anyone with the URL can download the file until it expires — so the **API** decides who gets a URL, and the URL itself is a short-lived capability. Consequence: clients must fetch a fresh URL (e.g. `GET /files/{id}/`) rather than caching URLs long-term.

20.5 How files flow in this project

1. **Generic upload:** `POST /files/` (multipart) → `FileViewSet.perform_create` saves `uploaded_by`, `mime_type` (from the upload's `content_type`), `size`. Storage path via `upload_path()` → `"<entity_type>/<entity_id or 'misc'>/<filename>"`. Client then puts the returned `id` into e.g. `school_photo`.
2. **InquiBuddy upload:** `POST inquibuddy-submissions/{id}/upload-photo/` → `services.attach_submission_file()` creates the `File` and a `SubmissionFile` in one call.
3. **Generated PDFs:** `services.get_or_create_individual_report()` wraps bytes in `ContentFile` and saves a `File` (`entity_type="FeedbackReport"`).
4. **Download:** `FileResponse(report.file.file.open("rb"), as_attachment=True, filename=...)` streams the PDF **through Django** (not via signed URL).

Observations: - Old files are never deleted from storage: re-uploading a photo deletes the `SubmissionFile` row but leaves the `File` row and the stored object. Django doesn't delete files when rows are deleted. - No validation of upload size or type (a client can upload any content type). Django's `DATA_UPLOAD_MAX_MEMORY_SIZE` doesn't limit file uploads (files stream to temp disk above `FILE_UPLOAD_MAX_MEMORY_SIZE`). - `entity_type` / `entity_id` are client-supplied and become folder names. Django's storage layer rejects path traversal (`..`), but arbitrary folder names are accepted. **Recommended:** whitelist `entity_type`. - In local mode, `idea_photo_url` passed to the PDF template is a relative `/media/...` URL and `HTML(string=html)` is called without `base_url`, so WeasyPrint likely can't load the image locally; with S3 enabled the URL is absolute and signed, which WeasyPrint can fetch. (Not covered by tests — verify.)

21. URLs and routing

21.1 Root URLconf (`config/urls.py`)

```
api_v1 = [path("", include("apps.accounts.urls")), path("", include("apps.programs.urls")), ...]  # 11 apps
urlpatterns = [path("admin/", admin.site.urls), path("api/v1/", include(api_v1))]
if settings.DEBUG and not getattr(settings, "USE_S3_STORAGE", False):
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

- `path(route, view_or_include)` matches a URL prefix; `include()` hands the rest of the URL to another URLconf.
- All apps are included at `""` under `api/v1/`, so their routes share one flat namespace. Route names must not collide across apps.
- The `v1` prefix allows a future `v2` without breaking existing clients.

21.2 Routers

Every app except `reports` uses DRF's **`DefaultRouter`**:

```
router = DefaultRouter()
router.register("kit-deliveries", KitDeliveryViewSet, basename="kit-delivery")
urlpatterns = router.urls
```

`register(prefix, viewset, basename)` generates:

URL	Name	Methods → actions
`kit-deliveries/`	`kit-delivery-list`	GET→list, POST→create
`kit-deliveries/{pk}/`	`kit-delivery-detail`	GET→retrieve, PUT→update, PATCH→partial_update, DELETE→destroy
`kit-deliveries/{pk}/<url_path>/`	`kit-delivery-<url_name>`	`@action(detail=True)`
`kit-deliveries/<url_path>/`		`@action(detail=False)`

`basename` is required when the viewset has no `queryset` attribute (e.g. `SchoolViewSet` defines `get_queryset()` only). `DefaultRouter` (vs `SimpleRouter`) also adds an **API root** view at `""` listing registered routes and optional format suffixes (`.json`). Since each app has its own `DefaultRouter` mounted at `""`, the first one (accounts) wins at `/api/v1/` — the browsable root only lists `users`. `SimpleRouter` isn't used.

`@action` example from `SchoolViewSet`:

```
@action(detail=True, methods=["post"], url_path="submit-form1")
def submit_form1(self, request, pk=None): ...
```

→ `POST /api/v1/schools/{pk}/submit-form1/`.

Plain `path()` routes: `auth/login/`, `auth/refresh/`, `auth/me/`, `session-tracker/`, `session-progress/`, `reports/summary/`.

21.3 Complete API routing map

```
/admin/                                        Django admin (unfold)
/media/<path>                                  dev only (DEBUG and not S3)
/api/v1/
├── auth/login/                          POST
├── auth/refresh/                        POST
├── auth/me/                             GET
├── users/  users/{id}/                  CRUD (admin)
├── partners/  partners/{id}/            CRUD
├── instances/ instances/{id}/           CRUD
├── schools/   schools/{id}/             CRUD
│   ├── {id}/submit-form1/               POST
│   ├── {id}/submit-form2/               POST
│   └── {id}/progress/                   GET
├── school-grade-enrollments/[{id}/]     CRUD
├── school-teachers/[{id}/]              CRUD
├── session-schedules/[{id}/]            CRUD
├── kit-deliveries/[{id}/]               CRUD
├── sl-selections/[{id}/]                CRUD
│   └── bulk-submit/                     POST
├── student-headcounts/[{id}/]           CRUD (+ mismatch_warning)
├── clusters/[{id}/]                     CRUD
├── teams/[{id}/]                        CRUD
├── students/[{id}/]                     CRUD
├── session-observations/[{id}/]         CRUD
├── session-tracker/                     GET ?unit&grade
├── session-progress/                    GET
├── inquibuddy-submissions/[{id}/]       GET (read-only)
│   ├── {id}/upload-photo/               POST multipart
│   ├── {id}/upload-audio/               POST multipart
│   ├── {id}/generate-feedback/          POST
│   ├── generate-feedback-bulk/          POST {school,grade,section}
│   ├── {id}/report/                     GET → PDF
│   └── report-all/                      GET ?school&grade&section → PDF
├── files/  files/{id}/                  POST, GET
└── reports/summary/                     GET
```

22. Pagination, filtering, searching, ordering

22.1 Pagination

`PageNumberPagination`, `PAGE_SIZE=50`, applied to every `list` action of generic views/viewsets. Query parameter `?page=N`. There's no `page_size_query_param`, so clients can't change the page size.

```
GET /api/v1/schools/?page=2
{
  "count": 57,
  "next": null,
  "previous": "http://localhost:8000/api/v1/schools/",
  "results": [ ... up to 50 schools ... ]
}
```

Not paginated: `session-tracker/`, `session-progress/`, `reports/summary/` (plain `APIView`s returning full lists), and the list returned by `sl-selections/bulk-submit/`. Clients must read `results` from paginated endpoints but a raw list/object from these.

22.2 Filtering (django-filter)

`DjangoFilterBackend` + `filterset_fields` = exact-match filters. Each listed field becomes a query parameter; related lookups use `__`:

Endpoint	Filters
`schools/`	`instance`, `instance__partner`, `school_code`, `district`
`school-grade-enrollments/`	`school`, `grade`
`school-teachers/`	`school`
`session-schedules/`	`school`, `grade`
`kit-deliveries/`	`school`, `date_of_delivery`
`sl-selections/`	`school`, `grade`, `section`, `sl_status`
`student-headcounts/`	`school`, `grade`, `section`, `extraction_status`, `count_validation`
`clusters/`	`school`, `grade`, `section`
`teams/`	`cluster`
`students/`	`team`
`session-observations/`	`school`, `grade`, `section`, `unit`
`inquibuddy-submissions/`	`team__cluster`, `status`, `team__cluster__school`, `team__cluster__grade`, `team__cluster__section`

Examples:

```
GET /api/v1/schools/?instance__partner=1          (all schools of partner 1 — used by the Partner→School picker)
GET /api/v1/student-headcounts/?count_validation=mismatch
GET /api/v1/inquibuddy-submissions/?team__cluster__school=3&team__cluster__grade=8&team__cluster__section=A
```

Filters are applied **after** role scoping, so a school user can't widen their view with filters. An invalid filter value (e.g. `?school=abc`) returns 400.

`session-tracker/` reads `unit` / `grade` manually with `int(request.query_params.get(...))` — a non-numeric value raises `ValueError` → **500** (Observation).

22.3 Search and ordering

Not implemented. No `SearchFilter` / `OrderingFilter` backends are configured and no `search_fields` / `ordering_fields` exist. Ordering comes from model `Meta.ordering` or explicit `order_by()` in querysets. (The Django admin has `search_fields` for schools only.)

23. Error handling

23.1 How errors travel

No custom `EXCEPTION_HANDLER` exists, so DRF's default `rest_framework.views.exception_handler` is used:

```
raise ValidationError / PermissionDenied / NotFound / ... (anywhere in the view, serializer, permission)
   ▼
APIView.handle_exception()
   ├─ NotAuthenticated / AuthenticationFailed → adds WWW-Authenticate header, may switch 403→401
   ├─ exception_handler(exc, context)
   │     APIException subclasses → Response({"detail": ...} or field dict, status=exc.status_code)
   │     django Http404          → NotFound (404)
   │     django PermissionDenied → PermissionDenied (403)
   │     anything else           → returns None → exception re-raised → Django 500
   ▼
Response (JSON)  — or Django's 500 page / DEBUG traceback
```

There's no `try/except` in the views; services and tasks log and re-raise (task) or log and continue (`reconcile_all_schools`).

23.2 Status codes used

Code	Raised by	Example body
400 Bad Request	`serializer.is_valid(raise_exception=True)`; explicit `ValidationError(...)` in views	`{"principal_name": ["This field is required."]}` / `["Form 1 must be submitted before Form 2."]` / `{"file": "This field is required."}`
401 Unauthorized	missing/invalid/expired JWT; wrong login credentials	`{"detail": "Authentication credentials were not provided."}` / `{"detail":"Given token not valid for any token type","code":"token_not_valid",...}`
403 Forbidden	permission class returns False for an authenticated user; explicit `PermissionDenied` (Form 1 lock on PATCH, second kit delivery)	`{"detail": "You do not have permission to perform this action."}` / `{"detail": "Kit handover has already been recorded..."}`
404 Not Found	`get_object()` when the row is outside the scoped queryset or doesn't exist	`{"detail": "No KitDelivery matches the given query."}` (message varies by DRF version)
405 Method Not Allowed	e.g. `DELETE /inquibuddy-submissions/1/` (read-only viewset), `GET /auth/login/`	`{"detail": "Method \"DELETE\" not allowed."}`
409 Conflict	**not used** — "already submitted" conflicts are reported as 400 or 403	—
429 Too Many Requests	login throttle	`{"detail": "Request was throttled. Expected available in 42 seconds."}`
500 Server Error	unhandled exceptions, e.g. `int("abc")` in session-tracker, `IntegrityError` when admin/IIF omit `school` on a kit delivery, WeasyPrint missing native libraries on Windows	HTML error page (or traceback when `DEBUG=True`)

Observation — inconsistent status for "already submitted": re-submitting Form 1/Form 2 or an SL section returns **400** (`ValidationError`), while PATCHing a submitted school or creating a second kit delivery returns **403** (`PermissionDenied`). Clients must handle both. A single convention (e.g. 409 Conflict) would be clearer — **recommended improvement**, not current behaviour.

23.3 Validation errors

- Field errors: `{"field": ["msg", ...]}`.
- Non-field errors from `validate()` returning a dict → keyed by that field (Form 2 `session_schedules`).
- `ValidationError("plain string")` raised in a view → `["plain string"]` (a JSON list).
- Nested list errors: `{"teachers": [{}, {"name": ["This field is required."]}]}`.

24. Database migrations

24.1 Concepts

- A **migration** is a Python file describing a schema change (create table, add field, add constraint) or data change (`RunPython`).
- `python manage.py makemigrations` compares your models to the latest migration state and writes new migration files.
- `python manage.py migrate` applies unapplied migrations to the database and records them in the `django_migrations` table.
- **Dependencies** order migrations across apps: a migration adding an FK to `schools.School` depends on the migration that creates `School`.
- Model change → `makemigrations` → review the file → commit → `migrate` on every environment.

24.2 Migrations in this project

App	Migration	What it does
accounts	`0001_initial`	Creates `User` (depends on `auth`).
accounts	`0002_initial`	Adds `User.school` FK (and M2M `groups` / `user_permissions`); depends on `schools.0001`. Split into two files because `accounts` and `schools` refer to each other (`School` needs `files` / `programs`; `User.school` needs `School`) — Django breaks the cycle by adding the FK later.
core	`0001_initial`	`AuditLog` + its two indexes.
programs	`0001_initial`	`Partner`, `ProgramInstance`.
files	`0001_initial`	`File` + index.
schools	`0001_initial`	`School`, `SchoolGradeEnrollment`, `SchoolTeacher`, `SessionSchedule`, constraints.
schools	`0002_form1_fields`	Adds `form1_submitted(_at)`. **Converts `smart_board` from a nullable boolean to a 3-choice string** safely: add `smart_board_tmp` → `RunPython` copies `True→"yes"`, `False→"no"`, `NULL→""` (with a reverse function) → remove old field → rename tmp to `smart_board`. This is the standard pattern for changing a column's type while keeping data.
schools	`0003_form2_fields`	Adds `form2_submitted(_at)`.
kits, sl_selection, headcounts, teams, observations, inquibuddy	`0001_initial`	Their models and constraints.
reports	`0001_initial`	`SchoolProgress`; depends on `schools.0003_form2_fields`.

Data migrations use `apps.get_model("schools", "School")` (the **historical** model) rather than importing `School` directly — so the migration still works after the real model changes in future.

25. Signals and middleware

25.1 Signals

There is **one** signal receiver in the project:

```
# apps/inquibuddy/signals.py
@receiver(post_save, sender=Team)
def create_inquibuddy_submission(sender, instance: Team, created, **kwargs):
    if created:
        InquibuddySubmission.objects.get_or_create(team=instance)
```

- **When it executes:** synchronously, right after a `Team` row is inserted by `Model.save()` (including `Team.objects.create()`, admin "add", and `POST /teams/`), inside the same request/transaction.
- **Why:** every team must have exactly one InquiBuddy submission (status `pending`) so the InquiBuddy screens can list all teams without a separate "create submission" step.
- **How it's registered:** `InquibuddyConfig.ready()` imports `apps.inquibuddy.signals`; importing the module runs the `@receiver` decorator which connects the function. Without that import, the receiver would never be connected.
- **Doesn't fire for:** `Team.objects.bulk_create()`, raw SQL, or `QuerySet.update()`. A future bulk importer for teams would need to create submissions itself.
- `get_or_create` (not `create`) makes it safe if a submission somehow already exists (e.g. `loaddata` re-runs).

25.2 Middleware

No custom middleware exists; see §4.3 for the built-in stack and why `CorsMiddleware` is placed high.

26. Services, selectors and business logic

26.1 Architecture used

The project uses a **light "services + selectors" pattern**, only where logic is non-trivial:

- **Views** handle HTTP: parse input, call serializer, enforce simple rules, return responses. Most apps keep their rules directly in views (`schools`, `kits`, `sl_selection`) — fine for small rules.
- **Services** (`services.py`) hold *write-side* business logic that is reused or complex: `apps/inquibuddy/services.py`, `apps/reports/services.py`, plus `apps/core/audit.py`.
- **Selectors** (`selectors.py`) hold *read-side* queries/aggregations: `apps/observations/selectors.py`, `apps/reports/selectors.py`.
- **Tasks** are thin wrappers that call services (`reconcile_school_progress` → `reconcile_all_schools`).
- **lib/** holds external integrations.

Why separate? The same logic is called from several places (the nightly task, the management command, the headcount view and the progress endpoint all use `reports.services`). Keeping it out of views means it can be called without an HTTP request, and tested directly (most `reports/tests.py` tests call services, not endpoints).

26.2 `apps/inquibuddy/services.py`

Function	Inputs	Output	DB interaction	Exceptions	Called by
`attach_submission_file(*, submission, uploaded_file, kind, user)`	submission, Django `UploadedFile`, `"photo"` / `"audio"`, user	new `SubmissionFile`	INSERT `File` (stores the upload), DELETE previous `SubmissionFile` of that kind, INSERT `SubmissionFile`	storage errors	`InquibuddySubmissionViewSet._uploa`
`_report_context(submission)`	submission	dict for templates	reads team, cluster, school, instance, partner, photo, students, questions	—	the two report functions
`get_or_create_individual_report(submission, user)`	submission, user	`FeedbackReport`	SELECT latest individual report; else render PDF, INSERT `File` + `FeedbackReport` + `AuditLog`	WeasyPrint `ImportError` / `OSError` if native libs missing	`download_report` action
`create_all_teams_report(submissions, user)`	list of submissions, user	`FeedbackReport` (`submission=None`)	render PDF, INSERT `File`, `FeedbackReport`, `AuditLog`	same	`download_all_report` action
`_pdf_content_file(pdf_bytes, filename)`	bytes, name	`ContentFile`	—	—	the two report functions

Observation: the individual report is **cached forever** — once generated it's returned for every later download, even after feedback is regenerated (`evaluation_count` increments, questions change). The all-teams report is regenerated on every call (a new `File` each time). **Recommended:** invalidate the individual report when questions change.

26.3 `apps/reports/services.py`

Function	Purpose	DB interaction	Called by
`_normalize_section(section)`	`" a " → "A"`	—	helpers
`_sl_counts_by_grade_section(school)`	`{(grade, SECTION): n}` count of SLSelection rows	reads `school.sl_selections` (`values_list`)	`check_headcount_mismatch`, `reconcile_school`
`check_headcount_mismatch(headcount, sl_counts=None)`	Compare `total_sl` with actual SL rows; sync `count_validation`	may UPDATE `count_validation`	`reconcile_school`, `reconcile_section`
`_form_status(submitted, has_partial_data)`	`complete` / `in_progress` / `not_started`	—	`reconcile_school`
`_compute_kit_status(school)`	kit status from latest delivery vs offered grades	reads `kit_deliveries.first()`	`reconcile_school`
`reconcile_school(school)`	Recompute and upsert one `SchoolProgress`	reads children, `update_or_create`	`reconcile_all_schools`, tests
`reconcile_section(school, grade, section)`	Synchronous mismatch check for one section; doesn't touch `SchoolProgress`	reads headcount	`StudentHeadcountViewSet._mismatch_warning`
`reconcile_all_schools()`	Loop all schools with prefetch; log and skip failures	many	`reconcile_school_progress` task + command

Validation: none (inputs are model instances). Exceptions: `reconcile_all_schools` catches **all** exceptions per school and logs them.

26.4 Selectors

Function	Returns	SQL idea
`observations.selectors.session_tracker(unit, grade)`	`values()` queryset of observations for that unit+grade	`SELECT school_id, school.code, section, session_no, date, observed_by FROM ... JOIN school WHERE unit=? AND grade=? ORDER BY code, section, session_no`
`observations.selectors.session_progress()`	per school × unit completed-session counts	`GROUP BY school_id, code, unit` with `COUNT(id)`
`reports.selectors.compute_kpis(progress_qs)`	`{schools_enrolled, forms_completed_pct, kits_delivered, active_student_leaders}`	several `COUNT(*)` queries

`compute_kpis` details: `schools_enrolled = School.objects.count()` (all schools, not only those with a progress row); `forms_completed_pct` = % of schools whose progress row is complete across all five dimensions; `kits_delivered` = rows with `kit_status="complete"`; `active_student_leaders` = `SLSelection` rows with `sl_status="selected"` (all schools). It's only called for admin/IIF staff.

Observation: `schools_enrolled` counts all schools, but `forms_completed` / `kits_delivered` only count schools that already have a `SchoolProgress` row (created by the nightly job or first progress read). Until reconciliation has run for every school, percentages can be understated.

26.5 Audit helper — `apps/core/audit.py`

```
def record_audit(*, user, action, entity_type, entity_id="", before=None, after=None):
```

Keyword-only args (the `*`). Stores `user` only if authenticated (so `AnonymousUser` / `None` → `NULL`), converts `entity_id` to string. Called by: `UserViewSet.perform_create/perform_update`, both report services, `generate_ai_feedback`.

---

27. Repositories

There is no repository layer in this project (no `repositories.py`, no query-object classes). Views and services use the Django ORM directly, and read-only aggregations live in `selectors.py`.

For reference: a *repository* wraps data access behind an interface (`SchoolRepository.get_for_user(user)`), so services don't touch the ORM. In Django projects this is usually unnecessary because the ORM's QuerySet/Manager API already plays that role; custom **managers/QuerySet methods** (e.g. `School.objects.for_user(user)`) are the idiomatic equivalent. The project's `scope_queryset_to_role()` function is the closest thing — a reusable query rule shared by every viewset.

28. Complete feature flows

28.1 Login → authenticated request → token refresh

```
Client ──POST /api/v1/auth/login/ {username, password}──▶ config/urls → accounts/urls → LoginView
   ScopedRateThrottle("login", 10/min per IP)          → 429 if exceeded
   ThinkAndMakeTokenObtainPairSerializer.validate()
       authenticate() → User row → check_password (PBKDF2) → is_active?
       get_token(user) → RefreshToken + claims role, school_id
◀── 200 {refresh, access}
Client stores tokens
Client ──GET /api/v1/auth/me/  Authorization: Bearer <access>──▶ MeView
   JWTAuthentication → request.user ; IsAuthenticated
◀── 200 {id, username, email, display_name, role, school, is_active, date_joined}
... 15 minutes later: access expired → 401 token_not_valid
Client ──POST /api/v1/auth/refresh/ {refresh}──▶ TokenRefreshView
◀── 200 {access, refresh(new)}
```

28.2 Admin creates a user

```
POST /api/v1/users/ {username, password, role:"school", school:3, display_name}
 → IsAdmin
 → get_serializer_class() → UserCreateSerializer (action == "create")
 → validate: username unique (model), password min_length=8, role choice, school PK exists
 → perform_create → serializer.save() → create(): pop password, User(**data), set_password(), save()
 → record_audit(action="create_user", entity_type="User", entity_id=user.id)
◀ 201 {id, username, email, display_name, role, school, is_active}   (password is write_only → not returned)
```

Update: `PATCH /users/{id}/` → `UserSerializer` → `perform_update` snapshots `before`, saves, audits `before/after`. Note `UserSerializer` has no password field, so passwords can't be changed through this endpoint.

28.3 Form 1 — School Enrollment

```
Frontend: GET /partners/  → pick partner → GET /schools/?instance__partner=<id> → pick school
(optional draft saves) PATCH /schools/{id}/ {visited_by: "Asha"}   ← allowed until submitted
Final: POST /schools/{id}/submit-form1/ {visited_by, visit_date, location, district, distance_to_iif_km,
        principal_name, principal_phone, gender_type, school_type, medium, grades:[6,7],
        lab_room, internet, smart_board, kit_storage, maps_link, principal_acknowledged, ...}
 → IsAdminOrIIFStaffOrReadOnly  (school role → 403)
 → get_object()  (scoped; 404 if not visible)
 → already submitted and not admin? → 400 "Form 1 data has already been submitted..."
 → SchoolForm1SubmitSerializer(school, data).is_valid()      → 400 with field errors
 → grades = sorted(ints)
 → serializer.save(grades_offered="6,7", total_sections=2, form1_submitted=True, form1_submitted_at=now)
 → for each grade: SchoolGradeEnrollment.get_or_create(school, grade)   (keeps existing counts)
◀ 200 SchoolSerializer(school)  (with nested grade_enrollments/teachers/session_schedules)
```

28.4 Form 2 — Schools Contact Info

```
POST /schools/{id}/submit-form2/ {iif_poc, teachers:[{name, phone, grades_taught}], session_schedules:[{grade, day_of_week, time}]}
 → permission (admin/IIF) → get_object()
 → not form1_submitted → 400 "Form 1 must be submitted before Form 2."
 → form2_submitted and not admin → 400 "Form 2 data has already been submitted..."
 → SchoolContactSubmitSerializer.is_valid(): ≥1 teacher; schedule for every grade in grades_offered
 → pop teachers/schedules; save(iif_poc, form2_submitted=True, form2_submitted_at=now)
 → school.teachers.all().delete(); school.session_schedules.all().delete()
 → bulk_create SchoolTeacher rows; bulk_create SessionSchedule rows
◀ 200 SchoolSerializer(school)
```

28.5 Kit delivery

```
(1) POST /files/ multipart {file, entity_type:"KitDelivery", entity_id:""}  → {id: 41, ...}   (optional proof photo)
(2) POST /kit-deliveries/ {date_of_delivery, delivered_by, received_by, grade_6_kit..grade_9_kit,
                           delivery_proof_photo: 41, acknowledgement_letter: 42, school?}
 → IsAuthenticated → mixin.get_serializer (school users: school forced)
 → KitDeliverySerializer.is_valid()
 → KitDeliveryViewSet.perform_create: school = user.school (school role) or validated school
       if not admin and a delivery exists for that school → 403 "Kit handover has already been recorded..."
 → mixin.perform_create → save(school=...)
◀ 201 KitDelivery JSON
```

28.6 Form 3 — Student headcount with instant mismatch warning

```
POST /student-headcounts/ {grade:6, section:"A", total_sl:7, total_clusters, total_teams, total_students, school?}
 → mixin injects school → StudentHeadcountSerializer.is_valid()
       UniqueTogetherValidator(school, grade, section) → 400 if the section already has a headcount
 → super().create() → INSERT → 201 response built
 → _mismatch_warning(id): get headcount → reports.services.reconcile_section(school, 6, "A")
       → check_headcount_mismatch(): count SLSelection rows for (6,"A")
            none → None (Form 4 not started) ; equal → None ; different → flag + count_validation="mismatch"
 → response.data["mismatch_warning"] = {"grade":6,"section":"A","total_sl":7,"sl_selection_count":5} or null
◀ 201 {..., "mismatch_warning": ...}
```

28.7 Form 4 — SL Selection bulk submit

```
POST /sl-selections/bulk-submit/ {school?, grade, section, teacher, teacher_acknowledged, entries:[{sl_name, ...}, ...]}
 → IsAuthenticated
 → SLSelectionBulkSubmitSerializer.is_valid()  (≥1 entry; each entry fully specified)
 → school = user.school (school role, body ignored) or data["school"]; None → 400 {"school": "This field is required."}
 → not admin and rows exist for (school, grade, section) → 400 "SL Selection data has already been submitted..."
 → build SLSelection objects (shared grade/section/teacher + each entry) → bulk_create
◀ 201 [SLSelection, SLSelection, ...]
```

28.8 Session observation → tracker/progress

```
POST /session-observations/ {grade, section, unit, session_no, date, observed_by, notes, school?}
 → mixin + UniqueTogetherValidator on (school, grade, section, unit, session_no) → 400 if duplicate
◀ 201
GET /session-tracker/?unit=1&grade=6  → selectors.session_tracker(1, 6) → filter to own school if school role
◀ 200 [ {school_id, school__school_code, section, session_no, date, observed_by}, ... ]
GET /session-progress/ → selectors.session_progress() → [ {school_id, school__school_code, unit, sessions_completed}, ... ]
```

28.9 InquiBuddy — from team to PDF

```
(0) Team created (admin/IIF POST /teams/ or admin) → post_save signal → InquibuddySubmission(status="pending")
(1) GET /inquibuddy-submissions/?team__cluster__school=3&team__cluster__grade=8&team__cluster__section=A
(2) POST /inquibuddy-submissions/{id}/upload-photo/  multipart file=<jpg>
      → get_object() (scoped) → services.attach_submission_file(kind="photo")
            File.objects.create(file=upload, entity_type="InquibuddySubmission", entity_id=id)  → storage write
            delete previous photo SubmissionFile; create new SubmissionFile
      → re-fetch submission (fresh prefetch cache) → 200 submission JSON
(2b) POST .../upload-audio/  (optional, same flow, kind="audio")
(3) POST /inquibuddy-submissions/{id}/generate-feedback/
      → no photo → 400 "Upload an idea photo before generating feedback."
      → status="processing"; save → generate_ai_feedback.delay(id)  → message to Redis
      ◀ 200 {status:"processing", ...}
    (3-bulk) POST /inquibuddy-submissions/generate-feedback-bulk/ {school, grade, section}
      → every scoped submission in that section that has a photo → processing + .delay() → {"queued": n}
(4) Celery worker: generate_ai_feedback(id)
      → read photo/audio bytes from storage → lib.gemini_client.generate_feedback_questions()
            Gemini "gemini-2.5-flash", response_mime_type=application/json → parse JSON → first 5
      → replace AIFeedbackQuestion rows → status="evaluated", evaluation_count+=1 → AuditLog
      → on error: retry up to 2× after 30 s
(5) Client polls GET /inquibuddy-submissions/{id}/ until status == "evaluated"
(6) GET /inquibuddy-submissions/{id}/report/
      → not evaluated → 400
      → services.get_or_create_individual_report(): existing? return it : render template → WeasyPrint → File + FeedbackReport + AuditLog
      ◀ FileResponse (PDF attachment "InquiBuddy_T001.pdf")
(7) GET /inquibuddy-submissions/report-all/?school=3&grade=8&section=A
      → evaluated submissions in section → one page per team → new PDF every time
```

28.10 Progress dashboard (nightly + on demand)

```
02:00 IST  Celery Beat → message "apps.reports.tasks.reconcile_school_progress" → Redis → worker
   → reconcile_all_schools(): School.objects.prefetch_related(6 relations)
       for school: reconcile_school(school)   (exceptions logged, loop continues)
          denom      = Σ grade_enrollments.total_sections
          form3_pct  = min(round(#headcounts / denom * 100), 100)
          form4_pct  = min(round(#distinct (grade,section) with SL rows / denom * 100), 100)
          mismatch_flags = [check_headcount_mismatch(hc) for each headcount] (non-None only)
          form1_status = complete if form1_submitted else in_progress if visited_by or enrollments else not_started
          form2_status = complete if form2_submitted else in_progress if teachers/schedules exist else not_started
          kit_status   = from latest KitDelivery vs grades_offered (none/some/all delivered)
          SchoolProgress.update_or_create(school, defaults=...)
Dashboard: GET /reports/summary/ → scoped SchoolProgress list + KPIs (admin/IIF) or kpis=null (school)
Detail:    GET /schools/{id}/progress/ → SchoolProgress.get_or_create(school) (defaults if never reconciled)
Manual:    python manage.py reconcile_school_progress
```

Note the detail endpoint's `get_or_create` returns an **un-reconciled** default row (`not_started`, 0%) for a school the job hasn't processed yet (tested by `test_school_progress_detail_creates_row_on_first_read`).

28.11 Legacy data import

```
python manage.py import_legacy_sheets --file "../docs/Datla-Think & Make _ 2026-2027.xlsx"
 → openpyxl.load_workbook(data_only=True)
 → Partner/ProgramInstance get_or_create (defaults Datla / DATLA-HYD-2026 / 2026)
 → transaction.atomic():
     School_Enrollment       → School.update_or_create(instance, school_code) + parse "Grade Data" regex → SchoolGradeEnrollment
     Schools_Contact_Info    → maps link / IIF PoC update; parse "Teachers" and "Session Schedule" regexes
     Kits_Info               → KitDelivery.update_or_create(school, external_ref)
     SL_Selection_Assessment → SLSelection.update_or_create(school, external_ref)
     Students_Count_Info     → StudentHeadcount.update_or_create(school, grade, section)
 → unparseable free-text entries are reported on stderr and skipped
python manage.py import_master_users
 → transaction.atomic(): for each hard-coded row: resolve/create School (placeholder partner/instance for
   unknown prefixes M, U, I) → User.update_or_create(username) → set_password(PIN) → save
```

29. API reference

Conventions: base URL `/api/v1/`; JSON unless stated; `Authorization: Bearer <access>` required unless stated. Common errors on every authenticated endpoint: **401** (no/invalid token). IDs are integers. Paginated lists return `{count, next, previous, results}`.

Auth

`POST /api/v1/auth/login/`

- **Purpose:** obtain JWT pair. **Authentication:** none. **Permissions:** open; throttled 10/min per IP.
- **Request**

```
{ "username": "TMDATURKAPALLY", "password": "••••••••" }
```

- **Validation:** both required; credentials must match an active user.
- **Success 200**

```
{ "refresh": "eyJ...", "access": "eyJ..." }
```

- **Errors:** 400 missing fields · 401 wrong credentials/inactive · 429 throttled.

`POST /api/v1/auth/refresh/`

- **Request** `{ "refresh": "eyJ..." }` → **200** `{ "access": "eyJ...", "refresh": "eyJ..." }`
- **Errors:** 400 missing · 401 invalid/expired refresh token.

`GET /api/v1/auth/me/`

- **Success 200**

```
{ "id": 17, "username": "TMDATURKAPALLY", "email": "", "display_name": "TURKAPALLY",
  "role": "school", "school": 3, "is_active": true, "date_joined": "2026-09-01T10:00:00+05:30" }
```

Users (admin only)

`GET /api/v1/users/` · `GET /api/v1/users/{id}/`

- Ordered by `username`, paginated. Items as in `/auth/me/`. **Errors:** 403 non-admin, 404.

`POST /api/v1/users/`

- **Request**

```
{ "username": "newstaff@inqui-lab.org", "email": "newstaff@inqui-lab.org", "display_name": "New Staff",
  "role": "iif_staff", "school": null, "password": "at-least-8-chars", "is_active": true }
```

- **Validation:** `username` unique/valid chars; `role` in choices; `school` existing id or null; `password` ≥ 8 chars (Django's password validators are **not** applied).
- **Success 201** — same fields minus `password`. **Errors:** 400, 403.

`PUT/PATCH /api/v1/users/{id}/` · `DELETE /api/v1/users/{id}/`

- `UserSerializer` fields (`username, email, display_name, role, school, is_active`). Audited on update (not on delete). DELETE → 204.

Programs

`GET|POST /api/v1/partners/`, `GET|PUT|PATCH|DELETE /api/v1/partners/{id}/`

- Body/response: `{ "id": 1, "name": "Datla" }`. Writes admin/IIF only (403 for school). Deleting a partner with instances → `ProtectedError` → 500 (not handled).

`GET|POST /api/v1/instances/`, `.../instances/{id}/`

- `{ "id": 1, "code": "DATLA-HYD-2026", "label": "", "partner": 1, "year": 2026, "is_active": true }`

Schools

`GET /api/v1/schools/` (filters `instance`, `instance__partner`, `school_code`, `district`) · `GET /api/v1/schools/{id}/`

- **Success 200 (one item)**

```
{
  "id": 3, "instance": 1, "school_code": "DA", "name": "School A", "district": "Sangareddy",
  "location": "Turkapally", "distance_to_iif_km": "12.50", "gender_type": "co-ed",
  "school_type": "government", "medium": "telugu", "grades_offered": "6,7", "total_sections": 2,
  "principal_name": "Principal Rao", "principal_phone": "9876543210", "principal_email": "",
  "principal_acknowledged": true, "lab_room": true, "internet": false, "smart_board": "yes_not_working",
  "kit_storage": true, "maps_link": "https://www.google.com/maps?q=1,2", "school_photo": null,
  "iif_poc": "Ravi", "observations": "", "next_steps": "", "visited_by": "Asha", "visit_date": "2026-09-17",
  "form1_submitted": true, "form1_submitted_at": "2026-09-17T11:02:03+05:30",
  "form2_submitted": false, "form2_submitted_at": null,
  "grade_enrollments": [ { "id": 1, "school": 3, "grade": 6, "total_sections": 0, "total_students": 0 } ],
  "teachers": [], "session_schedules": []
}
```

(Decimal fields are serialized as strings by DRF by default.)

`POST /api/v1/schools/` · `PUT|PATCH /api/v1/schools/{id}/` · `DELETE`

- Admin/IIF only. After `form1_submitted`, PUT/PATCH by non-admin → **403** `"Form 1 data has already been submitted for this school. Please contact admin to make any changes."` Read-only fields: `form1_submitted`, `form1_submitted_at`, `form2_submitted`, `form2_submitted_at` (ignored if sent). Nested lists are read-only.

`POST /api/v1/schools/{id}/submit-form1/`

- **Permissions:** admin/IIF; school role → 403.
- **Request**

```
{
  "visited_by": "Asha", "visit_date": "2026-09-17", "location": "Turkapally", "district": "Sangareddy",
  "distance_to_iif_km": "12.5", "principal_name": "Principal Rao", "principal_phone": "9876543210",
  "principal_email": "", "gender_type": "co-ed", "school_type": "government", "medium": "telugu",
  "grades": [6, 7], "lab_room": true, "internet": false, "smart_board": "yes_not_working",
  "kit_storage": true, "maps_link": "https://www.google.com/maps?q=1,2", "school_photo": null,
  "observations": "", "next_steps": "", "principal_acknowledged": true
}
```

- **Validation:** all fields required except `principal_email`, `school_photo`, `observations`, `next_steps`; choices for `gender_type` / `school_type` / `medium` / `smart_board`; `grades` ⊆ {6,7,8,9}; `maps_link` valid URL. (`principal_acknowledged` must be present but `false` is accepted.)
- **Success 200:** full `SchoolSerializer` with `form1_submitted: true`, `grades_offered: "6,7"`, `total_sections: 2`.
- **Errors:** 400 validation / already submitted (non-admin) · 403 school role · 404.

`POST /api/v1/schools/{id}/submit-form2/`

- **Request**

```
{
  "iif_poc": "Ravi",
  "teachers": [ { "name": "Teacher One", "phone": "9000000001", "grades_taught": "6,7" } ],
  "session_schedules": [ { "grade": 6, "day_of_week": "mon", "time": "10:30:00" },
                         { "grade": 7, "day_of_week": "tue", "time": "14:45:00" } ]
}
```

- **Validation:** Form 1 must be submitted; `iif_poc` non-blank; ≥1 teacher (`name` required); every grade in `grades_offered` needs a schedule; `day_of_week` ∈ mon..sat.
- **Success 200:** `SchoolSerializer` with `form2_submitted: true` and the new `teachers` / `session_schedules`.
- **Errors:** 400 (incl. `"Form 1 must be submitted before Form 2."`, `"Missing a session schedule for grade(s): 7."`) · 403 · 404.

`GET /api/v1/schools/{id}/progress/`

- **Permissions:** any logged-in user; scoped (other school → 404).
- **Success 200**

```
{ "school": 3, "school_code": "DA", "school_name": "School A", "form1_status": "complete",
  "form2_status": "not_started", "form3_pct": 50, "form4_pct": 0, "kit_status": "in_progress",
  "mismatch_flags": [], "updated_at": "2026-09-18T02:00:05+05:30" }
```

`/api/v1/school-grade-enrollments/`, `/school-teachers/`, `/session-schedules/`

- CRUD; read scoped; write admin/IIF. Bodies: `{school, grade, total_sections, total_students}`, `{school, name, phone, grades_taught}`, `{school, grade, day_of_week, time}`. Duplicate `(school, grade)` enrollment → 400.

Kit deliveries — `/api/v1/kit-deliveries/`

- **Request (POST)**

```
{ "date_of_delivery": "2026-06-01", "delivered_by": "Ravi", "received_by": "HM",
  "grade_6_kit": true, "grade_7_kit": true, "grade_8_kit": false, "grade_9_kit": false,
  "delivery_proof_photo": 41, "acknowledgement_letter": null, "school": 3 }
```

`school` ignored/forced for school role; must be supplied by admin/IIF (omitting → 500, see §14.9). - **Success 201** `{ "id": 9, "school": 3, ...same fields }` - **Errors:** 400 validation · 403 `"Kit handover has already been recorded for this school. Please contact admin to make any changes."` (non-admin, second create) · 404.

SL selections — `/api/v1/sl-selections/`

`POST /api/v1/sl-selections/bulk-submit/`

- **Request**

```
{
  "school": 3, "grade": 6, "section": "A", "teacher": "Ms. Rao", "teacher_acknowledged": true,
  "entries": [
    { "sl_name": "Asha", "interested_in_role": true, "attendance_above_90": true, "sl_status": "selected",
      "speaks_clearly": true, "speaks_loudly": true, "understands_english": true }
  ]
}
```

- **Validation:** `grade`, `section`, `teacher`, `teacher_acknowledged`, `entries` required; each entry's 7 fields required; `sl_status` ∈ selected/not_selected/pending; ≥1 entry; `school` must exist (if given) and is required for non-school roles.
- **Success 201**

```
[ { "id": null, "school": 3, "grade": 6, "section": "A", "teacher": "Ms. Rao", "sl_name": "Asha",
    "interested_in_role": true, "attendance_above_90": true, "sl_status": "selected",
    "speaks_clearly": true, "speaks_loudly": true, "understands_english": true, "teacher_acknowledged": true } ]
```

(`id` is populated after `bulk_create` on PostgreSQL and on SQLite ≥ 3.35; older backends may return `null`.) - **Errors:** 400 validation / `"SL Selection data has already been submitted for this grade/section. Please contact admin to make any changes."` / `{"school": "This field is required."}`.

Standard CRUD: `{school?, grade, section, teacher, sl_name, interested_in_role, attendance_above_90, sl_status, speaks_clearly, speaks_loudly, understands_english, teacher_acknowledged}`.

Student headcounts — `/api/v1/student-headcounts/`

- **Request (POST)**

```
{ "grade": 6, "section": "A", "total_sl": 7, "total_clusters": 8, "total_teams": 8, "total_students": 32,
  "teams_info_photo": null, "student_database_file": null, "extraction_status": "ok",
  "count_validation": "ok", "school": 3 }
```

- **Success 201 / 200 (update)** — headcount fields plus

```
"mismatch_warning": { "grade": 6, "section": "A", "total_sl": 7, "sl_selection_count": 5 }
```

or `"mismatch_warning": null`. - **Errors:** 400 (incl. `{"non_field_errors": ["The fields school, grade, section must make a unique set."]}`) · 404.

Teams — `/api/v1/clusters/`, `/api/v1/teams/`, `/api/v1/students/`

- Cluster item:

```
{ "id": 1, "school": 3, "grade": 8, "section": "A", "cluster_number": 1,
  "teams": [ { "id": 1, "cluster": 1, "team_code": "T001", "sl_name": "Ravi Kumar",
               "students": [ { "id": 1, "team": 1, "name": "Student 1" } ] } ] }
```

- Writes admin/IIF only; nested lists are read-only (create teams/students via their own endpoints).

Session observations

- `/api/v1/session-observations/` body `{school?, grade, section, unit (1-3), session_no, observed_by, date, notes}`; duplicate → 400.
- `GET /api/v1/session-tracker/?unit=1&grade=6` → `[{"school_id":3,"school__school_code":"DA","section":"A","session_no":1,"date":"2026-07-01","observed_by":"Asha"}]` (non-integer params → 500).
- `GET /api/v1/session-progress/` → `[{"school_id":3,"school__school_code":"DA","unit":1,"sessions_completed":4}]`.

InquiBuddy — `/api/v1/inquibuddy-submissions/`

- **Item**

```
{
  "id": 5, "team": 1, "team_code": "T001", "sl_name": "Ravi Kumar", "status": "evaluated", "evaluation_count": 1,
  "submission_files": [ { "id": 2, "kind": "photo", "file": 57, "created_at": "..." } ],
  "ai_feedback_questions": [ { "id": 11, "order": 1, "question_en": "What problem ...?", "question_te": "..." } ],
  "feedback_reports": [ { "id": 3, "type": "individual", "file": 60, "created_at": "..." } ]
}
```

- `POST {id}/upload-photo/` / `upload-audio/` — **multipart** field `file`. 400 `{"file": "This field is required."}` if missing. 200 → item.
- `POST {id}/generate-feedback/` — no body. 400 `["Upload an idea photo before generating feedback."]`. 200 → item with `status: "processing"`.
- `POST generate-feedback-bulk/` — `{ "school": 3, "grade": 8, "section": "A" }` → `{ "queued": 4 }`. 400 `["school, grade and section are all required."]`.
- `GET {id}/report/` → `application/pdf` attachment. 400 `["Feedback has not been generated for this team yet."]`.
- `GET report-all/?school=3&grade=8&section=A` → PDF. 400 missing params or `["No evaluated teams in this grade/section yet."]`.
- Errors common: 404 outside scope; 405 for PUT/PATCH/DELETE/POST on the collection.

Files — `/api/v1/files/`

- `POST /files/` multipart: `file` (required), `entity_type` (required), `entity_id` (optional) → **201** `{ "id": 41, "file": "http://.../media/KitDelivery/misc/proof.jpg", "entity_type": "KitDelivery", "entity_id": "" }`.
- `GET /files/{id}/` → `{ "id": 41, "url": "http://...", "mime_type": "image/jpeg", "size": 20480, "entity_type": "KitDelivery", "entity_id": "", "created_at": "..." }` (with S3: signed URL valid 10 min).

Reports — `GET /api/v1/reports/summary/`

```
{
  "kpis": { "schools_enrolled": 57, "forms_completed_pct": 12, "kits_delivered": 20, "active_student_leaders": 310 },
  "schools": [ { "school": 3, "school_code": "DA", "school_name": "School A", "form1_status": "complete", "...": "..." } ]
}
```

School role → `"kpis": null` and only its own row.

---

30. Function Reference

Custom project functions and overridden methods, grouped by file. Framework internals are only mentioned where they call project code.

`apps/core/permissions.py`

```
Function: scope_queryset_to_role(queryset, request, school_field="school")
File: apps/core/permissions.py
Purpose: Enforce "school users see only their school's rows" in one place.
Called by: SchoolScopedViewSetMixin.get_queryset; SchoolViewSet / SchoolGradeEnrollmentViewSet /
           SchoolTeacherViewSet / SessionScheduleViewSet / ClusterViewSet / TeamViewSet /
           StudentViewSet / InquibuddySubmissionViewSet .get_queryset; ReportsSummaryView.get
Calls: QuerySet.filter
Parameters: queryset (any QuerySet); request (DRF Request with authenticated user);
            school_field (lookup path to the school id, e.g. "id", "cluster__school_id")
Return value: the same queryset (admin/iif_staff) or a filtered one (school)
Database interaction: none directly (lazy filter)
Possible exceptions: AttributeError if request.user is anonymous (no .role) — prevented by permissions
Step-by-step: read user → if role == "school": filter(**{school_field: user.school_id}) → else return unchanged
```

`IsAdmin.has_permission`, `IsAdminOrIIFStaff.has_permission`, `IsSchoolUser.has_permission`, `IsAdminOrIIFStaffOrReadOnly.has_permission(request, view)` — called by DRF's `APIView.check_permissions()` before the handler; return `bool`. No DB access beyond the already-loaded user.

`apps/core/mixins.py` — see §14 for full explanation

- `get_queryset(self)` → `scope_queryset_to_role(super().get_queryset(), ...)`.
- `get_serializer(self, *args, **kwargs)` → injects `school` for school users when `data` is passed.
- `perform_create(self, serializer)` → `serializer.save(school=user.school)` for school users, else `serializer.save()`.

`apps/core/audit.py`

```
Function: record_audit(*, user, action, entity_type, entity_id="", before=None, after=None)
File: apps/core/audit.py
Purpose: Write one AuditLog row for a sensitive mutation.
Called by: UserViewSet.perform_create/perform_update, get_or_create_individual_report,
           create_all_teams_report, generate_ai_feedback
Calls: AuditLog.objects.create
Parameters: keyword-only; user may be None/AnonymousUser (stored as NULL); entity_id coerced to str
Return value: None
Database interaction: INSERT core_auditlog
Possible exceptions: DB errors
Step-by-step: user if authenticated else None → create row
```

`apps/core/management/commands/import_legacy_sheets.py`

Function / method	Purpose
`_headers(sheet)`	`{header_text: column_index}` from row 1.
`_rows(sheet, headers)`	Generator of dicts for rows 2..n, skipping rows whose first cell is empty.
`_bool(value)`	`"yes"/"y"/"true"/"1"` (any case) → True; `None` → False.
`_smart_board(value)`	Maps free text to `yes` / `no` / `yes_not_working` / `""`.
`_date(value)`	`datetime` → `date`; else unchanged.
`_clean_str(value)`	`1.0` → `"1"`; strips strings; `None` → `""`.
`Command.add_arguments`	`--file` (required), `--instance-code`, `--partner-name`, `--year`.
`Command.handle`	Load workbook (FileNotFoundError → `CommandError`), get/create partner+instance, run five importers inside one `transaction.atomic()`, print counts.
`_import_school_enrollment`	`School.update_or_create(instance, school_code)`; then `_parse_grade_data`.
`_parse_grade_data`	Regex `Grade 6: 41 students, 1 sections` per line → recreate `SchoolGradeEnrollment` rows.
`_map_choice(value, choices)`	Match value against choice keys or labels (case-insensitive) → key or `""`.
`_import_schools_contact_info`	Update maps link (only if empty) and IIF PoC via `QuerySet.update`; parse teachers & schedule.
`_parse_teachers`	Regex `Teacher 1: Name - 9876543210 - Grade 6,7` → recreate `SchoolTeacher` rows.
`_parse_session_schedule`	Regex `6th Grade - Tue - 2:45 PM` → recreate `SessionSchedule` rows.
`_import_kits`, `_import_sl_selection`	`update_or_create(school, external_ref)`.
`_import_headcounts`	`update_or_create(school, grade, section)`.
`_map_extraction_status`, `_map_count_validation`	Keyword matching ("error", "valid", "mismatch") → enum keys.

Edge cases: unknown school codes are skipped with a stderr message; unparseable free-text entries are skipped individually; the whole import rolls back if any unexpected exception occurs.

`apps/core/management/commands/import_master_users.py`

- `Command.handle()` — inside `transaction.atomic()`, for every hard-coded `ROWS` entry: map role, resolve school (school role only), `User.update_or_create(username, defaults=...)` (admins get `is_staff` and `is_superuser`), `set_password(pin)`, save. Re-running resets passwords to the listed

values.
- `Command._resolve_school(code, name)` — find `School` by `school_code` (any instance); else create placeholder `Partner "Partner <X> (pending)"`, `ProgramInstance "<X>-2026"` and a bare `School`.

`apps/accounts`

```
Function: ThinkAndMakeTokenObtainPairSerializer.get_token(cls, user)   [classmethod override]
Purpose: Add role and school_id claims to the JWT.
Called by: TokenObtainPairSerializer.validate() (SimpleJWT) during login
Calls: super().get_token(user) → RefreshToken.for_user(user)
Return value: RefreshToken with extra claims (access token derived from it inherits them)
```

```
Function: UserCreateSerializer.create(self, validated_data)
Purpose: Create a user with a hashed password.
Called by: serializer.save() in UserViewSet.perform_create
Step-by-step: pop "password" → User(**validated_data) → user.set_password(password) → user.save() → return user
Database interaction: INSERT accounts_user
Edge cases: password never stored in plaintext; write_only so never returned
```

- `MeView.get(request)` → `Response(UserSerializer(request.user).data)`.
- `UserViewSet.get_serializer_class()` → `UserCreateSerializer` if `self.action == "create"` else `UserSerializer`.
- `UserViewSet.perform_create(serializer)` → save + `record_audit("create_user")`.
- `UserViewSet.perform_update(serializer)` → snapshot `before` → save → `record_audit("update_user", before, after)`.

`apps/schools/serializers.py`

- `SchoolContactSubmitSerializer.validate_teachers(value)` → error if empty list.
- `SchoolContactSubmitSerializer.validate(attrs)` → set difference between `grades_offered` and scheduled grades; error listing missing grades. Called by DRF during `is_valid()` after field validation.

`apps/schools/views.py`

```
Function: SchoolViewSet.get_queryset(self)
Purpose: Base school queryset with related data preloaded, scoped to role.
Calls: select_related("instance"), prefetch_related("grade_enrollments","teachers","session_schedules"),
       scope_queryset_to_role(..., school_field="id")
Called by: DRF list() and get_object()
```

```
Function: SchoolViewSet.perform_update(self, serializer)
Purpose: Lock the school record after Form 1 for non-admins.
Called by: UpdateModelMixin.update() (PUT/PATCH) after validation
Possible exceptions: PermissionDenied (403) with ALREADY_SUBMITTED_MESSAGE
```

```
Function: SchoolViewSet.submit_form1(self, request, pk=None)   [@action detail, POST, "submit-form1"]
Purpose: Validate and finalize Form 1.
Calls: get_object, SchoolForm1SubmitSerializer, serializer.save, SchoolGradeEnrollment.objects.get_or_create,
       timezone.now, SchoolSerializer
Return value: Response(SchoolSerializer(school).data) — 200
Database interaction: UPDATE schools_school; SELECT/INSERT schools_schoolgradeenrollment per grade
Possible exceptions: ValidationError (400) — already submitted (non-admin) or invalid fields; 404; 403
Step-by-step: get_object → lock check → validate → pop+sort grades → save with derived fields + flags
              → get_or_create enrollments → serialize
```

```
Function: SchoolViewSet.submit_form2(self, request, pk=None)   [@action detail, POST, "submit-form2"]
Purpose: Validate and finalize Form 2, replacing teachers and schedules.
Database interaction: UPDATE school; DELETE teachers + schedules; bulk INSERT both
Possible exceptions: ValidationError (Form 1 not submitted / already submitted / invalid)
Edge cases: not atomic — see §8
```

```
Function: SchoolViewSet.progress(self, request, pk=None)   [@action detail, GET, "progress"]
Purpose: Return the school's SchoolProgress row, creating a default one if missing.
Calls: get_object, SchoolProgress.objects.get_or_create, SchoolProgressSerializer
```

`SchoolGradeEnrollmentViewSet/SchoolTeacherViewSet/SessionScheduleViewSet.get_queryset()` — `select_related("school")` + scoping.

`apps/kits/views.py`

```
Function: KitDeliveryViewSet.perform_create(self, serializer)
Purpose: Allow only one kit delivery per school for non-admins, then delegate to the mixin.
Called by: CreateModelMixin.create()
Calls: KitDelivery.objects.filter(school=...).exists(); SchoolScopedViewSetMixin.perform_create via super()
Possible exceptions: PermissionDenied (403); IntegrityError (500) if school is None for non-school roles
Step-by-step: school = user.school (school role) or validated_data["school"] → if school and not admin and
              exists → 403 → super().perform_create(serializer)
```

`apps/sl_selection/views.py`

```
Function: SLSelectionViewSet.bulk_submit(self, request)   [@action list, POST, "bulk-submit"]
Purpose: Create one SLSelection row per entry for a grade/section in a single request.
Calls: SLSelectionBulkSubmitSerializer.is_valid, SLSelection.objects.filter(...).exists, bulk_create,
       SLSelectionSerializer(many=True)
Return value: 201 with list of created rows
Database interaction: SELECT exists; one multi-row INSERT
Possible exceptions: ValidationError (400) for invalid input, missing school, or duplicate section (non-admin)
Edge cases: school role → body "school" ignored; bulk_create skips save()/signals; the duplicate check
            compares section exactly (no normalisation, unlike reports.services)
```

`SLSelectionBulkSubmitSerializer.validate_entries(value)` → error if empty.

`apps/headcounts/views.py`

- `create(request)` / `update(request)` → call `super()` then set `response.data["mismatch_warning"] = self._mismatch_warning(id)`. (`partial_update` calls `update` internally, so PATCH gets the key too.)
- `_mismatch_warning(headcount_id)` → `self.get_queryset().get(pk=id)` → `reconcile_section(school, grade, section)` → dict or None. **Side effect:** may update `count_validation` on the row *after* the response data was serialized, so the response's `count_validation` can be stale by one step.

`apps/observations`

- `selectors.session_tracker(unit, grade)` / `selectors.session_progress()` — see §26.4.
- `SessionTrackerView.get(request)` → `int()` params (defaults 1 and 6) → selector → filter to own school for school role → `Response(list(rows))`.
- `SessionProgressView.get(request)` → selector → filter for school role → list.

(These two views compare `request.user.role == "school"` with a string literal instead of the `SCHOOL` constant — equivalent, but inconsistent.)

`apps/teams/views.py`

`ClusterViewSet/TeamViewSet/StudentViewSet.get_queryset()` — preload + scope via `school_id`, `cluster__school_id`, `team__cluster__school_id`.

`apps/inquibuddy/signals.py`

`create_inquibuddy_submission(sender, instance, created, **kwargs)` — see §25.

`apps/inquibuddy/services.py` **— see §26.2**

Step-by-step for `get_or_create_individual_report(submission, user)`: 1. `submission.feedback_reports.filter(type="individual").order_by("-created_at").first()` → return if found. 2. `_report_context(submission)` → dict. 3. `render_team_feedback_pdf(**ctx)` → bytes. 4. `File.objects.create(file=ContentFile(bytes, name="InquiBuddy_<team>.pdf"), mime_type="application/pdf", ...)` → storage write. 5. `FeedbackReport.objects.create(submission, type="individual", file)`. 6. `record_audit("download_individual_report")` — note it is only recorded when the PDF is **first generated**, not on each download.

`apps/inquibuddy/tasks.py` — `generate_ai_feedback(self, submission_id)` **— see §17.4**

`apps/inquibuddy/views.py`

Method	Purpose	Notes
`get_queryset()`	`select_related("team__cluster__school__instance__partner")`, `prefetch_related("submission_files__file", "ai_feedback_questions", "feedback_reports")`, scoped	
`upload_photo` / `upload_audio`	`@action` POST detail → `_upload(kind)`	
`_upload(request, pk, kind)`	get_object → require `request.FILES["file"]` → service → re-fetch → serialize	Re-fetch needed because the prefetch cache is stale.
`generate_feedback(request, pk)`	require photo → `status=processing` → `.delay(id)`	Returns immediately.
`generate_feedback_bulk(request)`	`_require_section(request.data)` → scoped submissions with a photo (`.distinct()`) → processing + `.delay` each	Returns `{"queued": submissions.count()}` — a second query **after** status changes; still the same set since the filter doesn't involve status.
`download_report(request, pk)`	require evaluated → service → `FileResponse(as_attachment=True)`	GET with side effects (creates rows on first call).
`download_all_report(request)`	`_require_section(query_params)` → evaluated scoped submissions → service → FileResponse	New PDF + `File` each call.
`_require_section(params)` (staticmethod)	Return `(school, grade, section)` or raise 400	Values are strings; not type-validated (non-numeric `school` → ORM `ValueError` /500 or 400 depending on backend).

`apps/files`

- `upload_path(instance, filename)` → `f"{entity_type}/{entity_id or 'misc'}/{filename}"` — called by `FileField` when saving.
- `FileSerializer.get_url(obj)` → absolute URL if a request is in context.
- `FileViewSet.get_serializer_class()` → upload vs read serializer.
- `FileViewSet.perform_create(serializer)` → `request.FILES["file"]` (KeyError → 500 if missing but serializer would already have failed validation with 400 first) → save metadata.

`apps/reports` — see §26.3, §26.4

- `ReportsSummaryView.get(request)` → scoped `SchoolProgress` (`select_related("school")`) → `compute_kpis` unless school role → `{"kpis", "schools"}`.
- `tasks.reconcile_school_progress()` → `reconcile_all_schools()`.
- `Command.handle()` (reconcile_school_progress) → `reconcile_all_schools()` + success message.

`lib/gemini_client.py`

```
Function: generate_feedback_questions(*, photo_bytes, photo_mime, audio_bytes=None, audio_mime=None)
File: lib/gemini_client.py
Purpose: Ask Gemini for 5 bilingual reflection questions about a team's idea.
Called by: apps.inquibuddy.tasks.generate_ai_feedback
Calls: types.Part.from_bytes, types.Part.from_text, _client().models.generate_content, json.loads
Parameters: raw bytes + MIME types; audio optional (default MIME "audio/mp4")
Return value: list of up to 5 dicts {"question_en", "question_te"}
Database interaction: none
Possible exceptions: network/auth errors from google-genai; json.JSONDecodeError if the model returns non-JSON;
                     KeyError later in the task if keys are missing
Step-by-step: build parts [photo, (audio), prompt text with optional audio clause] → call model
              "gemini-2.5-flash" with response_mime_type="application/json" → parse → slice [:5]
```

`_client()` → `genai.Client(api_key=settings.GEMINI_API_KEY)` (new client per call).

`lib/pdf_builder.py`

- `render_team_feedback_pdf(*, school_name, partner_name, grade, section, team_code, sl_name, student_names, idea_photo_url, questions, generated_at)` → renders `inquibuddy/feedback_report.html` with `report={...}` (student names joined with ", ") → `_html_to_pdf`.
- `render_all_teams_feedback_pdf(team_reports)` → renders `feedback_report_all_teams.html` → `_html_to_pdf`.
- `_html_to_pdf(html)` → **lazy** `from weasyprint import HTML` (so Django boots on Windows without GTK) → `HTML(string=html).write_pdf(buf)` → bytes.

Observation: in `_report_page.html` the Telugu column header is written as `&#0c24;&#0c46;...` — decimal-style character references containing hex digits. Valid hex references need an `x` (`త`), so this header will not render as Telugu text. The question text itself comes from data and is unaffected.

---

31. Class Reference

```
Class: TimeStampedModel            File: apps/core/models.py      Parent: models.Model (abstract)
Purpose: created_at/updated_at on every table.   Used by: nearly all models.   Lifecycle: no table.

Class: AuditLog                    File: apps/core/models.py      Parent: models.Model
Purpose: audit trail.  Attributes: user, action, entity_type, entity_id, before, after, created_at
Used by: record_audit, AuditLogAdmin (read-only).

Class: SchoolScopedViewSetMixin    File: apps/core/mixins.py      Parent: object (mixin)
Purpose: role scoping + forced school on write.  Attributes: school_field="school"
Methods: get_queryset, get_serializer, perform_create
Used by: KitDeliveryViewSet, SLSelectionViewSet, StudentHeadcountViewSet, SessionObservationViewSet
Dependencies: scope_queryset_to_role, SCHOOL
Lifecycle: instantiated per request as part of the viewset (DRF creates a new view instance per request).

Class: IsAdmin / IsAdminOrIIFStaff / IsSchoolUser / IsAdminOrIIFStaffOrReadOnly
File: apps/core/permissions.py     Parent: rest_framework.permissions.BasePermission
Methods: has_permission(request, view)
Lifecycle: DRF instantiates each class in permission_classes per request (get_permissions()).

Class: User                        File: apps/accounts/models.py  Parent: AbstractUser
Attributes: role, school, display_name (+ AbstractUser fields); constants ADMIN, IIF_STAFF, SCHOOL, ROLE_CHOICES
Used by: everything (request.user), JWT, admin.

Class: ThinkAndMakeTokenObtainPairSerializer  File: apps/accounts/serializers.py
Parent: TokenObtainPairSerializer   Methods: get_token (classmethod)   Used by: LoginView

Class: UserSerializer / UserCreateSerializer   Parent: ModelSerializer   Used by: UserViewSet, MeView

Class: LoginView                   Parent: TokenObtainPairView    Attributes: serializer_class, throttle_scope="login"
Class: MeView                      Parent: APIView                Methods: get
Class: UserViewSet                 Parent: ModelViewSet           Attributes: queryset, permission_classes=[IsAdmin]
Methods: get_serializer_class, perform_create, perform_update
Class: UserAdmin                   Parent: DjangoUserAdmin + unfold ModelAdmin   (unfold forms for user/password)

Class: Partner, ProgramInstance    Parent: TimeStampedModel
Class: PartnerViewSet, ProgramInstanceViewSet   Parent: ModelViewSet

Class: School, SchoolGradeEnrollment, SchoolTeacher, SessionSchedule   Parent: TimeStampedModel
Class: SchoolSerializer, SchoolForm1SubmitSerializer, SchoolContactSubmitSerializer (ModelSerializer);
       ContactTeacherInputSerializer, ContactSessionScheduleInputSerializer (Serializer)
Class: SchoolViewSet               Parent: ModelViewSet   Actions: submit_form1, submit_form2, progress
Class: SchoolGradeEnrollmentViewSet / SchoolTeacherViewSet / SessionScheduleViewSet   Parent: ModelViewSet

Class: KitDelivery / KitDeliverySerializer / KitDeliveryViewSet(SchoolScopedViewSetMixin, ModelViewSet)
Class: SLSelection / SLSelectionSerializer / SLEntryInputSerializer / SLSelectionBulkSubmitSerializer /
       SLSelectionViewSet(SchoolScopedViewSetMixin, ModelViewSet) with action bulk_submit
Class: StudentHeadcount / StudentHeadcountSerializer /
       StudentHeadcountViewSet(SchoolScopedViewSetMixin, ModelViewSet) overriding create/update
Class: Cluster, Team, Student + serializers + ClusterViewSet/TeamViewSet/StudentViewSet (ModelViewSet)
Class: SessionObservation / SessionObservationSerializer /
       SessionObservationViewSet(SchoolScopedViewSetMixin, ModelViewSet); SessionTrackerView, SessionProgressView (APIView)

Class: InquibuddySubmission, SubmissionFile, AIFeedbackQuestion, FeedbackReport   Parent: TimeStampedModel
Class: InquibuddySubmissionSerializer (+ 3 nested)   Parent: ModelSerializer
Class: InquibuddySubmissionViewSet  Parent: ReadOnlyModelViewSet   Actions: 6 (see §30)
Class: InquibuddyConfig            Parent: AppConfig   Methods: ready() → imports signals

Class: File                        Parent: TimeStampedModel   Attributes: file, mime_type, size, uploaded_by,
                                   entity_type, entity_id, legacy_url
Class: FileSerializer, FileUploadSerializer   Parent: ModelSerializer
Class: FileViewSet                 Parent: CreateModelMixin, RetrieveModelMixin, GenericViewSet

Class: SchoolProgress              Parent: TimeStampedModel
Class: SchoolProgressSerializer    Parent: ModelSerializer
Class: ReportsSummaryView          Parent: APIView
Class: Command (reconcile_school_progress / import_legacy_sheets / import_master_users)
                                   Parent: django.core.management.base.BaseCommand   Method: handle()

Admin classes: every model is registered with an unfold ModelAdmin subclass; customised ones:
  AuditLogAdmin (list/filter, all read-only), UserAdmin, SchoolAdmin (list_display, search, filters),
  StudentHeadcountAdmin, InquibuddySubmissionAdmin, SchoolProgressAdmin.
```

32. Relevant concepts to learn

Concepts required to understand the school-scoping code

- **Object-level vs queryset-level authorization** — this project filters querysets instead of `has_object_permission`, so unauthorized rows simply don't exist from the user's point of view (404).
- **IDOR (Insecure Direct Object Reference)** — accessing someone else's object by guessing its id. Queryset scoping is the main defence.
- **Python MRO & mixins** — why `SchoolScopedViewSetMixin` must be listed *before* `ModelViewSet`, and how `super()` walks the chain.
- **DRF generic view hooks** — `get_queryset`, `get_serializer`, `perform_create`.

`ScopedRateThrottle`

- DRF throttling, rate limiting, token-bucket/sliding-window idea, **HTTP 429**, `Retry-After`, per-IP vs per-user identity, shared cache needed across processes.

`select_related()` / `prefetch_related()`

- **SQL JOIN**, the **N+1 query problem**, forward vs reverse relations, `Prefetch` objects, measuring with `django.db.connection.queries` or `assertNumQueries`.

`transaction.atomic()`

- **Database transactions**, **ACID** (Atomicity, Consistency, Isolation, Durability), **rollback** on exception, autocommit mode, savepoints for nested `atomic()`.

`UniqueConstraint` + `UniqueTogetherValidator`

- DB-level vs application-level uniqueness; race conditions (two concurrent requests can both pass the validator; the DB constraint then raises `IntegrityError`).

JWT (SimpleJWT)

- Header.payload.signature, HMAC-SHA256, claims (`exp`, `iat`, `jti`), stateless auth, access vs refresh tokens, rotation, blacklisting, why JWTs can't be revoked without server state.

Celery `@shared_task(bind=True, max_retries=2)`

- Message queues, producer/consumer, at-least-once delivery, idempotency, retries & backoff, why you pass IDs not objects (JSON serialization + stale data).

`crontab(hour=2, minute=0)`

- Cron syntax (minute, hour, day-of-month, month, day-of-week), time zones, single-scheduler requirement.

`post_save` signals

- Observer pattern, implicit coupling, what bypasses signals (`bulk_create`, `update()`).

`FileField` + django-storages

- Storage abstraction (`default_storage`), `upload_to` callables, S3 object keys, presigned URLs (AWS SigV4), path vs virtual-hosted addressing.

WeasyPrint

- HTML/CSS → PDF, text shaping (HarfBuzz) for complex scripts like Telugu, native library dependencies (Pango), `base_url` for resolving relative resources.

`update_or_create` / `get_or_create` / idempotent imports

- Upserts, natural keys vs surrogate keys, why `external_ref` makes re-runs safe.

Data migrations (`RunPython`)

- Historical models (`apps.get_model`), reversible migrations, the add-copy-remove-rename pattern for type changes.

33. How to rebuild this project from scratch

A roadmap adjusted to what this project actually contains.

1. **Environment** `python -m venv .venv` → activate → `pip install django djangorestframework djangorestframework-simplejwt django-cors-headers django-filter python-decouple dj-database-url psycopg2-binary`.
2. **Project** — `django-admin startproject config .` (creates `config/` + `manage.py`).
3. **Apps package** — `mkdir apps && touch apps/__init__.py`; for each app `python manage.py startapp core apps/core` (etc.) and set `name = "apps.core"` in each `apps.py`.
4. **Settings** — read secrets with decouple, `DATABASES` via `dj_database_url`, `INSTALLED_APPS`, `REST_FRAMEWORK` defaults (JWT auth, `IsAuthenticated`, `DjangoFilterBackend`, pagination), `TIME_ZONE="Asia/Kolkata"`, `USE_TZ=True`.
5. **Custom user FIRST** — `accounts.User(AbstractUser)` with `role`, `display_name`; set `AUTH_USER_MODEL="accounts.User"` **before** the first `migrate`. Add `school` FK once `schools` exists.
6. **Core** — `TimeStampedModel`, role constants, permission classes, `scope_queryset_to_role`, `AuditLog` + `record_audit`.
7. **PostgreSQL** — run locally or on Supabase; export `DATABASE_URL`.
8. **Models** in dependency order: programs → files → schools → kits / sl_selection / headcounts / observations / teams → inquibuddy → reports. Add unique constraints and `related_name`s.
9. **Migrations** — `makemigrations` → review → `migrate`.
10. **Serializers** — ModelSerializers for CRUD; separate input serializers for form submissions (Form 1, Form 2, SL bulk).
11. **Views** — ModelViewSets with `get_queryset()` scoping; `SchoolScopedViewSetMixin` for school-writable resources; custom `@action`s for submit flows; APIViews for read models.
12. **URLs** — one `DefaultRouter` per app, `config/urls.py` under `api/v1/`.
13. **Authentication** — SimpleJWT: `LoginView` (custom claims serializer), `TokenRefreshView`, `MeView`.
14. **Permissions** — role matrix (§13); write tests proving cross-school access fails.
15. **Throttling** — `ScopedRateThrottle` with a `login` scope (and a shared Redis cache — improvement).
16. **CORS** — `django-cors-headers`, middleware near the top, allowed origins from env.
17. **Redis** — run locally; broker URL in env.
18. **Celery** — `config/celery.py`, import in `config/__init__.py`, `@shared_task`s in `apps/*/tasks.py`; worker command.
19. **Celery Beat** — `CELERY_BEAT_SCHEDULE` with `crontab`; beat command.
20. **Storage** — `FileField` + `File` model; `django-storages` + `boto3` behind `USE_S3_STORAGE`; signed URLs.
21. **Integrations** — `lib/gemini_client.py` (`google-genai`), `lib/pdf_builder.py` (WeasyPrint + templates).
22. **Signals** — `post_save(Team)` → create submission; import in `AppConfig.ready()`.
23. **Services** — reconciliation service + task + management command.
24. **Admin** — register models (`django-unfold` theme optional; put `"unfold"` before `django.contrib.admin`).
25. **Import commands** — openpyxl-based idempotent import from the legacy xlsx.
26. **Tests** — `pytest`, `pytest-django`, `conftest.py` fixtures (`two_schools`, role clients with `force_authenticate`), mock `.delay`.
27. **Dockerize** *(not present in this repo — recommended)* — image with Pango/fonts for WeasyPrint; services: web (gunicorn), worker, beat, redis.
28. **Deploy** *(no config in this repo)* — see §38.

34. Development commands

All from `backend/`.

```
# 1. Virtual environment
python -m venv .venv
.venv/Scripts/activate            # Windows (Git Bash: source .venv/Scripts/activate)
source .venv/bin/activate         # macOS / Linux

# 2. Dependencies
pip install -r requirements.txt

# 3. Environment
cp .env.example .env              # defaults: SQLite + local ./media, no external services

# 4. Database
python manage.py makemigrations   # only after changing models
python manage.py migrate
python manage.py showmigrations   # see what's applied

# 5. Admin user
python manage.py createsuperuser  # then set role=admin in the admin if you'll use the API as admin

# 6. Run the API
python manage.py runserver        # http://127.0.0.1:8000/api/v1/  and /admin/

# 7. Data import (legacy)
python manage.py import_legacy_sheets --file "../docs/Datla-Think & Make _ 2026-2027.xlsx"
python manage.py import_master_users
python manage.py reconcile_school_progress

# 8. Redis (one option)
docker run -d --name redis -p 6379:6379 redis:7

# 9. Celery (the app module is "config")
celery -A config worker -l info
celery -A config worker -l info --pool=solo   # Windows: prefork pool isn't supported
celery -A config beat -l info                 # separate process; run exactly one

# 10. Tests
pytest
pytest apps/schools/tests.py -k form1 -v

# 11. Useful checks
python manage.py check
python manage.py check --deploy               # production-readiness warnings
python manage.py shell
```

Using PostgreSQL locally: export `DATABASE_URL=postgres://user:pass@localhost:5432/think_and_make` **in the shell** (dj-database-url reads the real environment, see §5.1), then `migrate`.

Note: `createsuperuser` does not prompt for `role` (it's required on the model but not in `REQUIRED_FIELDS`), so the superuser is created with `role=""`. Such a user can use the Django admin, but in the API they are neither admin nor school (they pass `IsAuthenticated`, fail `IsAdmin`, and see unscoped data because `scope_queryset_to_role` only restricts `school`). Set the role in the admin.

35. Testing

35.1 Setup

- **pytest + pytest-django** (`pytest.ini`: `DJANGO_SETTINGS_MODULE = config.settings`; files `tests.py`, `test_*.py`, `*_tests.py`).
- `@pytest.mark.django_db` gives each test a transactional test database.
- DRF's `APIClient` with `force_authenticate(user=...)` skips JWT and sets the user directly.
- `factory_boy` and `Faker` are installed but **not used**.
- Result at time of writing: **54 passed** (`pytest`, ~65 s).

35.2 Fixtures (`conftest.py`)

Fixture	Provides
`_isolated_media_root` (autouse)	`settings.MEDIA_ROOT = tmp_path` so uploads don't touch `./media`
`instance`	Partner "Datla" + ProgramInstance `DATLA-HYD-2026`
`two_schools`	School A (`DA`) and School B (`DB`)
`school_client`	`(APIClient, school_a)` authenticated as a school user of A
`admin_client`	APIClient as `role=admin`
`iif_staff_client`	APIClient as `role=iif_staff`

35.3 What is tested

File	#	Covers
`apps/schools/tests.py`	19	School scoping (list, cross-school PATCH), admin list, 401 anonymous, partner filter, Form 1 success/validation/lock/admin override/draft PATCH/lock on PATCH, grade enrollments not clobbered, Form 2 ordering, success, missing schedule, ≥1 teacher, lock, admin resubmit
`apps/kits/tests.py`	5	Auto-assign school, spoofing prevention, one delivery per school (school + IIF), admin override
`apps/sl_selection/tests.py`	6	Bulk create, duplicate section lock, admin repeat, ≥1 entry, auto-assign, spoofing
`apps/headcounts/tests.py`	3	Auto-assign, unique `(school, grade, section)`, different sections allowed
`apps/inquibuddy/tests.py`	3	Signal creates submission; upload → generate (with `patch(...delay)`); photo required
`apps/reports/tests.py`	20	form3/form4 %, zero denominator, mismatch flag + `count_validation` sync + clearing, section normalisation, `reconcile_section`, kit status cases (none/partial/complete/latest row), failing school skipped, summary for admin vs school, progress detail scoping + auto-create, `mismatch_warning` in headcount responses

Test types present: **API tests** (most), **service/unit tests** (reports), **permission/scoping tests**, a **signal test**. Model tests as such are absent (models are exercised through APIs).

35.4 Important functionality lacking tests

- Login, refresh, `/auth/me/`, custom JWT claims, **login throttling**.
- `UserViewSet` (admin-only access, password hashing, audit rows).
- `FileViewSet` (upload metadata, retrieval, **cross-school access**).
- Observations: CRUD, uniqueness, `session-tracker` / `session-progress` (incl. bad params).
- Teams/Clusters/Students endpoints and their scoping.
- InquiBuddy: `generate_ai_feedback` task body (success, retry, no photo), bulk generate, PDF report endpoints, `lib/gemini_client` parsing.
- Import commands (`import_legacy_sheets` regex parsers, `import_master_users`).
- Update/delete paths for school-role users on kits/SL/headcounts.
- Partners/instances write permissions.

36. Security Considerations

Based only on the code in this repository. **Nothing was exploited**; items are described at the level needed to fix them.

#	Area	Why it matters	Where	Current handling	Status
1	**Authentication**	Every API call must identify the user	`REST_FRAMEWORK` defaults	JWT required by default; anonymous → 401	Handled
2	**Authorization / permission boundaries**	Roles must not exceed their rights	`core/permissions.py`, per-view `permission_classes`	Role classes + scoped querysets; tested for schools/kits/SL/headcounts/reports	Mostly handled — see 3–5
3	**IDOR on files**	File ids are sequential integers; files include student/school photos and reports	`FileViewSet` (`queryset = File.objects.all()`)	Any authenticated user can retrieve any file's metadata + signed URL	**Missing** — scope by `uploaded_by`, or by the owning entity's school
4	**IDOR via FK references**	A client can point its record at someone else's file	`school_photo`, `delivery_proof_photo`, `acknowledgement_letter`, `teams_info_photo`, `student_database_file`	Serializers accept any existing `File` id	**Missing** — validate ownership
5	**Submission locks**	"Contact admin to make changes" implies immutability	kits, sl_selection	Only `create` is locked; PATCH/DELETE remain available to non-admins (SL rows can be deleted then re-submitted)	**Partial**
6	**Allow-by-default scoping**	New roles would see everything	`scope_queryset_to_role` returns unfiltered queryset for any non-school role (including a superuser created with empty `role`)	Only `school` is restricted	**Recommended:** explicit allow-list (admin, iif_staff) and deny otherwise
7	**CORS**	Wrong origins let other sites read API responses in a browser	`CORS_ALLOWED_ORIGINS`, `CORS_ALLOW_CREDENTIALS=True`	Explicit origin list from env	Handled; credentials flag unnecessary for header-based JWT
8	**CSRF**	Cookie-authenticated requests can be forged	Admin uses sessions + `CsrfViewMiddleware`; API uses Bearer headers (not sent automatically by browsers)	JWT-in-header is not CSRF-prone	Handled. If tokens are ever moved to cookies, CSRF protection must be added
9	**Secret management**	Leaked secrets = full compromise	`.env` git-ignored; `.env.example` has placeholders	Good	Handled
10	**Default credentials in source**	Passwords in a repo are readable by anyone with repo access and stay in git history	`apps/core/management/commands/import_master_users.py` contains every account's initial password (predictable pattern), and re-running the command **resets** all passwords to them	Present in source	**Risk** — move to a private file/secret store, force password change on first login, stop resetting on re-run
11	**Password handling**	Weak passwords + brute force	`set_password` (PBKDF2); `UserCreateSerializer` `min_length=8`	Hashing correct	**Partial** — `AUTH_PASSWORD_VALIDATORS` not applied in the API (call `validate_password()`); no password-change/reset endpoints
12	**JWT security**	Stolen tokens	HS256 with `SECRET_KEY`; 15 min access; 7 d refresh; rotation	Short access lifetime	**Partial** — blacklist app missing so rotated/stolen refresh tokens stay valid for 7 days; no logout; insecure default `SECRET_KEY` if env not set
13	**Rate limiting**	Brute-force login	`ScopedRateThrottle` `login: 10/min`	Present	**Partial** — per-process LocMem cache; proxy IP handling not configured
14	**File upload risks**	Malware, huge files, content-type spoofing, stored XSS if served inline	`FileViewSet`, InquiBuddy uploads	No size/type/extension checks; `mime_type` trusts client `content_type`; client-chosen `entity_type` becomes a folder	**Missing** — whitelist types/sizes, validate magic bytes, whitelist `entity_type`; serve with `Content-Disposition: attachment`
15	**SQL injection**	Arbitrary DB access	All queries use the ORM with parameters; no `raw()` / `extra()` /cursor SQL	ORM parameterisation	Handled
16	**Sensitive data exposure**	Student names, principal phone/email	Scoped serializers; `UserSerializer` excludes password	Scoped for schools; admin/IIF see all	Handled for schools. Note JWT payload exposes `role` / `school_id` (not secret)
17	**Debug mode**	Tracebacks leak settings/code	`DEBUG` default **True**	Must be overridden	**Risk** if env var forgotten — default should be False
18	**Allowed hosts**	Host-header poisoning	env CSV, default localhost	Explicit	Handled (set in prod)
19	**Storage access**	Bucket keys give full access	Keys only in env; `AWS_DEFAULT_ACL=None`	Private-by-default intent	Handled if bucket is private

#	Area	Why it matters	Where	Current handling	Status
20	**Signed URLs**	URLs are bearer capabilities	`AWS_QUERYSTRING_AUTH=True`, 600 s	Short expiry	Handled; combine with fix #3
21	**Transport / cookie security**	Tokens over plain HTTP	No `SECURE_*`, HSTS, secure-cookie settings	Relies on deployment	**Missing** — add for production
22	**Audit**	Accountability	`AuditLog` for user changes, AI jobs, report generation	Partial coverage (form submissions, deletes and file access aren't audited)	Partial
23	**Admin exposure**	Admin at `/admin/` with full data	Session auth, `is_staff` only for admin role	Standard	Consider IP restriction / 2FA in prod
24	**External AI**	Student data sent to a third party	`lib/gemini_client.py` sends idea photos/audio to Google	By design	Ensure consent/data-processing terms cover it

37. Performance considerations

Present optimisations

- `select_related` on almost every viewset queryset (school, instance, cluster→school, the long InquiBuddy chain).
- `prefetch_related` for nested serializers: schools (3 relations), clusters (`teams__students`), teams (`students`), InquiBuddy (files, questions, reports), `reconcile_all_schools` (6 relations).
- **Indexes:** automatic FK indexes; composite indexes on `AuditLog(entity_type, entity_id)`, `AuditLog(user, created_at)`, `File(entity_type, entity_id)`; unique constraints double as indexes (`School(instance, school_code)`, headcount, observation, cluster, team, enrollment).
- **Pagination:** 50 per page on all viewset lists.
- **Bulk operations:** `bulk_create` for teachers, schedules, SL rows, AI questions.
- `save(update_fields=...)` for small targeted updates.
- `.exists()` instead of `.count()` for existence checks.
- **Celery** offloads the slow Gemini call; **SchoolProgress** is a precomputed cache for the dashboard.
- `conn_max_age=600` persistent DB connections.

Caching

No Django cache is configured (`CACHES` absent → LocMem). Redis is used only by Celery.

Possible N+1 / heavy query spots (visible in code)

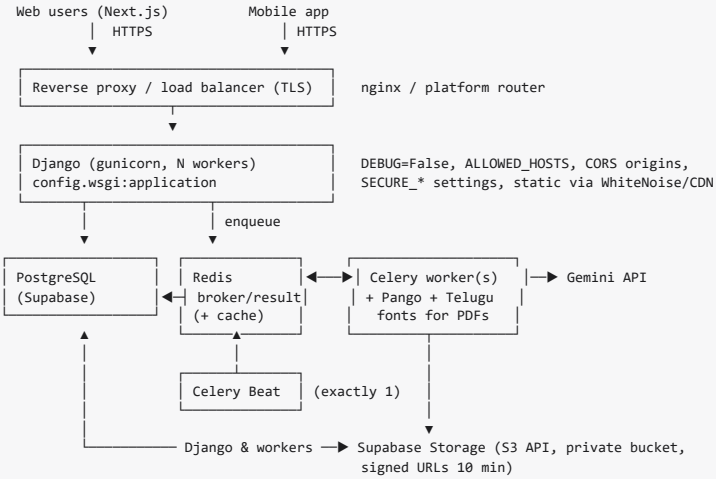
1. `_report_context()` runs separate queries per submission (`submission_files.filter(...)`, `team.students.values_list(...)`, `ai_feedback_questions.all().order_by(...)`) — calling `.filter()` / `.order_by()` on a relation bypasses prefetched data. For `report-all` with 30 teams that's ~90+ queries (plus PDF rendering time). Fix: use the prefetched lists or `Prefetch(...)` with `to_attr`.
2. `reconcile_school()` is called per school; `school.teachers.exists()`, `school.session_schedules.exists()`, `school.kit_deliveries.first()` and `school.sl_selections.values_list(...)` may issue their own queries despite the prefetch (`.first()` / `.values_list()` build new querysets). With hundreds of schools this is a few queries per school — acceptable nightly, but worth measuring.
3. `check_headcount_mismatch()` may `save()` each headcount individually.
4. `compute_kpis()` issues ~5 `COUNT` queries — fine.
5. `SessionTrackerView` / `SessionProgressView` / `ReportsSummaryView` are unpaginated — response size grows with data.
6. `generate_feedback_bulk` saves and enqueues per submission in a loop (fine at section scale).
7. `evaluation_count += 1` in Python (race-prone rather than slow) — use `F("evaluation_count") + 1`.
8. `FileResponse` **streams PDFs through Django workers** — occupies a worker for large files; redirecting to a signed URL would offload it.
9. Every `SchoolSerializer` list page serializes 3 nested lists per school — prefetch keeps this to 4 queries per page.

38. Deployment architecture

38.1 What exists in the repository

No deployment configuration exists: no Dockerfile, docker-compose, Procfile, gunicorn/uvicorn dependency, nginx config, CI workflow, or `STATIC_ROOT`. What *is* prepared for deployment: - `DATABASE_URL` → Supabase Postgres (`.env.example` comment). - `USE_S3_STORAGE` → Supabase Storage. - Redis URLs for Celery. - Notes in `backend/README.md` and `lib/pdf_builder.py`: on Linux install `libpango-1.0-0 libpangoft2-1.0-0` and the **Noto Sans Telugu** font for correct Telugu PDFs.

38.2 Recommended production architecture (not implemented)



Steps typically needed: add `gunicorn`, set `STATIC_ROOT` + `collectstatic` (or WhiteNoise), run `migrate` on release, run `check --deploy`, configure `CACHES` to Redis, install `token_blacklist`, build a container image that includes Pango and fonts, run web/worker/beat as separate processes, monitor Celery queues and the nightly job.

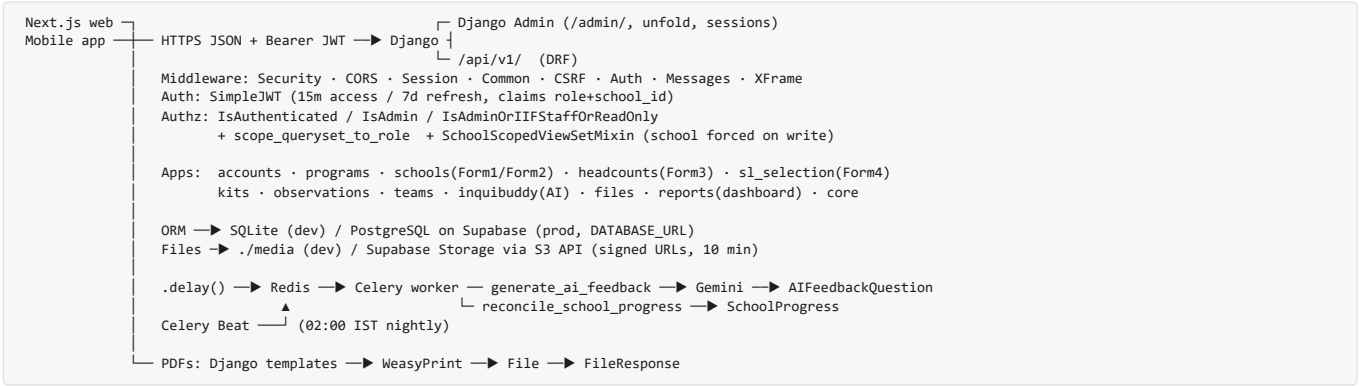
39. Glossary

Term	Meaning
API	Application Programming Interface — here, HTTP endpoints clients call to read/write data.
REST	Style of API using resources (URLs) and HTTP verbs (GET read, POST create, PUT/PATCH update, DELETE remove).
Django	Python web framework: ORM, URL routing, admin, auth, migrations.
DRF	Django REST Framework: serializers, viewsets, routers, authentication, permissions, throttling for building APIs.
ORM	Object-Relational Mapper — Python classes/methods instead of hand-written SQL.
Model	A Python class mapped to a database table.
QuerySet	A lazy, chainable database query over a model.
ForeignKey	A many-to-one link between tables (e.g. many kit deliveries → one school).
OneToOneField	A unique FK (e.g. one submission per team).
Migration	A versioned file that changes the database schema to match models.
Serializer	Converts model instances ⇄ JSON and validates input.
Serializer validation	Field checks → `validate_<field>` → `validate()` → errors become HTTP 400.
ViewSet	A class grouping list/create/retrieve/update/delete for one resource, wired to URLs by a router.
Router	Generates URL patterns for viewsets.
Middleware	Code that wraps every request/response (CORS, sessions, CSRF…).
Authentication	Establishing who the user is.
Authorization	Deciding what that user may do.
JWT	JSON Web Token — signed token carrying claims (user id, role, expiry).
Access / refresh token	Short-lived token for requests / long-lived token to obtain new access tokens.
CORS	Browser mechanism controlling which other origins may read an API's responses.
CSRF	Cross-Site Request Forgery — tricking a browser into sending an authenticated request with its cookies.
Throttling	Limiting request rate; exceeding it returns HTTP 429.
IDOR	Accessing another user's object by changing an id in the request.
Celery	Distributed task queue for running Python functions in background workers.
Celery Beat	Scheduler that sends periodic tasks to the queue.
Broker	Message queue carrying tasks (Redis here).
Redis	In-memory data store, used here as Celery broker/result backend.
PostgreSQL	Relational database used in production (hosted by Supabase).
SQLite	File-based database used by default in local development.
S3	Amazon's object storage API — many providers (incl. Supabase) implement it.
Supabase Storage	Supabase's file storage, reachable through an S3-compatible endpoint.
Bucket	Top-level container for stored objects.
Signed URL	Time-limited URL whose signature grants access to a private object.
Signal	Django hook that runs a function when something happens (e.g. after a model is saved).
Service / Selector	Project convention: write-side business logic / read-side queries, kept out of views.
Idempotent	Safe to run multiple times with the same end result (reconciliation, imports).
WeasyPrint	Library that renders HTML/CSS to PDF.
Gemini	Google's generative AI model family, called via `google-genai`.
Unfold	Third-party theme/extension for the Django admin.

40. Recommended Learning Order

1. **Python** essentials used here: classes & inheritance, `super()` /MRO (mixins), decorators (`@action`, `@shared_task`, `@receiver`), keyword-only args (`*`), f-strings, comprehensions, walrus operator (`reconcile_school`), type hints.
2. **HTTP & REST**: methods, status codes (200/201/400/401/403/404/429/500), headers (`Authorization`, `Content-Type`), JSON, multipart uploads.
3. **Django fundamentals**: project vs app, settings, `INSTALLED_APPS`, URLconf, `manage.py`, admin, templates (for PDFs), management commands.
4. **Django models & ORM**: fields, FK/OneToOne, `on_delete`, `related_name`, constraints, QuerySets, `select_related` / `prefetch_related`, `get_or_create` / `update_or_create`, transactions.
5. **Migrations**: `makemigrations` / `migrate`, dependencies, data migrations (read `schools/0002_form1_fields.py`).
6. **Django REST Framework**: serializers & validation, ModelViewSet, routers, `@action`, pagination, django-filter.
7. **Authentication with SimpleJWT**: tokens, claims, refresh/rotation (read `accounts/`).
8. **Permissions & multi-tenant scoping**: permission classes, `scope_queryset_to_role`, `SchoolScopedViewSetMixin` (read `core/` + the kits tests).
9. **Testing with pytest-django**: fixtures, `APIClient`, `force_authenticate`, mocking.
10. **PostgreSQL & dj-database-url**: connection strings, running migrations against Postgres.
11. **Redis** basics.
12. **Celery**: tasks, workers, retries, Beat (read `inquibuddy/tasks.py`, `reports/tasks.py`, `reports/services.py`).
13. **File storage**: `FileField`, django-storages, S3 API, signed URLs.
14. **Integrations**: Gemini SDK, WeasyPrint/HTML-to-PDF.
15. **Security hardening & deployment**: `check --deploy`, HTTPS settings, gunicorn, Docker, process layout.

41. Backend in One Picture



42. If I had to explain this project in an interview

> "Think & Make is a school innovation program that used to run entirely on Google Forms and Sheets. I worked on the Django backend that replaced it. It's a Django 5 and Django REST Framework API under `/api/v1/`, serving both a Next.js website and a React Native app from one codebase.
>
> The domain is organised as one Django app per entity family: programs and partners at the top, then schools, and under each school the program's forms — enrollment, contact info, student headcounts, student-leader selection, kit deliveries and session observations — plus a cluster/team/student hierarchy. A shared `File` model abstracts storage: local disk in development and Supabase Storage through the S3-compatible API in production, with ten-minute signed URLs.
>
> Authentication is JWT via SimpleJWT, with short-lived access tokens, rotating refresh tokens, and a throttled login endpoint. There are three roles — admin, IIF staff and school. The key security design is that school accounts are scoped server-side: every queryset goes through a single `scope_queryset_to_role` function, and school-writable resources use a mixin that overwrites the `school` field in the incoming data *before* validation. That detail matters because DRF's `UniqueTogetherValidator` runs during validation, so setting the school later in `perform_create` would be too late. The result is that a school can never read or write another school's data, regardless of what the client sends, and that's covered by tests.
>
> Business rules like 'Form 1 must be submitted before Form 2' or 'once submitted, only an admin can change it' live in custom viewset actions with dedicated input serializers, so the base model stays permissive for draft data while submissions are strict.
>
> There are two background workloads on Celery with Redis as the broker. The first is InquiBuddy: a team uploads a photo of their idea, the API sets the status to 'processing' and enqueues a task that sends the photo and optional audio to Google Gemini and stores five English and Telugu reflection questions; the client polls status, then downloads a PDF rendered with WeasyPrint, chosen because it shapes Telugu correctly. The second is a nightly Celery Beat job that rebuilds a `SchoolProgress` table for the dashboard. That table is a derived cache — every value is recomputable from the source tables, so the job is idempotent and safe to rerun, and the same service is also called synchronously after a headcount submit to show an instant mismatch warning.
>
> Finally, there are idempotent management commands to import the legacy spreadsheet using the original submission IDs, and a pytest suite of about fifty tests focused on role scoping, form locking and the reconciliation logic. If I kept going, my next steps would be scoping the file endpoint, enabling the token blacklist, adding a shared cache for throttling, making multi-step writes atomic, and adding deployment configuration."

43. Questions I should be able to answer

Beginner

1. What framework and major libraries does the backend use? Django 5.2, Django REST Framework 3.18, SimpleJWT, django-cors-headers, django-filter, python-decouple, dj-database-url, Celery 5 + Redis, django-storages/boto3, django-unfold, WeasyPrint, google-genai, openpyxl, pytest-django.

2. What is the base URL of the API? `/api/v1/`. The admin is at `/admin/`.

3. What database is used locally and in production? SQLite (`backend/db.sqlite3`) by default; PostgreSQL on Supabase when `DATABASE_URL` is set in the environment.

4. What is `config/` and why is it named that way? The Django project package holding settings, root URLs, WSGI/ASGI and Celery setup. Naming it `config` makes its role obvious.

5. Why are the apps inside an `apps/` folder? To separate project apps from configuration and libraries. That's why `AppConfig.name` is `"apps.<name>"`.

6. What does `TimeStampedModel` do? It's an abstract base adding `created_at` and `updated_at` to models without creating its own table.

7. What are the three user roles? `admin`, `iif_staff`, `school`.

8. How does a client log in? `POST /api/v1/auth/login/` with username and password; it receives `access` and `refresh` JWTs.

9. How long does an access token last? 15 minutes. The refresh token lasts 7 days.

10. How does the client send the token? `Authorization: Bearer <access token>` header.

11. What does `GET /auth/me/` return? The current user serialized with `UserSerializer`.

12. What is the default page size? 50 items (`PageNumberPagination`).

13. How do you filter schools by partner? `GET /api/v1/schools/?instance__partner=<partner id>`.

14. Where are uploaded files stored in development? In `backend/media/`, served at `/media/` only when `DEBUG` is on and S3 is off.

15. What command runs the tests? `pytest` from `backend/`.

16. How do you run the Celery worker? `celery -A config worker -l info` (add `--pool=solo` on Windows).

17. What is the `external_ref` field for? It stores the original Google Form Submission ID so the import command can upsert rows without duplicating them.

18. Which environment variable switches to Supabase Storage? `USE_S3_STORAGE=True`, plus the `SUPABASE_STORAGE_*` credentials.

Intermediate

19. Why is `AUTH_USER_MODEL` set, and when must it be set? To use `accounts.User` with `role` / `school`. It must be set before the first migration, because changing it later breaks existing FKs and migrations.

20. What extra claims are in the JWT, and does the server trust them? `role` and `school_id`, added in `ThinkAndMakeTokenObtainPairSerializer.get_token`. The server doesn't use them for authorization; it reloads the user from the database.

21. Why does an unauthenticated request get 401 and not 403? `JWTAuthentication` provides an `authenticate_header`, so DRF converts `NotAuthenticated` into 401 with `WWW-Authenticate`.

22. What does `IsAdminOrIIFStaffOrReadOnly` allow? Any authenticated user may use safe methods (GET/HEAD/OPTIONS). Only admin or IIF staff may write.

23. Can a school user submit Form 1? No. `SchoolViewSet` uses `IsAdminOrIIFStaffOrReadOnly`, so POST to `submit-form1` returns 403 for school users. The tests submit as IIF staff.

24. What happens when a school user requests another school's record by id? 404, because `get_queryset()` filters the row out before `get_object()` looks it up.

25. What does `scope_queryset_to_role` do for admin and IIF staff? It returns the queryset unfiltered, so they see everything. Per-instance assignment isn't implemented.

26. Why does `SchoolViewSet` pass `school_field="id"`? Because the model being scoped *is* the school, so it filters `id == user.school_id`.

27. How does Form 1 store the chosen grades? As a CSV string in `grades_offered`, with `total_sections = len(grades)`. It also calls `get_or_create` for one `SchoolGradeEnrollment` per grade without overwriting existing counts.

28. What does Form 2 validate across fields? That every grade in `grades_offered` has a session schedule (`SchoolContactSubmitSerializer.validate`) and that there's at least one teacher.

29. Why are `teachers` and `session_schedules` popped from `validated_data` in `submit_form2`? They aren't model fields on `School`. The view uses them to replace child rows, and `ModelSerializer.update()` doesn't handle nested writes.

30. What prevents a second kit delivery? `KitDeliveryViewSet.perform_create` checks `.exists()` for the school and raises 403 for non-admins.

31. What does `sl-selections/bulk-submit/` do differently from a normal create? It validates one grade/section plus a list of entries, then creates all rows with a single `bulk_create`. School users' `school` comes from the account, not the body, and non-admins can't resubmit a section.

32. What is `mismatch_warning` in the headcount response? The result of `reconcile_section()`: a dict if `total_sl` differs from the number of SL Selection rows for that grade/section, otherwise `null`. That includes when Form 4 hasn't started.

33. How are the session tracker and progress built? From plain queries in `observations/selectors.py`: `values()` for the tracker and `values().annotate(Count("id"))` for progress. There are no stored pivot tables.

34. What is `SchoolProgress` and why can it be rebuilt at any time? A per-school dashboard cache. Every field is derived from source tables, so `reconcile_school` is idempotent.

35. How is `form3_pct` calculated? `min(round(number_of_headcounts / sum(grade_enrollments.total_sections) * 100), 100)`. It's 0 if the denominator is 0.

36. How is `kit_status` decided? From the latest `KitDelivery` (ordering `-date_of_delivery`) compared with the grades in `grades_offered`. None delivered → `not_started`, some → `in_progress`, all → `complete`.

37. What happens when a `Team` is created? A `post_save` receiver creates its `InquibuddySubmission` with status `pending`. It's registered in `InquibuddyConfig.ready()`.

38. Why does `_upload` re-fetch the submission after attaching a file? The `prefetch_related` cache from `get_object()` was loaded before the new file existed, so serializing the old instance would miss it.

39. What does `.delay()` do? It serializes the arguments as JSON and publishes a task message to the Redis broker. A worker executes the task later. It's shorthand for `apply_async(args=...)`.

40. What is the Celery Beat schedule? `reconcile-school-progress-nightly` runs `apps.reports.tasks.reconcile_school_progress` with `crontab(hour=2, minute=0)`: daily at 02:00 Asia/Kolkata.

41. What's the difference between Celery worker and Beat? The worker executes tasks. Beat only sends scheduled task messages, and exactly one Beat should run.

42. What does `AWS_QUERYSTRING_EXPIRE = 600` mean? Signed file URLs expire after 10 minutes.

43. Why is `AWS_S3_ADDRESSING_STYLE = "path"`? Supabase's S3 endpoint expects `endpoint/bucket/key` URLs rather than bucket subdomains.

Advanced

44. Why does `SchoolScopedViewSetMixin` inject the school in `get_serializer()` instead of only in `perform_create()`? DRF's auto-generated `UniqueTogetherValidator` (for constraints that include `school`) needs `school` in the input and runs during `is_valid()`, which is before `perform_create`. Injecting early lets school clients omit the field and ensures the uniqueness check runs against the correct school.

45. Why does the mixin also pass `school=` to `serializer.save()`? Defence in depth. `save()` kwargs override `validated_data`, so ownership is guaranteed even if injection were bypassed.

46. Name an authorization gap in the current code. `FileViewSet.retrieve` isn't scoped, so any authenticated user can fetch any file's metadata and signed URL by id (IDOR). File FK fields also aren't ownership-validated. Kit/SL locks only guard create, not update/delete.

47. Why is `BLACKLIST_AFTER_ROTATION=True` ineffective here? `rest_framework_simplejwt.token_blacklist` isn't in `INSTALLED_APPS`, so SimpleJWT can't record blacklisted tokens. Old refresh tokens remain valid until they expire.

48. Why might the login throttle be weaker in production than it looks? It uses Django's default per-process LocMem cache, so each gunicorn worker has its own counter. Behind a proxy without `NUM_PROXIES`, all clients may share one IP.

49. What can leave an InquiBuddy submission stuck in `processing`? The task returning early when no photo exists, exhausting its two retries, or no worker running. Nothing resets the status on final failure.

50. What consistency risk exists in `submit_form2`? It updates the school, deletes children and bulk-creates new ones without `transaction.atomic()`. A failure mid-way can leave the form marked submitted with missing teachers or schedules.

51. Why is `evaluation_count += 1` a potential race condition, and how would you fix it? It's a read-modify-write in Python, so two concurrent task runs could both read N and write N+1. `F("evaluation_count") + 1` makes it an atomic SQL update.

52. How is Telugu rendered correctly in PDFs, and what deployment requirement follows? WeasyPrint shapes text via Pango/HarfBuzz. Servers need Pango libraries and a Telugu font (e.g. Noto Sans Telugu). WeasyPrint is imported lazily so Django still boots on Windows without GTK.

53. Why does `generate_feedback_bulk` use `.distinct()`? Filtering on `submission_files__kind` joins to `SubmissionFile`, which could duplicate a submission. `distinct()` removes duplicates.

54. Why is `DATABASE_URL` in `.env` not picked up automatically? `dj_database_url.config()` reads `os.environ` directly, while python-decouple reads `.env` without exporting it into `os.environ`.

55. How does migration `schools.0002` change `smart_board` from boolean to a choice field without losing data? It adds `smart_board_tmp`, copies values with `RunPython` using the historical model (True→"yes", False→"no", NULL→""), removes the old column, then renames the temp column. It also has a reverse function.

56. Why does `reconcile_all_schools` catch every exception per school? So one bad school doesn't abort the nightly run for all the others. The failure is logged and the next run retries naturally (tested).

57. How would you make `GET /files/{id}/` safe? Override `get_queryset()` to restrict school users to files they uploaded or files linked to their school's records (via `entity_type` / `entity_id` or reverse lookups). Also validate FK file ownership in serializers.

58. Why does `ReportsSummaryView` return `kpis: null` for school users? KPIs aggregate across all schools, which a school can't see. The aggregate would be meaningless and would leak cross-school information.

59. What does `import_master_users` do on a re-run, and why is that risky? It upserts users and resets every password to the value hard-coded in source. Anyone with repository access knows the initial credentials, and a re-run silently undoes password changes.

60. How would you scale the InquiBuddy workload? Run more Celery workers (or a dedicated queue via `apply_async(queue=...)`), keep IDs as task arguments, make the task idempotent, and add rate-limit/backoff for the Gemini API. Monitor queue depth in Redis and add a failed status for visibility.

End of document.